

Entornos de desarrollo

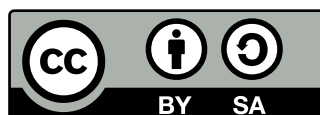
**Libro de texto para el módulo
“Entornos de desarrollo (0487)”
de Formación Profesional**

José J. Durán

Para acceder a los contenidos de este libro, y otros títulos, visita:
<https://github.com/cavefish-dev/libros>

Edición: 27 de abril de 2026

Esta obra está bajo una licencia Creative Commons
«Atribución-CompartirIgual 4.0 Internacional».



Índice general

A Introducción

1	Sobre este libro	11
1.1	Estructura del libro	11
1.2	Elementos del libro	12
2	Resultados de Aprendizaje y contenidos del libro	15

B Contenidos

3	Desarrollo de software	19
3.1	Concepto de programa informático	19
3.2	Código fuente, objeto y ejecutable. Tecnologías de virtualización.	20
3.2.1	Código fuente	20
3.2.2	Código objeto	21
3.2.3	Código ejecutable	21
3.2.4	Tecnologías de virtualización	21
3.3	Tipos de lenguajes de programación. Paradigmas.	23
3.3.1	Programación estructurada	23
3.3.2	Programación orientada a objetos	24
3.3.3	Programación funcional	24
3.3.4	Programación lógica	24
3.4	Características de los lenguajes más difundidos	25
3.4.1	Python	25
3.4.2	Java	26
3.4.3	C++	26
3.4.4	JavaScript	26
3.5	Fases del desarrollo de una aplicación	27
3.5.1	Análisis de requisitos	27

3.5.2	Diseño de la arquitectura	27
3.5.3	Codificación	28
3.5.4	Pruebas	28
3.5.5	Documentación	28
3.5.6	Explotación (Despliegue)	28
3.5.7	Mantenimiento	29
3.6	Proceso de obtención de código ejecutable a partir del código fuente	29
3.7	Metodologías ágiles y tradicionales	30
3.7.1	Agile	31
3.7.2	Scrum	31
3.7.3	Kanban	31
3.7.4	Extreme Programming (XP)	31
3.7.5	Desarrollo en cascada	31
3.7.6	Desarrollo iterativo	32
4	Instalación y uso de entornos de desarrollo	35
4.1	Funciones de un entorno de desarrollo	35
4.2	Instalación de un entorno de desarrollo	36
4.3	Uso básico de un entorno de desarrollo	38
4.4	Personalización del entorno de desarrollo	40
4.5	Edición de programas	43
4.6	Generación de ejecutables en distintos entornos	44
4.6.1	Automatización en el entorno	46
4.7	Herramientas y automatización	46
4.7.1	Ejemplos de herramientas de automatización	47
4.7.2	Integración en el entorno de desarrollo	47
5	Diseño y realización de pruebas	49
5.1	Planificación de Pruebas	49
5.2	Tipos de pruebas: Funcionales, estructurales y regresión, entre otras	50
5.3	Procedimientos y casos de prueba	51
5.4	Pruebas de Código: Cubrimiento, valores límite y clases de equi- valencia, entre otras	52
5.5	Pruebas unitarias; herramientas de automatización	53
5.6	Documentación de las incidencias	56
5.7	Dobles de prueba. Tipos. Características	57
6	Optimización y documentación	63
6.1	Refactorización	63
6.1.1	Concepto de refactorización	63

6.1.2	Limitaciones de la refactorización	64
6.1.3	Patrones de refactorización más usuales	64
6.1.4	Otras técnicas de refactorización	67
6.1.5	Refactorización y pruebas	69
6.1.6	Herramientas de ayuda a la refactorización	70
6.2	Analizadores de código	71
6.2.1	Ejemplo práctico de análisis de código	71
6.2.2	Normas de estilo en el código fuente	72
6.3	Control de versiones	74
6.3.1	Estructura de las herramientas de control de versiones	74
7	Herramientas de control de versiones y documentación	77
7.1	Repositorios remotos	77
7.1.1	Subversion (SVN)	77
7.1.2	Git	78
7.1.3	Uso integrado en el entorno de desarrollo	82
7.2	Integración continua. Herramientas	83
7.2.1	GitHub Actions	83
7.2.2	Jenkins	83
7.2.3	Otras herramientas	84
7.3	Documentación y observabilidad	85
7.3.1	Uso de comentarios	85
7.3.2	Herramientas de documentación	85
7.3.3	Distribución de la documentación	86
7.3.4	Buenas prácticas	87
7.3.5	Observabilidad y herramientas	88
8	Elaboración de diagramas de clases	93
8.1	Clases	93
8.2	Objetos	94
8.3	Relaciones	95
8.3.1	Recomendaciones para la elaboración de diagramas de clases	99
8.4	Herramientas	100
8.5	Generación automática de código. Ingeniería inversa.	101
8.5.1	Generación automática de código	101
8.5.2	Ingeniería inversa	104
8.5.3	Limitaciones	105
9	Elaboración de diagramas de comportamiento	107
9.1	Diagrama de casos de uso	108
9.1.1	Caso de uso	108

- 9.1.2 Actores 108
 - 9.1.3 Escenario 109
 - 9.1.4 Relación de comunicación 109
- 9.2 Diagrama de secuencia 111
 - 9.2.1 Línea de vida de un objeto 111
 - 9.2.2 Activación 112
 - 9.2.3 Envío de mensajes 112
- 9.3 Diagrama de comunicación. Objetos, mensajes 114
 - 9.3.1 Objetos y enlaces 114
 - 9.3.2 Mensajes y numeración de secuencia 114
- 9.4 Diagrama de actividad 115
 - 9.4.1 Nodo de inicio y fin 115
 - 9.4.2 Acción 116
 - 9.4.3 Decisión 116
 - 9.4.4 Bifurcación y unión 116
- 9.5 Diagramas de estados 117
 - 9.5.1 Estado 118
 - 9.5.2 Transición 118
 - 9.5.3 Evento 118
 - 9.5.4 Estado inicial y final 118

C Contenidos avanzados

- 10 TDD: Desarrollo guiado por pruebas 125
 - 10.1 El problema 125
 - 10.2 Iteración 1: el primer test rojo 126
 - 10.3 Iteración 2: un paréntesis sin cerrar 127
 - 10.4 Iteración 3: el primer par y el paso Refactor 128
 - 10.5 Iteración 4: anidamiento y nombres expresivos 129
 - 10.6 Iteración 5: corchetes y llaves 129
 - 10.7 Iteración 6: tipos mezclados incorrectamente 131
 - 10.8 Código final 131
 - 10.9 Ejercicios 133
- 11 Uso de inteligencia artificial generativa 137
 - 11.1 Cómo funciona un modelo de lenguaje 137
 - 11.2 Herramientas de IA para el desarrollo 138
 - 11.2.1 GitHub Copilot 138
 - 11.2.2 Cursor 139
 - 11.2.3 Claude 139

11.2.4	Zed	140
11.3	El marco de las 4D	140
11.3.1	Delegación (<i>Delegation</i>)	140
11.3.2	Descripción (<i>Description</i>)	141
11.3.3	Discernimiento (<i>Discernment</i>)	142
11.3.4	Diligencia (<i>Diligence</i>)	142
11.4	La IA en el flujo de desarrollo	143
11.4.1	Escribir código nuevo	143
11.4.2	Revisar y mejorar código	144
11.4.3	Escribir tests	144
11.4.4	Documentar	145
11.5	Limitaciones y riesgos	145
11.6	Ejercicios	147

D Ejercicios

12	Ejercicios de autoevaluación	153
12.1	Desarrollo de software	153
12.2	Instalación y uso de entornos de desarrollo	154
12.3	Diseño y realización de pruebas	154
12.4	Optimización y documentación	155
12.5	Herramientas de control de versiones	156
12.6	Elaboración de diagramas de clases	156
12.7	Elaboración de diagramas de comportamiento	157
13	Ejercicios prácticos	165
13.1	Desarrollo de software	165
13.2	Instalación y uso de entornos de desarrollo	165
13.3	Pruebas unitarias	166
13.4	Optimización y documentación	167
13.5	Control de versiones	168
13.6	Diagramas de clases	168
13.7	Diagramas de comportamiento	169



Introducción

Sobre este libro

Este libro es un texto de apoyo para el módulo *Entornos de Desarrollo (0487)* de los ciclos formativos de grado superior en Desarrollo de Aplicaciones Multiplataforma (DAM) y Desarrollo de Aplicaciones Web (DAW). Cubre los contenidos del currículo oficial de forma progresiva y los amplía con dos áreas de especial relevancia en el desarrollo profesional actual: el desarrollo guiado por pruebas (TDD) y el uso de herramientas de inteligencia artificial generativa como asistentes en el flujo de trabajo del programador.

Los ejemplos de código están escritos en Java y los diagramas se generan con PlantUML. El libro puede leerse de forma secuencial a lo largo del curso, aunque cada capítulo es suficientemente autónomo para consultarse de forma independiente.

1.1. Estructura del libro

La parte A (*Introducción*) presenta el libro, sus elementos y el mapeado de los Resultados de Aprendizaje del módulo a los capítulos correspondientes.

La parte B (*Contenidos*) desarrolla los temas principales del módulo: el proceso de desarrollo de software, los entornos de desarrollo integrados (IntelliJ IDEA y VS Code), las pruebas unitarias con JUnit 5, la optimización y documentación del código, el control de versiones con Git, y la elaboración de diagramas UML de clases y de comportamiento con PlantUML.

La parte C (*Contenidos avanzados*) profundiza en el desarrollo guiado por pruebas (TDD) y en el uso de herramientas de inteligencia artificial generativa —Copilot, Cursor y Claude— como asistentes en el flujo de trabajo del programador.

La parte D (*Ejercicios*) reúne los ejercicios prácticos organizados por capítulo para que el lector pueda consolidar los conocimientos de forma autónoma.

1.2. Elementos del libro

Consejo



Los recuadros de esta sección se muestran en color en la versión digital del libro. En la versión impresa en blanco y negro se distinguen por su borde y sombreado.

A lo largo del libro se utilizan los siguientes elementos para organizar la información:

Bloques de código. Los ejemplos de código fuente aparecen con resaltado sintáctico. El lenguaje principal es Java; algunos capítulos incluyen también YAML o texto plano para prompts de inteligencia artificial.

```
1 public class Ejemplo {  
2     public static void main(String[] args) {  
3         System.out.println("¡Hola, mundo!");  
4     }  
5 }
```

Consejos. Sugerencias prácticas y buenas prácticas que complementan el texto principal.

Consejo



Leer los ejercicios de cada capítulo antes de empezar el texto ayuda a entender qué conceptos se van a aplicar.

Notas importantes. Aspectos críticos, errores frecuentes o información que conviene tener especialmente presente.

Importante



Algunos conceptos de este libro requieren conocimientos previos de programación en Java. Si no tienes esa base, es recomendable repasar los fundamentos antes de continuar.

Ejercicios. Actividades prácticas numeradas automáticamente. Cada capítulo incluye ejercicios al final para aplicar lo aprendido.

Ejercicio 1.1



Explora el entorno de desarrollo que utilizarás durante el curso e identifica las principales zonas de la interfaz: editor, árbol de proyecto, terminal integrado y panel de ejecución.

Chuleterios. Al final de cada capítulo relevante se incluye una tabla de referencia rápida con los comandos, atajos o sintaxis más importantes.

Elemento	Descripción
<code>git status</code>	Muestra el estado del repositorio
<code>git add .</code>	Añade todos los cambios al área de staging
<code>git commit -m «msg»</code>	Crea un commit con el mensaje indicado

Resultados de Aprendizaje y contenidos del libro

El módulo de **Entornos de Desarrollo (0487)** de los ciclos formativos DAM y DAW define 6 Resultados de Aprendizaje (RA) en los Reales Decretos 450/2010 y 686/2010. La siguiente tabla muestra qué capítulo de este libro desarrolla cada RA.

RA	Descripción	Capítulo
RA1	Reconoce los elementos y herramientas que intervienen en el desarrollo de un programa informático, analizando sus características y las fases en las que actúan.	Cap. 3
RA2	Evalúa entornos integrados de desarrollo analizando sus características para editar código fuente y generar ejecutables.	Cap. 4
RA3	Verifica el funcionamiento de programas diseñando y realizando pruebas.	Cap. 5
RA4	Optimiza código empleando las herramientas disponibles en el entorno de desarrollo, incluyendo el control de versiones.	Caps. 6–7
RA5	Genera diagramas de clases valorando su importancia en el desarrollo de aplicaciones y empleando herramientas específicas.	Cap. 8
RA6	Genera diagramas de comportamiento valorando su importancia en el desarrollo de aplicaciones y empleando herramientas específicas.	Cap. 9

Cuadro 2.1: Mapeado de Resultados de Aprendizaje del módulo 0487 a los capítulos del libro.

Consejo



Si eres docente, puedes usar esta tabla para planificar la temporalización del módulo asignando cada capítulo a una evaluación o unidad didáctica. Cada capítulo incluye ejercicios evaluables alineados con los criterios de evaluación del RA correspondiente.



Contenidos

Desarrollo de software

Este capítulo ofrece una visión general sobre el desarrollo de software, abordando desde los conceptos básicos de los programas informáticos y los tipos de código, hasta las tecnologías modernas de virtualización y las metodologías de gestión de proyectos. Se explican los principales lenguajes de programación, sus paradigmas y características, así como las fases fundamentales en la creación de una aplicación. El objetivo es proporcionar una base sólida para comprender el proceso completo de desarrollo y las herramientas empleadas en la industria actual.

3.1. Concepto de programa informático

Un programa informático es un conjunto de instrucciones escritas en un lenguaje de programación que una computadora puede interpretar y ejecutar para realizar tareas específicas. Los programas pueden ser simples, como una calculadora, o complejos, como un sistema operativo.

Desde los primeros algoritmos de Ada Lovelace en el siglo XIX hasta los sistemas operativos modernos, los programas informáticos han sido fundamentales para el avance tecnológico. En la década de 1940, los primeros ordenadores electrónicos ejecutaban programas escritos en lenguaje máquina. Con el tiempo, surgieron lenguajes de alto nivel que facilitaron el desarrollo y mantenimiento de software, impulsando la expansión de la informática en todos los ámbitos.

Actualmente, existen diversos tipos de programas, entre los que destacan:

- Aplicaciones de escritorio, como Microsoft Word o Photoshop.
- Aplicaciones móviles, como WhatsApp o Instagram.
- Sistemas operativos, como Windows o Linux.
- Videojuegos, como Minecraft o Fortnite.
- Software de gestión empresarial, como SAP o Salesforce.

- Herramientas científicas, como MATLAB o R.

Cada uno cumple funciones específicas y responde a necesidades concretas de usuarios y organizaciones.

Ejercicio 3.1



Busca y enumera dos ejemplos de programas informáticos para cada uno de los siguientes tipos: aplicaciones de escritorio, aplicaciones móviles, sistemas operativos, videojuegos, software de gestión empresarial y herramientas científicas. Indica brevemente la función principal de cada ejemplo.

3.2. Código fuente, objeto y ejecutable. Tecnologías de virtualización.

En el proceso de desarrollo de software, es fundamental comprender cómo se transforma el código escrito por el programador en un programa que puede ser ejecutado por una computadora. Este proceso implica varias etapas y tipos de código, cada uno con un propósito específico:

Código fuente Es el punto de partida, donde el desarrollador escribe las instrucciones en un lenguaje de programación.

Código objeto Tras la compilación, el código fuente se convierte en código objeto, que es una representación intermedia más cercana al lenguaje máquina.

Código ejecutable Finalmente, el código objeto se enlaza y genera el archivo ejecutable, listo para ser ejecutado por el sistema operativo.

Además, las tecnologías de virtualización han revolucionado la forma en que se ejecutan y gestionan los programas, permitiendo mayor flexibilidad, portabilidad y seguridad en los entornos de desarrollo y producción.

3.2.1. Código fuente

El código fuente es el conjunto de instrucciones y declaraciones escritas por el programador en un lenguaje de programación legible para humanos, como C, Java o Python. Este código describe la lógica y el comportamiento que debe seguir el programa. El código fuente suele almacenarse en archivos de texto y puede ser modificado, revisado y compartido fácilmente.

```
1 #include <stdio.h>
2 int main() {
3     printf("Hola, mundo!\n");
4     return 0;
5 }
```

Ejemplo 3.1: Archivo fuente `hola.c`

3.2.2. Código objeto

El código objeto es el resultado de la compilación del código fuente. Consiste en instrucciones en lenguaje máquina o en un formato intermedio, que no es directamente ejecutable pero está más cerca de la arquitectura del hardware. El código objeto suele estar en archivos binarios y puede requerir un proceso adicional de enlace (linking) para formar el ejecutable final. Este paso de enlace es importante porque combina varios archivos objeto y bibliotecas en un solo archivo ejecutable.

```
1 gcc -c hola.c
```

Ejemplo 3.2: Generar código objeto

Esto genera el archivo `hola.o`, que contiene el código objeto.

3.2.3. Código ejecutable

El código ejecutable es el archivo final generado tras el proceso de compilación y enlace. Este archivo contiene instrucciones que el sistema operativo puede cargar y ejecutar directamente en el hardware. Los ejecutables pueden tener extensiones como `.exe` en Windows o no tener extensión en sistemas Unix.

```
1 gcc hola.o -o hola
```

Ejemplo 3.3: Generar ejecutable

Esto genera el archivo ejecutable `hola`, que puede ejecutarse con `./hola`.

3.2.4. Tecnologías de virtualización

Las tecnologías de virtualización permiten ejecutar programas en entornos simulados, conocidos como máquinas virtuales. Esto facilita la portabilidad, el aislamiento y la gestión eficiente de recursos. Ejemplos incluyen:

Máquinas virtuales Software como VirtualBox o VMware permite ejecutar sistemas operativos completos dentro de otro sistema.

Contenedores Herramientas como Docker aíslan aplicaciones y sus dependencias, facilitando el despliegue y la escalabilidad.

Máquinas virtuales de lenguaje Algunos lenguajes, como Java, utilizan una máquina virtual (JVM) para ejecutar el código en diferentes plataformas.

La virtualización es clave en el desarrollo moderno, permitiendo pruebas, despliegue y mantenimiento más flexibles y seguros.

Ejemplo: Ejecutar un servidor de Minecraft en un contenedor Docker

Docker permite desplegar aplicaciones complejas, como servidores de videojuegos, de forma sencilla y aislada. A continuación se muestra cómo iniciar un servidor de Minecraft en un contenedor Docker de manera local:

```
1 docker run -d -p 25565:25565 --name minecraft-server \  
2   -e EULA=TRUE \  
3   itzg/minecraft-server
```

Ejemplo 3.4: Arrancar un servidor de Minecraft con Docker

Este comando descarga la imagen oficial `itzg/minecraft-server`, acepta el acuerdo de licencia (`EULA=TRUE`), y expone el puerto 25565 para que los jugadores puedan conectarse desde la red local. El servidor se ejecuta en segundo plano (`-d`) y puede ser gestionado fácilmente con comandos Docker.

Ejercicio 3.2



Responde las siguientes preguntas sobre el proceso de obtención de código ejecutable y tecnologías de virtualización:

1. ¿Cuál es la diferencia entre código fuente, código objeto y código ejecutable?
2. ¿Qué función cumple el compilador en el proceso de desarrollo de software?
3. Menciona dos ventajas que ofrecen las tecnologías de virtualización en el desarrollo de software.
4. Da un ejemplo de una máquina virtual y un ejemplo de un contenedor.
5. ¿Por qué es importante el proceso de enlace (*linking*) antes de obtener el ejecutable final?

3.3. Tipos de lenguajes de programación. Paradigmas.

Los lenguajes de programación se pueden clasificar según su nivel de abstracción y el paradigma que emplean. Los principales tipos son:

Lenguajes de bajo nivel Cercanos al hardware, como el ensamblador, permiten controlar directamente los recursos de la máquina.

Lenguajes de alto nivel Más abstractos y legibles para humanos, como Python, Java o C++, facilitan el desarrollo y mantenimiento.

Lenguajes compilados Requieren un proceso de compilación para generar ejecutables, por ejemplo, C y C++.

Lenguajes interpretados Se ejecutan línea a línea sin generar un ejecutable previo, como Python. JavaScript también se clasifica históricamente aquí, aunque los motores modernos (V8, SpiderMonkey) usan compilación JIT para mayor rendimiento.

Lenguajes híbridos Combinan compilación e interpretación, como Java (compilado a bytecode y ejecutado en la JVM).

Además, los lenguajes se agrupan según el paradigma de programación que soportan, como estructurada, orientada a objetos, funcional o lógica. Cada paradigma ofrece ventajas específicas para distintos tipos de problemas y proyectos.

3.3.1. Programación estructurada

La programación estructurada se basa en la división del programa en bloques o estructuras lógicas como secuencias, decisiones (if/else) y bucles (for, while). Favorece la claridad y el mantenimiento del código, evitando el uso excesivo de saltos incondicionales (goto).

```

1 #include <stdio.h>
2 int main() {
3     int x = 10;
4     if (x > 5) {
5         printf("x es mayor que 5\n");
6     }
7     for (int i = 0; i < x; i++) {
8         printf("%d ", i);
9     }
10    return 0;
11 }
```

Ejemplo 3.5: Ejemplo de programación estructurada en C

3.3.2. Programación orientada a objetos

Este paradigma organiza el software en objetos que combinan datos y métodos. Los conceptos clave son clases, herencia, encapsulamiento y polimorfismo. Lenguajes como Java, C++ y Python soportan este enfoque, facilitando la reutilización y escalabilidad.

```
1 class Persona:
2     def __init__(self, nombre):
3         self.nombre = nombre
4
5     def saludar(self):
6         print(f"Hola, soy {self.nombre}")
7
8 p = Persona("Ana")
9 p.saludar()
```

Ejemplo 3.6: Ejemplo de programación orientada a objetos en Python

3.3.3. Programación funcional

La programación funcional trata la computación como la evaluación de funciones matemáticas, evitando el estado mutable y los efectos secundarios. Utiliza funciones puras, composición y recursión. Ejemplos de lenguajes funcionales son Haskell y Lisp, aunque muchos lenguajes modernos incluyen características funcionales.

```
1 (defn doble [x] (* x 2))
2 (def lista [1 2 3 4])
3 (def resultado (map doble lista))
4 ;; resultado será (2 4 6 8)
```

Ejemplo 3.7: Ejemplo de programación funcional en Clojure (resaltado con léxico Lisp)

3.3.4. Programación lógica

En la programación lógica, los programas se definen mediante hechos y reglas, y la ejecución consiste en la resolución de consultas lógicas. Prolog es el lenguaje más representativo de este paradigma, utilizado en inteligencia artificial y resolución de problemas complejos.

```
1 padre(juan, maria).
2 padre(juan, pedro).
3 hermano(X, Y) :- padre(P, X), padre(P, Y), X \= Y.
4
```



```
5 | % Consulta: hermano(maria, Quien).
```

Ejemplo 3.8: Ejemplo de programación lógica en Prolog

Consejo



Puedes probar los distintos paradigmas de programación en un entorno de desarrollo para ver cuál se adapta mejor a tus necesidades. *Jupyter Notebooks* es un entorno específicamente creado para entornos de programación interactivos y experimentación con código.

Puedes ejecutar una imagen de Docker pensada para aprender distintos lenguajes de programación, de la siguiente manera:

```
1 | docker run -p 8888:8888 cavefish/jupyter-student-stack
```

3.4. Características de los lenguajes más difundidos

Lenguajes como Python, Java, C++ y JavaScript destacan por su sintaxis, tipado, gestión de memoria, soporte multiplataforma y comunidades activas. Python es conocido por su simplicidad, Java por su portabilidad, C++ por su rendimiento y JavaScript por su uso en la web.

3.4.1. Python

Python es un lenguaje interpretado, de tipado dinámico y sintaxis sencilla, lo que facilita el aprendizaje y la escritura de código. Su gestión automática de memoria y amplia biblioteca estándar lo hacen ideal para desarrollo rápido, ciencia de datos y automatización. Es multiplataforma y cuenta con una comunidad muy activa.

Ejemplo: Análisis de datos con Python

```
1 | import pandas as pd
2 |
3 | datos = pd.read_csv('datos.csv')
4 | print(datos.describe())
```

Ejemplo 3.9: Análisis de datos con pandas

3.4.2. Java

Java es un lenguaje compilado a bytecode y ejecutado en la Máquina Virtual de Java (JVM), lo que garantiza portabilidad entre sistemas. Tiene tipado estático, gestión automática de memoria mediante recolección de basura y una sintaxis orientada a objetos. Es ampliamente utilizado en aplicaciones empresariales y móviles.

Ejemplo: Aplicación web con Java (Servlet)

```
1 @WebServlet("/hola")
2 public class HolaServlet extends HttpServlet {
3     protected void doGet(HttpServletRequest req, ↵
4         HttpServletResponse resp)
5         throws ServletException, IOException {
6         resp.getWriter().println("¡Hola desde Java!");
7     }
8 }
```

Ejemplo 3.10: Servlet básico en Java

3.4.3. C++

C++ es un lenguaje compilado de tipado estático, conocido por su alto rendimiento y control sobre la gestión de memoria. Permite programación orientada a objetos y programación genérica. Es multiplataforma y se utiliza en sistemas, videojuegos y aplicaciones que requieren eficiencia.

Ejemplo: Desarrollo de videojuegos con C++

```
1 while (juegoActivo) {
2     procesarEventos();
3     actualizarEstado();
4     dibujarPantalla();
5 }
```

Ejemplo 3.11: Bucle principal de un juego en C++

3.4.4. JavaScript

JavaScript es un lenguaje de tipado dinámico diseñado principalmente para desarrollo web. Históricamente se ejecutaba mediante un intérprete, aunque los motores modernos (V8 en Chrome/Node.js, SpiderMonkey en Firefox) utilizan compilación JIT para convertirlo a código nativo en tiempo de ejecución. Se

ejecuta en navegadores y servidores (Node.js), soporta programación orientada a objetos y funcional, y su sintaxis flexible facilita la creación de aplicaciones interactivas. Es uno de los lenguajes más populares gracias a su ecosistema y comunidad.

Ejemplo: Interactividad en una página web

```
1 document.getElementById("boton").onclick = function() {  
2     alert(";Has hecho clic!");  
3 };
```

Ejemplo 3.12: Mostrar mensaje al hacer clic

Ejercicio 3.3



Escribe un programa sencillo que calcule el área de un triángulo en un lenguaje estáticamente tipado (por ejemplo, Java) y otro en un lenguaje dinámicamente tipado (por ejemplo, Python). Explica las diferencias en la declaración de variables y el manejo de tipos en cada caso. ¿Qué ventajas y desventajas observas en cada enfoque?

3.5. Fases del desarrollo de una aplicación

El desarrollo de una aplicación incluye varias fases esenciales para garantizar la calidad y funcionalidad del software. A continuación se describen cada una de ellas:

3.5.1. Análisis de requisitos

En esta fase se identifican y documentan las necesidades del cliente o usuario final. Se definen los objetivos, funcionalidades y restricciones del sistema, sirviendo como base para el resto del proceso.

Ejemplo: Para una aplicación de gestión de biblioteca, el cliente solicita un sistema que permita registrar libros, gestionar préstamos y consultar el historial de usuarios. Se documentan requisitos como “el sistema debe permitir buscar libros por título o autor” y “solo los administradores pueden eliminar registros”.

3.5.2. Diseño de la arquitectura

Se establece la estructura del sistema, especificando los componentes, su interacción y las tecnologías a emplear. El diseño puede ser conceptual, lógico y físico, y busca optimizar la solución antes de la implementación.

Ejemplo: Se decide que la aplicación de biblioteca tendrá una base de datos relacional (MySQL), una interfaz web desarrollada en React y una API REST en Python. Se dibuja un diagrama de arquitectura mostrando los módulos de usuarios, libros y préstamos, y cómo se comunican entre sí.

3.5.3. Codificación

Consiste en escribir el código fuente siguiendo el diseño previamente definido. Los desarrolladores implementan las funcionalidades utilizando lenguajes de programación y buenas prácticas de desarrollo.

Ejemplo: Se implementa la función para registrar un nuevo libro en la base de datos. En Python, se escribe el endpoint `/api/libros` que recibe los datos del libro y los almacena. En React, se crea el formulario de registro de libros.

3.5.4. Pruebas

Incluye pruebas unitarias, de integración, de sistema y de aceptación. El objetivo es detectar y corregir errores, asegurando que el software cumple con los requisitos y funciona correctamente en diferentes escenarios.

Ejemplo: Se desarrollan pruebas unitarias para la función de registro de libros, verificando que no se acepten títulos vacíos. Se realizan pruebas de integración para comprobar que el formulario web envía correctamente los datos al backend y que se almacenan en la base de datos.

3.5.5. Documentación

Se genera documentación técnica y de usuario, que describe el funcionamiento, instalación y uso del software. Facilita el mantenimiento, la formación y la transferencia de conocimiento.

Ejemplo: Se redacta un manual de usuario explicando cómo buscar libros y gestionar préstamos. Se documenta la API REST con ejemplos de uso y se crea un diagrama de flujo del proceso de préstamo de libros.

3.5.6. Explotación (Despliegue)

El software se instala y pone en funcionamiento en el entorno de producción. Se realizan configuraciones finales y se asegura que los usuarios puedan acceder y utilizar la aplicación.

Ejemplo: Se despliega la aplicación web en un servidor cloud, se configura el acceso a la base de datos y se habilita el acceso para los usuarios de la biblioteca. Se realiza una prueba piloto con usuarios reales para validar el funcionamiento.

3.5.7. Mantenimiento

Incluye la corrección de errores, la adaptación a nuevos requisitos y la mejora continua del software. El mantenimiento garantiza la vigencia y el correcto funcionamiento de la aplicación a lo largo del tiempo.

Ejemplo: Tras el despliegue, se detecta que algunos usuarios no pueden acceder a la función de búsqueda avanzada. Se corrige el error y se añade una nueva funcionalidad para exportar el historial de préstamos a Excel, según una petición del cliente.

3.6. Proceso de obtención de código ejecutable a partir del código fuente

El proceso de obtención de código ejecutable a partir del código fuente implica varias etapas y herramientas especializadas:

Edición del código fuente El programador escribe el código en un editor de texto o un entorno de desarrollo integrado (IDE) como Visual Studio Code, Eclipse o IntelliJ IDEA. Los IDEs suelen ofrecer funciones avanzadas como autocompletado, depuración y gestión de proyectos.

Compiladores Herramientas como `gcc` (para C/C++), `javac` (para Java) o `rustc` (para Rust) traducen el código fuente a código objeto o bytecode. El compilador detecta errores de sintaxis y realiza optimizaciones.

Linkers (enlazadores) El linker combina uno o varios archivos de código objeto y bibliotecas externas para generar el archivo ejecutable final. En C/C++, el proceso de enlace es fundamental para unir módulos y dependencias.

Intérpretes Lenguajes como Python o JavaScript utilizan intérpretes (`python`, `node`) que ejecutan el código fuente directamente, sin generar un ejecutable tradicional.

Herramientas de automatización y gestión de dependencias **make** Utilizado principalmente en proyectos C/C++, automatiza la compilación y el enlace mediante archivos `Makefile` que definen reglas y dependencias.

Maven y Gradle Herramientas populares en el ecosistema Java para gestionar dependencias, compilar, ejecutar pruebas y empaquetar aplicaciones. Permiten definir el ciclo de vida del proyecto y automatizar tareas repetitivas.

npm y yarn En proyectos JavaScript, gestionan paquetes y scripts de construcción.

Pruebas y análisis Herramientas como **JUnit** (Java), **pytest** (Python) o **Google Test** (C++) permiten ejecutar pruebas automáticas sobre el código antes de generar el ejecutable final.

Empaquetado y despliegue Una vez generado el ejecutable, herramientas como **Docker** pueden crear contenedores para facilitar el despliegue en diferentes entornos.

El flujo típico consiste en editar el código fuente, compilarlo, enlazarlo, ejecutar pruebas y finalmente desplegar el ejecutable. El uso de herramientas como **make**, **Maven** o **Gradle** permite automatizar y estandarizar este proceso, mejorando la productividad y la calidad del software.

Ejercicio 3.4



Crea un proyecto Java utilizando Maven que incluya al menos una clase principal y una clase de pruebas unitarias. Realiza los siguientes pasos:

1. Inicializa el proyecto con Maven (`mvn archetype:generate`) y describe la estructura de carpetas generada.
2. Implementa una clase Java con una función sencilla (por ejemplo, suma de dos números).
3. Crea una clase de prueba utilizando JUnit para verificar el correcto funcionamiento de la función.
4. Compila el proyecto y ejecuta las pruebas con los comandos `mvn compile` y `mvn test`.
5. Explica brevemente el propósito de los archivos `pom.xml`, `src/main/java` y `src/test/java`.

3.7. Metodologías ágiles y tradicionales

Las metodologías ágiles han transformado la gestión de proyectos de software al priorizar la flexibilidad, la colaboración y la entrega continua de valor. A continuación se describen las principales metodologías y enfoques utilizados en la industria:

3.7.1. Agile

El término *Agile* engloba un conjunto de principios y valores definidos en el Manifiesto Ágil (2001), que promueve la entrega rápida de software funcional, la colaboración con el cliente y la adaptación al cambio. Agile no es una metodología específica, sino un marco que da origen a métodos como Scrum, Kanban y Extreme Programming.

3.7.2. Scrum

Scrum es una metodología ágil basada en ciclos cortos llamados *sprints* (generalmente de 2 a 4 semanas). El equipo trabaja en un conjunto de tareas priorizadas (backlog) y realiza reuniones periódicas:

Daily Scrum Reunión diaria para sincronizar el trabajo.

Sprint Planning Planificación de tareas para el sprint.

Sprint Review Revisión del trabajo realizado.

Sprint Retrospective Reflexión sobre el proceso y mejora continua.

Los roles principales son Product Owner, Scrum Master y el equipo de desarrollo.

3.7.3. Kanban

Kanban es una técnica visual para gestionar el flujo de trabajo. Utiliza un tablero dividido en columnas (por ejemplo: «Por hacer», «En progreso», «Terminado») y tarjetas que representan tareas. Kanban enfatiza la entrega continua, la limitación del trabajo en curso (WIP) y la mejora incremental del proceso.

3.7.4. Extreme Programming (XP)

Extreme Programming es una metodología ágil que pone el foco en la calidad del software y la satisfacción del cliente. Sus prácticas incluyen desarrollo dirigido por pruebas (TDD), integración continua, programación en parejas y entregas frecuentes. XP fomenta la comunicación constante y la simplicidad en el diseño.

3.7.5. Desarrollo en cascada

El modelo en cascada es un enfoque tradicional y secuencial, donde cada fase (análisis, diseño, implementación, pruebas, despliegue y mantenimiento) debe completarse antes de pasar a la siguiente. Es adecuado para proyectos con requisitos bien definidos, pero puede ser rígido ante cambios.

3.7.6. Desarrollo iterativo

El desarrollo iterativo divide el proyecto en ciclos o iteraciones, donde en cada ciclo se planifica, diseña, implementa y prueba una parte del sistema. Permite incorporar retroalimentación y ajustar el producto progresivamente. Modelos como RUP (Rational Unified Process) combinan iteraciones con fases estructuradas.

Características comunes de las metodologías ágiles

Entrega incremental y continua El software se desarrolla y entrega en pequeños incrementos funcionales, permitiendo obtener valor desde las primeras etapas y ajustar el producto según la retroalimentación.

Adaptación al cambio Las metodologías ágiles facilitan la incorporación de cambios en los requisitos, incluso en fases avanzadas del desarrollo, respondiendo mejor a las necesidades del cliente.

Colaboración y comunicación Se promueve la interacción constante entre desarrolladores, clientes y otros interesados, asegurando que todos estén alineados y que las expectativas sean claras.

Transparencia y visibilidad El progreso y los obstáculos se comunican abiertamente mediante reuniones periódicas y tableros visuales, lo que permite una gestión más efectiva del proyecto.

Mejora continua Los equipos revisan regularmente sus procesos y resultados, buscando oportunidades para optimizar la forma de trabajar y aumentar la calidad del software.

La elección de la metodología más adecuada depende de factores como el tipo de proyecto, el tamaño y experiencia del equipo, y las necesidades específicas del cliente.

Ejercicio 3.5



Responde el siguiente cuestionario sobre las metodologías ágiles y tradicionales:

1. ¿Qué diferencia principal existe entre las metodologías ágiles y el modelo en cascada?
2. Enumera dos ventajas de utilizar Scrum en el desarrollo de software.
3. ¿Para qué sirve el tablero Kanban y qué representa cada columna?

4. Explica brevemente qué es la integración continua en Extreme Programming.
5. ¿Por qué es importante la entrega incremental en los proyectos ágiles?
6. ¿En qué casos puede ser preferible utilizar el modelo en cascada frente a una metodología ágil?
7. ¿Qué rol desempeña el Product Owner en Scrum?
8. Menciona una práctica clave de Extreme Programming y su beneficio.

Resumen

En este capítulo se han presentado los conceptos esenciales del desarrollo de software, desde la definición de programa informático y los distintos tipos de código, hasta las tecnologías de virtualización y las herramientas implicadas en la obtención de ejecutables. Se han analizado los principales tipos y paradigmas de lenguajes de programación, sus características y ejemplos prácticos. Además, se han descrito las fases del ciclo de vida de una aplicación y las metodologías ágiles y tradicionales que guían la gestión de proyectos en la industria actual. El conocimiento de estos fundamentos permite comprender el proceso completo de creación de software y elegir las mejores prácticas y herramientas para cada contexto.

Instalación y uso de entornos de desarrollo

En este capítulo se abordan en profundidad los conceptos fundamentales relacionados con los entornos de desarrollo, herramientas clave para cualquier programador. Se analiza qué es un entorno de desarrollo, sus funciones principales y cómo influyen en la productividad y calidad del software. El lector aprenderá a instalar y configurar diferentes IDEs y editores, como Visual Studio Code e IntelliJ IDEA, explorando sus características, ventajas y diferencias.

A lo largo del capítulo se detallan los pasos para la instalación, personalización y uso básico de estos entornos, incluyendo la gestión de proyectos, edición de código, depuración, integración con sistemas de control de versiones y automatización de tareas. Se presentan ejemplos prácticos y ejercicios que permiten experimentar con la creación de proyectos, la configuración de extensiones y plugins, y la generación de ejecutables en distintos lenguajes.

El objetivo es que el lector adquiriera una visión completa sobre cómo seleccionar, instalar y adaptar el entorno de desarrollo más adecuado a sus necesidades, comprendiendo su impacto en el flujo de trabajo y en la calidad del software desarrollado. Al finalizar, estará preparado para sacar el máximo provecho de estas herramientas y afrontar con éxito proyectos de programación en diversos contextos profesionales.

4.1. Funciones de un entorno de desarrollo

Un entorno de desarrollo proporciona herramientas para escribir, depurar, compilar y ejecutar código. Facilita la gestión de proyectos, la navegación por archivos, el control de versiones y la integración de utilidades adicionales para mejorar la productividad del programador.

Las funciones principales de un entorno de desarrollo incluyen:

Edición de código fuente Ofrece editores avanzados con resaltado de sintaxis, autocompletado, sugerencias y refactorización.

Depuración Permite ejecutar el programa paso a paso, establecer puntos de interrupción y analizar variables para detectar errores.

Compilación y ejecución Integra herramientas para compilar el código y ejecutar aplicaciones directamente desde el entorno.

Gestión de proyectos Facilita la organización de archivos, carpetas y dependencias, permitiendo trabajar en proyectos complejos.

Control de versiones Integra sistemas como Git para gestionar cambios, ramas y colaboraciones en el código.

Integración de herramientas externas Permite añadir extensiones, módulos y utilidades como linters, analizadores estáticos y gestores de paquetes.

Automatización de tareas Ofrece soporte para scripts y herramientas que automatizan procesos como pruebas, despliegue y generación de documentación.

Documentación y ayuda Incluye acceso rápido a documentación, ejemplos y asistentes para facilitar el aprendizaje y la resolución de problemas.

Estas funciones hacen que el entorno de desarrollo sea una herramienta esencial para programadores, mejorando la eficiencia y la calidad del software desarrollado.

4.2. Instalación de un entorno de desarrollo

La instalación de un entorno de desarrollo suele implicar la descarga de un IDE (como Visual Studio Code o IntelliJ IDEA), la configuración de compiladores o intérpretes, y la personalización de preferencias según el lenguaje y el tipo de proyecto.

Pasos generales para la instalación

1. **Selección del IDE:** Elige el entorno de desarrollo adecuado según el lenguaje y las necesidades del proyecto. Ejemplos populares incluyen Visual Studio Code para desarrollo web y multiplataforma, IntelliJ IDEA para Java y JVM, y PyCharm para Python.
2. **Descarga e instalación:** Accede al sitio oficial del IDE y descarga el instalador compatible con tu sistema operativo (Windows, macOS o Linux). Sigue las instrucciones del asistente de instalación.

3. **Configuración de compiladores o intérpretes:** Instala los compiladores (como GCC para C/C++, JDK para Java) o intérpretes (como Python, Node.js) necesarios para el desarrollo. Configura las rutas en el IDE para que los reconozca correctamente.
4. **Instalación de extensiones y plugins:** Añade extensiones para mejorar la funcionalidad del IDE, como soporte para lenguajes adicionales, linters, depuradores y herramientas de control de versiones.
5. **Configuración de preferencias:** Personaliza el entorno ajustando temas, atajos de teclado, formato de código y otras opciones según tus preferencias.
6. **Prueba de funcionamiento:** Crea un proyecto de prueba y ejecuta un programa sencillo para verificar que todo está correctamente instalado y configurado.

Importante

- Consulta la documentación oficial del IDE y de los lenguajes para resolver dudas durante la instalación.
- Mantén el entorno actualizado para acceder a nuevas funcionalidades y mejoras de seguridad.
- Considera la instalación de herramientas adicionales como gestores de paquetes, sistemas de control de versiones y entornos virtuales para facilitar el desarrollo.

Ejercicio 4.1

Instala tres entornos de desarrollo diferentes en tu equipo:

1. **Sublime Text:** Accede a <https://www.sublimetext.com/>, descarga el instalador para tu sistema operativo y realiza la instalación siguiendo las instrucciones.
2. **Visual Studio Code:** Ve a <https://code.visualstudio.com/>, descarga el instalador correspondiente y completa la instalación. Abre el programa y explora sus opciones básicas.
3. **IntelliJ IDEA:** Ingresa a <https://www.jetbrains.com/idea/>, descarga la versión Community o Ultimate según prefieras, instala el IDE y crea un proyecto de prueba.

Describe brevemente el proceso de instalación de cada uno y comenta las diferencias que observas en la interfaz y las funcionalidades principales.

4.3. Uso básico de un entorno de desarrollo

El uso básico de un entorno de desarrollo implica varias tareas fundamentales que permiten al programador comenzar a trabajar de manera eficiente:

Creación de proyectos Los IDEs suelen ofrecer asistentes para crear nuevos proyectos, seleccionando el tipo de aplicación, el lenguaje de programación y la estructura de carpetas. Esto facilita la organización inicial y la configuración de archivos necesarios.

Edición de archivos fuente El editor integrado permite escribir y modificar el código, con funciones como resaltado de sintaxis, autocompletado, sugerencias contextuales y navegación rápida entre archivos y funciones.

Ejecución de programas Es posible compilar y ejecutar el código directamente desde el entorno, utilizando botones o atajos de teclado. El IDE muestra la salida del programa y posibles errores en una consola integrada.

Depuración Los entornos de desarrollo incluyen herramientas para depurar el código, permitiendo establecer puntos de interrupción, inspeccionar variables, seguir el flujo de ejecución y detectar errores de manera interactiva.

Gestión de dependencias Muchos IDEs integran gestores de paquetes que facilitan la instalación y actualización de librerías externas necesarias para el proyecto.

Documentación y ayuda integrada Los entornos modernos ofrecen acceso rápido a documentación oficial, ejemplos de código y asistentes que ayudan a resolver dudas y aprender nuevas funcionalidades.

Control de versiones básico Es común encontrar integración con sistemas como Git, permitiendo realizar operaciones básicas como guardar cambios, crear ramas y sincronizar el código con repositorios remotos.

Estas funciones básicas permiten al usuario comenzar a desarrollar aplicaciones de manera rápida y eficiente, aprovechando las herramientas que el entorno proporciona para mejorar la productividad y la calidad del software.

Ejercicio 4.2



Crea un proyecto HTML con JavaScript básico en Visual Studio Code siguiendo estos pasos:

1. Abre Visual Studio Code y selecciona una carpeta donde crearás tu proyecto.
2. Haz clic en **Nuevo archivo** y nómbralo `index.html`.
3. Escribe el siguiente código en el archivo:

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Proyecto HTML con JavaScript</title>
5 </head>
6 <body>
7   <h1>¡Hola, mundo desde VS Code!</h1>
8   <button onclick="mostrarMensaje()">Haz clic aquí</
  button>
9   <p id="mensaje"></p>
10  <script>
11    function mostrarMensaje() {
12      document.getElementById('mensaje').textContent =
        '¡Has hecho clic en el botón!';
13    }
14  </script>
15 </body>
16 </html>
```

4. Instala la extensión **Live Server** desde el marketplace de VS Code.
5. Haz clic derecho en `index.html` y selecciona **Open with Live Server** para ver el resultado en tu navegador.
6. Haz clic en el botón y observa cómo aparece el mensaje usando JavaScript.

Describe los pasos que realizaste y comenta qué ventajas encontraste al usar Visual Studio Code para editar y probar tu proyecto HTML con JavaScript.

Ejercicio 4.3



Crea un proyecto Java sencillo en IntelliJ IDEA siguiendo estos pasos:

1. Abre IntelliJ IDEA y selecciona **New Project**.
2. Elige **Java** como tipo de proyecto y asegúrate de tener configurado el JDK.
3. Asigna un nombre al proyecto y selecciona la ubicación donde se guardará.
4. Crea una clase llamada **HolaMundo** en el paquete principal.
5. Escribe el siguiente código en la clase:

```
1 public class HolaMundo {  
2     public static void main(String[] args) {  
3         System.out.println(";Hola, mundo!");  
4     }  
5 }
```

6. Ejecuta el programa desde el IDE y verifica que se muestre el mensaje en la consola.

Describe los pasos que seguiste y comenta qué ventajas encontraste al usar IntelliJ IDEA para este proceso.

4.4. Personalización del entorno de desarrollo

La personalización de un entorno de desarrollo es fundamental para mejorar la comodidad, productividad y eficiencia del programador. Los entornos modernos permiten adaptar numerosos aspectos según las preferencias y necesidades del usuario. Entre las opciones más comunes de personalización se encuentran:

Temas visuales Es posible cambiar el aspecto del entorno, seleccionando entre temas claros, oscuros o personalizados. Esto ayuda a reducir la fatiga visual y adaptar el IDE al entorno de trabajo.

Estilos de codificación Los IDEs permiten configurar reglas de formato de código, como la indentación, el uso de espacios o tabuladores, el estilo de llaves y otros aspectos que facilitan la lectura y el mantenimiento del código.

Módulos y extensiones Se pueden instalar extensiones para añadir funcionalidades como linters, autocompletado avanzado, integración con frameworks, soporte para nuevos lenguajes, herramientas de análisis estático, depuradores y asistentes de refactorización.

Atajos de teclado La personalización de atajos permite acelerar tareas frecuentes, adaptando la navegación y edición a los hábitos del usuario.

Configuración de paneles y ventanas Los entornos permiten reorganizar paneles, ventanas y barras de herramientas para optimizar el espacio de trabajo y acceder rápidamente a las funciones más utilizadas.

Integración con herramientas externas Es posible conectar el IDE con sistemas de control de versiones, gestores de paquetes, terminales, servidores de desarrollo y otras utilidades externas.

Configuración de entornos de ejecución Se pueden definir diferentes perfiles de ejecución, variables de entorno y configuraciones específicas para compilar y ejecutar proyectos en distintos contextos.

La personalización permite adaptar el entorno a las preferencias del usuario, modificando temas visuales, estilos de codificación, instalando módulos y extensiones para añadir funcionalidades como linters, autocompletado y soporte para nuevos lenguajes. Esto contribuye a crear un espacio de trabajo más eficiente, agradable y alineado con las necesidades del proyecto y del equipo de desarrollo.

Ejercicio 4.4



Instala y configura una serie de extensiones recomendadas en Visual Studio Code siguiendo estos pasos:

1. Abre Visual Studio Code y accede al panel de extensiones haciendo clic en el icono de cuadrados en la barra lateral izquierda o presionando **Ctrl+Shift+X**.
2. Busca e instala las siguientes extensiones recomendadas:
 - 2.1. **Prettier - Code formatter**: Formatea automáticamente tu código según reglas configurables.
 - 2.2. **ESLint**: Analiza el código JavaScript/TypeScript y ayuda a mantener buenas prácticas.
 - 2.3. **Live Server**: Permite lanzar un servidor local para visualizar proyectos web en tiempo real.

- 2.4. **GitLens**: Mejora la integración y visualización de información de Git en el editor.
 - 2.5. **Spanish Language Pack for Visual Studio Code**: Cambia el idioma de la interfaz a español si lo prefieres.
 - 2.6. **Hide Git Ignored**: Permite ocultar archivos y carpetas ignorados por Git en el explorador de archivos.
 - 2.7. **Rainbow Brackets 2**: Colorea los pares de corchetes para facilitar la lectura del código.
 - 2.8. **vscode-icons**: Añade iconos personalizados al explorador de archivos para una mejor visualización.
3. Reinicia Visual Studio Code si es necesario para aplicar los cambios.
 4. Explora las opciones de configuración de cada extensión y realiza una prueba sencilla (por ejemplo, formatea un archivo con Prettier o inicia Live Server en un proyecto HTML).

Describe el proceso de instalación y configuración de las extensiones, y comenta cómo mejoran tu experiencia de desarrollo en Visual Studio Code.

Ejercicio 4.5



Instala y configura una serie de plugins recomendados en IntelliJ IDEA siguiendo estos pasos:

1. Abre IntelliJ IDEA y accede al menú **File** → **Settings** (o presiona **Ctrl+Alt+S**).
2. En el panel izquierdo, selecciona **Plugins**.
3. Busca e instala los siguientes plugins recomendados:
 - 3.1. **Key Promoter X**: Ayuda a aprender atajos de teclado mostrando sugerencias cuando usas el ratón.
 - 3.2. **Rainbow Brackets**: Colorea los pares de corchetes para facilitar la lectura del código.
 - 3.3. **GitToolBox**: Añade funciones avanzadas para trabajar con Git.
 - 3.4. **Lombok**: Permite usar anotaciones para reducir el código repetitivo en Java.

- 3.5. **Material Theme UI:** Cambia el aspecto visual del IDE con temas modernos.
- 3.6. **CodeGlance:** Muestra una vista previa en miniatura del código en el lateral del editor.
- 4. Reinicia IntelliJ IDEA si es necesario para activar los plugins.
- 5. Explora las opciones de configuración de cada plugin y realiza una prueba sencilla (por ejemplo, observa los colores de Rainbow Brackets o utiliza un atajo sugerido por Key Promoter X).

Describe el proceso de instalación y configuración de los plugins, y comenta cómo mejoran tu experiencia de desarrollo en IntelliJ IDEA.

4.5. Edición de programas

La edición de programas se realiza mediante editores de texto avanzados que ofrecen funcionalidades como resaltado de sintaxis, autocompletado, refactorización y navegación rápida entre archivos y símbolos. Estas herramientas ayudan a escribir código más limpio y eficiente, facilitando la detección de errores y la comprensión de la estructura del programa.

Los entornos de desarrollo modernos incluyen características adicionales para mejorar la experiencia de edición:

Resaltado de sintaxis Diferencia visualmente palabras clave, variables, comentarios y otros elementos del lenguaje, lo que facilita la lectura y el mantenimiento del código.

Autocompletado y sugerencias Propone automáticamente nombres de funciones, variables y estructuras, acelerando la escritura y reduciendo errores tipográficos.

Refactorización Permite modificar el código de manera segura, como renombrar variables, extraer métodos o reorganizar clases, manteniendo la funcionalidad original.

Navegación rápida Ofrece atajos para saltar entre archivos, funciones, clases y definiciones, mejorando la productividad en proyectos grandes.

Búsqueda y reemplazo avanzada Permite localizar y modificar fragmentos de código en todo el proyecto, utilizando expresiones regulares y filtros.

Integración con control de versiones Muestra cambios recientes, diferencias entre versiones y permite realizar operaciones como commit, push y pull directamente desde el editor.

Herramientas de análisis estático Detectan posibles errores, advertencias y sugerencias de mejora en tiempo real mientras se edita el código.

Soporte para múltiples lenguajes Los editores suelen admitir varios lenguajes de programación y permiten cambiar fácilmente entre ellos.

Además, la edición colaborativa es posible en algunos entornos, permitiendo que varios programadores trabajen simultáneamente sobre el mismo archivo, lo que resulta útil en equipos distribuidos.

En resumen, la edición de programas en un entorno de desarrollo moderno va mucho más allá de la simple escritura de texto, proporcionando herramientas inteligentes que optimizan el flujo de trabajo y contribuyen a la calidad del software.

4.6. Generación de ejecutables en distintos entornos

La generación de ejecutables varía según el lenguaje de programación y el entorno de desarrollo utilizado. Este proceso puede implicar la compilación de código fuente a binarios, la creación de paquetes instalables, o la preparación de scripts para ejecución directa. Los entornos modernos suelen automatizar estos procesos mediante tareas o comandos integrados, facilitando la distribución y el despliegue de aplicaciones.

Ejemplo en Java con IntelliJ IDEA

En Java, una forma común de generar ejecutables es utilizando Maven como sistema de construcción. IntelliJ IDEA integra Maven y facilita la creación de archivos `.jar` ejecutables:

1. Crea un proyecto Maven en IntelliJ IDEA (**File** → **New** → **Project** → **Maven**).
2. Escribe tu código Java en la carpeta `src/main/java`.
3. Añade el plugin `maven-jar-plugin` al archivo `pom.xml` para configurar la generación del `.jar` ejecutable. Por ejemplo:

```

1 <build>
2   <plugins>
3   <plugin>
4     <groupId>org.apache.maven.plugins</groupId>
5     <artifactId>maven-jar-plugin</artifactId>
6     <version>3.2.2</version>
7     <configuration>
8       <archive>
9         <manifest>
10          <mainClass>com.ejemplo.Main</mainClass>
11        </manifest>
12      </archive>
13    </configuration>
14  </plugin>
15 </plugins>
16 </build>

```

4. Haz clic derecho sobre el proyecto y selecciona **Run 'mvn package'** o ejecuta el comando en la terminal:

```
1 mvn package
```

5. El archivo `.jar` se genera en la carpeta `target` y puedes ejecutarlo con:

```
1 java -jar target/NombreDelProyecto.jar
```

Este método permite gestionar dependencias y empaquetar aplicaciones Java de forma automatizada y reproducible.

Ejemplo en C/C++ con Visual Studio Code

En C o C++, la compilación genera archivos binarios ejecutables específicos para el sistema operativo. Visual Studio Code, junto con extensiones como C/C++ y Code Runner, permite automatizar la compilación:

1. Instala la extensión C/C++ de Microsoft y configura el compilador (por ejemplo, GCC).
2. Crea un archivo `tasks.json` en la carpeta `.vscode` con la tarea de compilación:

```

1 {
2   "version": "2.0.0",
3   "tasks": [
4     {
5       "label": "Compilar C",
6       "type": "shell",

```

```
7     "command": "gcc",
8     "args": [
9         "programa.c",
10        "-o",
11        "programa.exe"
12    ],
13    "group": {
14        "kind": "build",
15        "isDefault": true
16    }
17 }
18 ]
19 }
```

3. Ejecuta la tarea desde el menú **Terminal** → **Run Build Task**.
4. El ejecutable `programa.exe` estará disponible en la carpeta del proyecto.

4.6.1. Automatización en el entorno

Muchos IDEs y editores modernos permiten definir tareas personalizadas para compilar, empaquetar y generar ejecutables. Por ejemplo, Visual Studio Code utiliza archivos `tasks.json` para automatizar comandos de compilación y ejecución. IntelliJ IDEA permite configurar perfiles de ejecución y artefactos para automatizar la generación de ejecutables y paquetes instalables. Además, herramientas como Make, Gradle o npm pueden integrarse para gestionar flujos de trabajo más complejos.

Importante

Es fundamental conocer el proceso de generación de ejecutables en el entorno y lenguaje que se utiliza, ya que esto facilita la distribución, el despliegue y la integración continua de aplicaciones en diferentes plataformas.

4.7. Herramientas y automatización

Las herramientas de automatización, como Make, Gradle o npm, permiten definir flujos de trabajo para compilar, probar y desplegar aplicaciones. La integración de estas herramientas en el entorno de desarrollo agiliza el proceso y reduce errores manuales.

4.7.1. Ejemplos de herramientas de automatización

Make Utilizado principalmente en proyectos C/C++, permite definir reglas en archivos `Makefile` para compilar y enlazar programas de forma automática.

Gradle Popular en proyectos Java y Android, gestiona dependencias, compilación, pruebas y empaquetado mediante archivos `build.gradle`.

npm (Node Package Manager) Usado en proyectos JavaScript/Node.js, facilita la instalación de paquetes, ejecución de scripts y automatización de tareas como pruebas y despliegue.

Maven Herramienta de construcción para proyectos Java, permite gestionar dependencias y el ciclo de vida del proyecto mediante archivos `pom.xml`.

Gulp y Webpack Automatizan tareas en proyectos web, como la minificación de archivos, compilación de CSS/JS y recarga automática del navegador.

4.7.2. Integración en el entorno de desarrollo

La mayoría de los IDEs y editores modernos permiten integrar estas herramientas, ejecutando tareas directamente desde el entorno, mostrando resultados en la consola y facilitando la configuración de scripts personalizados. Esto mejora la productividad y asegura que los procesos de construcción y despliegue sean reproducibles y eficientes.

Ejercicio 4.6



Elige una herramienta de automatización (Make, Gradle, npm, etc.) y configúrala en un proyecto sencillo. Describe los pasos realizados y comenta cómo facilita el flujo de trabajo en el desarrollo.

Resumen

En este capítulo se han revisado los conceptos fundamentales relacionados con los entornos de desarrollo, su instalación, uso básico, personalización y las herramientas que los complementan. Se ha explicado cómo los IDEs y editores modernos facilitan la edición de código, la depuración, la gestión de proyectos y la integración con sistemas de control de versiones y automatización. Además, se han presentado ejemplos prácticos para instalar y configurar entornos populares como Visual Studio Code e IntelliJ IDEA, así como la generación de ejecutables

y la utilización de herramientas de automatización. Dominar el entorno de desarrollo es clave para aumentar la productividad, mejorar la calidad del software y adaptarse a las necesidades de cada proyecto.

Chuletarío

Acción	IntelliJ IDEA	VS Code
<i>Edición</i>		
Duplicar línea	Ctrl+D	Shift+Alt+Down
Mover línea arriba	Shift+Alt+Up	Alt+Up
Mover línea abajo	Shift+Alt+Down	Alt+Down
Borrar línea	Ctrl+Y	Ctrl+Shift+K
Comentar/descomentar	Ctrl+/ 	Ctrl+/
Formato automático	Ctrl+Alt+L	Shift+Alt+F
<i>Navegación</i>		
Ir a clase	Ctrl+N	Ctrl+T
Ir a fichero	Ctrl+Shift+N	Ctrl+P
Ir a definición	Ctrl+B	F12
Volver atrás	Ctrl+Alt+Left	Alt+Left
Ir a línea	Ctrl+G	Ctrl+G
<i>Refactoring</i>		
Renombrar	Shift+F6	F2
Extraer método	Ctrl+Alt+M	Ctrl+Shift+R
Extraer variable	Ctrl+Alt+V	Ctrl+Shift+R
Refactoring contextual	Ctrl+Alt+Shift+T	Ctrl+Shift+R
<i>Depuración</i>		
Añadir/quitar breakpoint	Ctrl+F8	F9
Ejecutar en debug	Shift+F9	F5
Step over	F8	F10
Step into	F7	F11
Step out	Shift+F8	Shift+F11
Continuar	F9	F5
<i>Build y ejecución</i>		
Compilar proyecto	Ctrl+F9	(automático al guardar)
Ejecutar	Shift+F10	Ctrl+F5
Ejecutar tests	Ctrl+Shift+F10	(extensión Java)
Terminal integrado	Alt+F12	Ctrl+

Atajos para Windows/Linux. En macOS sustituir Ctrl por Cmd y Alt por Option.

Diseño y realización de pruebas

En este capítulo se exploran los fundamentos y buenas prácticas del diseño y realización de pruebas en el desarrollo de software. Se abordan los conceptos clave para garantizar la calidad del producto mediante la planificación, ejecución y documentación de pruebas. El contenido incluye la definición de tipos de pruebas (funcionales, estructurales, regresión, entre otras), la elaboración de procedimientos y casos de prueba, técnicas de cubrimiento de código, el uso de herramientas de automatización y la gestión de incidencias. Además, se explica el papel de los dobles de prueba y cómo contribuyen al aislamiento y verificación de componentes. El objetivo es proporcionar una visión integral que permita aplicar estrategias efectivas de pruebas en proyectos reales, optimizando recursos y minimizando riesgos.

5.1. Planificación de Pruebas

La planificación de pruebas es el proceso de definir los objetivos, alcance, recursos, cronograma y criterios de éxito para las actividades de prueba de un proyecto. Incluye la identificación de los tipos de pruebas a realizar, la asignación de responsabilidades, la estimación de tiempos y la selección de herramientas. Un buen plan de pruebas ayuda a detectar errores tempranamente y garantiza que el producto cumpla con los requisitos establecidos.

El proceso de planificación comienza con el análisis de los requisitos del sistema y la identificación de los riesgos asociados al desarrollo. Se establecen los objetivos de las pruebas, determinando qué funcionalidades y características serán verificadas y cuáles son las prioridades. El alcance de las pruebas delimita qué partes del sistema serán evaluadas y cuáles quedarán fuera del proceso.

La planificación también contempla la definición de los recursos necesarios, como el equipo de pruebas, el entorno técnico, los datos de prueba y las herramientas de automatización. Se asignan roles y responsabilidades, especificando quién será responsable de diseñar, ejecutar y documentar las pruebas.

El cronograma de pruebas detalla las fases y actividades, estableciendo fechas

clave para la ejecución, revisión y cierre de las pruebas. Se definen los criterios de entrada y salida, que determinan cuándo puede iniciarse una fase de pruebas y cuándo se considera finalizada.

Finalmente, el plan de pruebas debe incluir mecanismos para el seguimiento y control, como la gestión de incidencias, la revisión de resultados y la actualización del plan en función de los cambios en el proyecto. Una planificación adecuada permite optimizar recursos, minimizar riesgos y asegurar la calidad del producto final.

Importante

Es fundamental recordar que las pruebas no deben dejarse para el final del desarrollo. Realizarlas de forma continua durante todo el proceso permite detectar errores tempranamente, mejorar la calidad del producto y reducir costes asociados a correcciones tardías. Las pruebas son una actividad constante y esencial en el ciclo de vida del software.

5.2. Tipos de pruebas: Funcionales, estructurales y regresión, entre otras

Las pruebas de software son una actividad esencial en el ciclo de vida del desarrollo, ya que permiten verificar que el producto cumple con los requisitos y funciona correctamente en diferentes escenarios. Existen diversos tipos de pruebas, cada uno con objetivos y enfoques específicos, que ayudan a detectar errores, validar funcionalidades y asegurar la calidad del sistema. Comprender las diferencias entre pruebas funcionales, estructurales, de regresión y otras, es fundamental para diseñar una estrategia de pruebas completa y efectiva. A continuación se describen los principales tipos de pruebas utilizados en el desarrollo de software.

Las pruebas funcionales verifican que el software cumpla con los requisitos especificados, evaluando las funciones y características desde la perspectiva del usuario. Se diseñan a partir de los casos de uso y especificaciones, comprobando que cada funcionalidad responda correctamente ante diferentes entradas y situaciones. Ejemplos incluyen la validación de formularios, la navegación entre pantallas y la generación de informes.

Las pruebas estructurales, también llamadas pruebas de caja blanca, analizan la lógica interna y el flujo del código. Se centran en la cobertura de sentencias, ramas, condiciones y bucles, asegurando que todas las partes del código sean ejecutadas y verificadas. Estas pruebas requieren conocimiento del diseño y la implementación, y permiten detectar errores como condiciones no evaluadas o

caminos no recorridos.

Las pruebas de regresión aseguran que nuevas modificaciones no afecten funcionalidades existentes. Se ejecutan tras cambios en el código, como correcciones de errores o nuevas características, para comprobar que el sistema sigue funcionando correctamente. El uso de suites automatizadas facilita la detección rápida de regresiones y reduce el riesgo de introducir fallos inadvertidos.

Otros tipos de pruebas incluyen:

Pruebas de integración Verifican la interacción entre diferentes módulos o componentes, asegurando que trabajen juntos correctamente.

Pruebas de sistema Evalúan el sistema completo en su entorno real, comprobando el cumplimiento de los requisitos globales.

Pruebas de aceptación Realizadas por el cliente o usuario final, validan que el producto satisface sus necesidades y expectativas.

Pruebas de rendimiento Analizan aspectos como la velocidad, escalabilidad y estabilidad bajo diferentes cargas de trabajo.

Pruebas de seguridad Detectan vulnerabilidades y verifican la protección de datos y accesos.

Pruebas de usabilidad Evalúan la facilidad de uso y la experiencia del usuario.

La combinación de estos tipos de pruebas permite abordar la calidad del software desde diferentes perspectivas, garantizando su correcto funcionamiento, robustez y satisfacción del usuario final.

5.3. Procedimientos y casos de prueba

Los procedimientos de prueba describen los pasos detallados para ejecutar una prueba, incluyendo la preparación del entorno, los datos de entrada y las acciones a realizar. Un procedimiento de prueba bien definido especifica el orden de ejecución, los recursos necesarios, las precondiciones y las instrucciones para documentar los resultados. Esto permite que cualquier miembro del equipo pueda reproducir la prueba de manera consistente y fiable.

Los casos de prueba son escenarios específicos que definen las condiciones iniciales, las entradas, las acciones y los resultados esperados. Cada caso de prueba debe estar claramente identificado, incluir una descripción, los requisitos asociados, los pasos a seguir, los datos de entrada, el resultado esperado y el resultado obtenido tras la ejecución. La documentación de los casos de prueba

facilita la repetibilidad, el seguimiento de incidencias y la trazabilidad respecto a los requisitos del sistema.

Un ejemplo de caso de prueba podría ser:

- **ID:** CT-001
- **Descripción:** Verificar el acceso exitoso al sistema con credenciales válidas.
- **Precondiciones:** El usuario existe en la base de datos y la aplicación está disponible.
- **Datos de entrada:** Usuario: «juan», Contraseña: «1234».
- **Pasos:**
 1. Abrir la aplicación.
 2. Ingresar usuario y contraseña.
 3. Pulsar el botón «Acceder».
- **Resultado esperado:** El sistema permite el acceso y muestra la pantalla principal.

La elaboración de procedimientos y casos de prueba debe ser exhaustiva, cubriendo tanto escenarios positivos como negativos, valores límite y situaciones excepcionales. Además, es recomendable mantener los casos de prueba actualizados conforme evoluciona el sistema, y utilizar herramientas de gestión que permitan organizar, ejecutar y registrar los resultados de las pruebas de manera eficiente.

5.4. Pruebas de Código: Cubrimiento, valores límite y clases de equivalencia, entre otras

Las pruebas de código buscan asegurar la calidad y robustez del software mediante la verificación exhaustiva de la lógica interna. Entre las técnicas más relevantes se encuentran el cubrimiento de código, las pruebas de valores límite y las clases de equivalencia.

El **cubrimiento de código** es una métrica que indica el porcentaje de líneas, ramas, condiciones o caminos ejecutados durante la realización de pruebas. Existen diferentes tipos de cubrimiento:

Cubrimiento de sentencias Verifica que todas las líneas de código se ejecuten al menos una vez.

Cubrimiento de ramas Asegura que todas las alternativas de las estructuras de control (`if`, `switch`, etc.) sean evaluadas.

Cubrimiento de condiciones Comprueba que todas las condiciones booleanas sean evaluadas tanto como verdaderas como falsas.

Cubrimiento de caminos Evalúa todos los posibles recorridos a través del código, aunque suele ser inviable en sistemas complejos.

El uso de herramientas de análisis de cubrimiento permite identificar partes del código no probadas y mejorar la calidad de las pruebas.

Las **pruebas de valores límite** se centran en verificar el comportamiento del sistema en los extremos de los rangos de entrada, donde suelen aparecer errores. Por ejemplo, si una función acepta valores entre 1 y 100, se deben probar los valores 1, 100, 0 y 101 para detectar posibles fallos en la validación.

Las **clases de equivalencia** agrupan los datos de entrada en conjuntos que se espera que sean tratados de manera similar por el sistema. Se selecciona un representante de cada clase para reducir el número de pruebas necesarias sin perder cobertura. Por ejemplo, para una entrada que acepta números positivos, negativos y cero, se definen tres clases de equivalencia y se prueba un valor de cada una.

Además, existen otras técnicas como:

Pruebas de condición Evalúan combinaciones de condiciones lógicas complejas.

Pruebas de bucles Verifican el comportamiento de estructuras repetitivas, probando casos como cero iteraciones, una iteración y múltiples iteraciones.

Pruebas de excepciones Comprueban la correcta gestión de errores y situaciones inesperadas, asegurando que el sistema responde adecuadamente ante entradas o estados inválidos.

La aplicación combinada de estas técnicas permite detectar errores sutiles, mejorar la cobertura y aumentar la fiabilidad del software. Es recomendable documentar los resultados y utilizar herramientas que faciliten la automatización y el seguimiento de las pruebas de código.

5.5. Pruebas unitarias; herramientas de automatización

Las pruebas unitarias validan el funcionamiento de componentes individuales del software, como funciones, métodos o clases, de manera aislada. El objetivo

principal es asegurar que cada unidad de código realiza correctamente su tarea según lo especificado, permitiendo detectar errores de manera temprana y facilitando el mantenimiento y la evolución del sistema.

Para diseñar pruebas unitarias efectivas, es importante definir casos de prueba que cubran tanto escenarios normales como situaciones límite y excepcionales. Cada prueba debe ser independiente, reproducible y rápida de ejecutar. Además, se recomienda utilizar dobles de prueba (mocks, stubs, fakes, spies) para simular dependencias externas y aislar el comportamiento de la unidad bajo prueba.

Las herramientas de automatización juegan un papel fundamental en la ejecución y gestión de pruebas unitarias. Entre las más utilizadas se encuentran:

JUnit Framework para pruebas unitarias en Java, permite definir, ejecutar y organizar pruebas de manera sencilla.

pytest Herramienta popular en Python, facilita la escritura de pruebas simples y complejas, soporta fixtures y parametrización.

NUnit Framework para .NET, ofrece funcionalidades avanzadas para la ejecución y reporte de pruebas unitarias.

Jest Utilizado en JavaScript y TypeScript, especialmente en proyectos web, permite pruebas rápidas y cobertura de código.

unittest Módulo estándar de Python para la creación y ejecución de pruebas unitarias.

Estas herramientas permiten ejecutar pruebas de forma automática, generar informes detallados de resultados y realizar integración continua, lo que mejora la eficiencia y la calidad del desarrollo. La integración de pruebas unitarias en el flujo de trabajo ayuda a detectar errores rápidamente tras cada cambio en el código, reduce el riesgo de regresiones y facilita la refactorización.

```
1 import org.junit.jupiter.api.Test;
2 import static org.assertj.core.api.Assertions.*;
3 import org.junit.jupiter.params.provider.Arguments;
4 import org.junit.jupiter.params.ParameterizedTest;
5 import org.junit.jupiter.params.provider.ValueSource;
6 import org.junit.jupiter.params.provider.MethodSource;
7
8 import java.util.stream.Stream;
9
10 class FechaUtilsTest {
11
12     @Test
13     void testEsBisiesto() {
14         assertThat(FechaUtils.esBisiesto(2020)).isTrue();
15     }
16 }
```

```

15     assertThat(FechaUtils.esBisiesto(1900)).isFalse();
16     assertThat(FechaUtils.esBisiesto(2000)).isTrue();
17     assertThat(FechaUtils.esBisiesto(2023)).isFalse();
18 }
19
20 @ParameterizedTest
21 @ValueSource(ints = {2024, 2400})
22 void testEsBisiesto_Positivo(int año) {
23     assertThat(FechaUtils.esBisiesto(año)).isTrue();
24 }
25
26 @ParameterizedTest
27 @ValueSource(ints = {2023, 2100, 1900})
28 void testEsBisiesto_Negativo(int año) {
29     assertThat(FechaUtils.esBisiesto(año)).isFalse();
30 }
31
32 @ParameterizedTest
33 @MethodSource("aniosBisiestosProvider")
34 void testEsBisiesto_MethodSource(int año, boolean esperado) {
35     assertThat(FechaUtils.esBisiesto(año)).isEqualTo(esperado);
36 }
37
38 static Stream<Arguments> aniosBisiestosProvider() {
39     return Stream.of(
40         Arguments.of(2020, true),
41         Arguments.of(1900, false),
42         Arguments.of(2000, true),
43         Arguments.of(2023, false),
44         Arguments.of(2024, true),
45         Arguments.of(2100, false),
46         Arguments.of(2400, true)
47     );
48 }
49 }
50
51 class FechaUtils {
52     public static boolean esBisiesto(int año) {
53         return (año % 4 == 0 && año % 100 != 0) || (año % 400 == 0);
54     }
55 }

```

Ejemplo 5.1: Ejemplo de test unitario con JUnit y AssertJ para comprobar si un año es bisiesto

En resumen, la adopción de pruebas unitarias y herramientas de automatización es esencial para garantizar la fiabilidad, mantenibilidad y calidad del software, contribuyendo a un desarrollo ágil y seguro.

Ejercicio 5.1



Realiza el siguiente ejercicio práctico:

1. Crea un proyecto Java en IntelliJ IDEA.
2. Implementa una clase `Calculadora` con métodos para sumar, restar, multiplicar y dividir dos números.
3. Añade la dependencia de JUnit 5 en el archivo `pom.xml` (si usas Maven) o en la configuración de Gradle.
4. Crea una clase de prueba `CalculadoraTest` en el directorio `src/test/java`.
5. Escribe pruebas unitarias para cada método de la clase `Calculadora` usando anotaciones de JUnit (`@Test`).
6. Ejecuta las pruebas desde IntelliJ y verifica que todas pasen correctamente.
7. Modifica uno de los métodos para provocar un fallo y observa el resultado de las pruebas.

Objetivo: Familiarizarte con el uso de JUnit en IntelliJ IDEA, la creación de pruebas unitarias y la interpretación de los resultados.

5.6. Documentación de las incidencias

La documentación de incidencias consiste en registrar de manera detallada los errores, fallos o comportamientos inesperados detectados durante las pruebas. Este proceso es fundamental para asegurar la trazabilidad, el seguimiento y la resolución eficiente de los problemas encontrados en el software.

Una incidencia debe incluir al menos los siguientes elementos:

1. **Identificador único:** Permite distinguir cada incidencia y facilitar su seguimiento.
2. **Descripción:** Explica claramente el problema observado, incluyendo los síntomas y el impacto en el sistema.
3. **Pasos para reproducir:** Detalla las acciones necesarias para que cualquier miembro del equipo pueda replicar el error.

4. **Entorno de prueba:** Especifica la configuración del sistema, versión del software, hardware, sistema operativo y otros factores relevantes.
5. **Datos de entrada:** Indica los valores o condiciones que provocaron la incidencia.
6. **Resultado esperado y obtenido:** Compara lo que debería ocurrir según los requisitos con lo que realmente sucede.
7. **Gravedad y prioridad:** Clasifica la incidencia según su impacto en el funcionamiento y la urgencia de su resolución.
8. **Estado:** Indica si la incidencia está abierta, en proceso, resuelta o cerrada.
9. **Responsable:** Persona asignada para investigar y solucionar la incidencia.
10. **Historial de acciones:** Registra las actividades realizadas, cambios de estado y comentarios relevantes.

La gestión adecuada de incidencias implica el uso de herramientas especializadas, como sistemas de seguimiento (*issue trackers*) que permiten organizar, priorizar y monitorizar el ciclo de vida de cada incidencia. Ejemplos de estas herramientas son Jira, Redmine, GitHub Issues y Bugzilla.

Una documentación clara y completa de las incidencias facilita la comunicación entre los equipos de desarrollo, pruebas y soporte, contribuye a la mejora continua del producto y ayuda a prevenir la repetición de errores en futuras versiones. Además, permite analizar tendencias, identificar áreas problemáticas y tomar decisiones informadas para optimizar la calidad del software.

5.7. Dobles de prueba. Tipos. Características

Los dobles de prueba son objetos o componentes que sustituyen a dependencias reales del sistema durante la ejecución de pruebas, especialmente en pruebas unitarias. Su objetivo principal es aislar la unidad bajo prueba, permitiendo controlar el entorno y las respuestas de las dependencias externas, como bases de datos, servicios web, archivos o componentes complejos. Esto facilita la creación de escenarios controlados, la reproducibilidad de las pruebas y la detección de errores de manera más precisa.

Existen varios tipos de dobles de prueba, cada uno con características y usos específicos:

Mocks Simulan objetos reales y permiten verificar que ciertas interacciones han ocurrido, como la llamada a métodos con parámetros específicos. Los

mocks suelen configurarse para esperar ciertas acciones y fallan la prueba si estas no se cumplen. Son útiles para comprobar la colaboración entre componentes.

Stubs Proporcionan respuestas predefinidas a llamadas realizadas durante la prueba, sin verificar interacciones. Se utilizan para devolver datos específicos o simular comportamientos de dependencias, permitiendo probar el código en diferentes condiciones.

Fakes Son implementaciones simplificadas de componentes reales, que funcionan pero con lógica reducida o sin persistencia real. Por ejemplo, una base de datos en memoria en lugar de una base de datos completa. Los fakes permiten realizar pruebas rápidas y aisladas.

Spies Registran información sobre las llamadas realizadas, como cuántas veces se invocó un método y con qué parámetros. Permiten verificar el comportamiento del sistema sin necesidad de configurar expectativas previas, combinando características de mocks y stubs.

La elección del tipo de doble de prueba depende del objetivo de la prueba y del nivel de aislamiento requerido. El uso de dobles de prueba presenta varias ventajas:

- Permiten probar el código en ausencia de dependencias externas, evitando problemas de disponibilidad o configuración.
- Facilitan la simulación de escenarios difíciles de reproducir con componentes reales, como errores de red o respuestas inesperadas.
- Mejoran la velocidad y fiabilidad de las pruebas, al eliminar la necesidad de acceder a recursos externos.
- Ayudan a detectar errores en la lógica interna y en la interacción entre componentes.

Existen numerosas herramientas y librerías para la creación de dobles de prueba, como Mockito (Java), unittest.mock (Python), sinon (JavaScript), entre otras. Estas herramientas permiten definir, configurar y verificar dobles de prueba de manera sencilla y eficiente.

```
1 import static org.mockito.Mockito.*;
2 import org.junit.jupiter.api.Test;
3
4 class VentaEntradasTest {
5     @Test
```

```

6 void testVenderEntrada_UsaServicioPago() {
7     // Crear el mock de ServicioPago
8     ServicioPago servicioPagoMock = mock(ServicioPago.class);
9
10    // Instanciar VentaEntradas con el mock
11    VentaEntradas ventaEntradas = new VentaEntradas(↵
servicioPagoMock);
12
13    // Ejecutar el método a probar
14    ventaEntradas.venderEntrada("cliente@example.com", 25.0);
15
16    // Verificar que se llamó al método procesarPago con los ↵
parámetros correctos
17    verify(servicioPagoMock).procesarPago("cliente@example.com",↵
25.0);
18 }
19 }
20
21 interface ServicioPago {
22     void procesarPago(String cliente, double cantidad);
23 }
24
25 class VentaEntradas {
26     private final ServicioPago servicioPago;
27
28     public VentaEntradas(ServicioPago servicioPago) {
29         this.servicioPago = servicioPago;
30     }
31
32     public void venderEntrada(String cliente, double cantidad) {
33         servicioPago.procesarPago(cliente, cantidad);
34     }
35 }

```

Ejemplo 5.2: Ejemplo de test unitario con Mockito para venta de entradas

En resumen, los dobles de prueba son fundamentales para el desarrollo de pruebas unitarias efectivas, ya que permiten aislar el código bajo prueba, controlar el entorno y validar el comportamiento del sistema en diferentes situaciones.

Ejercicio 5.2



Realiza el siguiente ejercicio práctico:

1. Crea un proyecto Java en IntelliJ IDEA.
2. Implementa una interfaz `ServicioEmail` con un método `enviar(String destinatario, String mensaje)`.

3. Crea una clase `Notificador` que reciba un objeto `ServicioEmail` por el constructor y tenga un método `notificarUsuario(String usuario, String mensaje)` que utilice el servicio para enviar el mensaje.
4. Añade la dependencia de Mockito en el archivo `pom.xml` (si usas Maven) o en la configuración de Gradle.
5. Crea una clase de prueba `NotificadorTest` en el directorio `src/test/java`.
6. Utiliza Mockito para crear un mock de `ServicioEmail` y verifica que, al llamar a `notificarUsuario`, se invoca el método `enviar` con los parámetros correctos.
7. Ejecuta la prueba y comprueba que pasa correctamente.

Objetivo: Aprender a utilizar Mockito para crear dobles de prueba (mocks) y verificar interacciones en pruebas unitarias dentro de IntelliJ IDEA.

Resumen

En este capítulo se han abordado los conceptos fundamentales relacionados con el diseño y realización de pruebas en el desarrollo de software. Se ha destacado la importancia de una planificación adecuada, que permita definir objetivos, alcance, recursos y criterios de éxito para las actividades de prueba. Se han descrito los principales tipos de pruebas, como funcionales, estructurales, de regresión, integración, sistema, aceptación, rendimiento, seguridad y usabilidad, mostrando cómo cada una contribuye a la calidad global del producto.

Se ha explicado la elaboración de procedimientos y casos de prueba, enfatizando la necesidad de documentar de manera clara y precisa los escenarios, datos de entrada, pasos y resultados esperados. Las técnicas de pruebas de código, como el cubrimiento, los valores límite y las clases de equivalencia, permiten mejorar la cobertura y detectar errores sutiles en la lógica interna.

Las pruebas unitarias y el uso de herramientas de automatización facilitan la detección temprana de errores, la integración continua y el mantenimiento del software. La documentación de incidencias asegura la trazabilidad y resolución eficiente de los problemas encontrados, mientras que los dobles de prueba permiten aislar componentes y simular dependencias externas para realizar pruebas

más precisas y controladas.

En conjunto, la aplicación sistemática de estos conceptos y técnicas contribuye a la construcción de software robusto, fiable y alineado con las necesidades del usuario, optimizando recursos y minimizando riesgos durante el ciclo de vida del desarrollo.

Lecturas complementarias

Para profundizar en los temas tratados en este capítulo, se recomiendan las siguientes lecturas:

- **Effective Software Testing: A Developer's Guide**, Maurício Aniche, Manning, 2022.
Libro que aborda técnicas modernas de pruebas unitarias, automatización y buenas prácticas.
- **JUnit 5 User Guide**. <https://junit.org/junit5/docs/current/user-guide/>
Manual oficial para aprender a utilizar JUnit en proyectos Java.
- **Mockito Documentation**. <https://site.mockito.org/>
Guía para crear y gestionar dobles de prueba en Java.
- **Test-Driven Development: By Example**, Kent Beck, Addison-Wesley, 2002.
Referencia clásica sobre desarrollo guiado por pruebas y su aplicación práctica.
- **Clean Code: A Handbook of Agile Software Craftsmanship**, Robert C. Martin, Prentice Hall, 2008.
Capítulos dedicados a la importancia de las pruebas en el desarrollo de software limpio y mantenible.

Chuletario

Elemento	Uso
<i>Anotaciones JUnit 5</i>	
@Test	Marca el método como caso de prueba
@BeforeEach	Se ejecuta antes de cada test
@AfterEach	Se ejecuta después de cada test

Elemento	Uso
<code>@BeforeAll</code>	Se ejecuta una vez antes de todos los tests (método estático)
<code>@AfterAll</code>	Se ejecuta una vez después de todos los tests (método estático)
<code>@ParameterizedTest</code>	Test parametrizado; combinar con <code>@ValueSource</code> , <code>@CsvSource</code> , etc.
<code>@DisplayName(«...»)</code>	Nombre descriptivo del test en los informes
<code>@Disabled</code>	Deshabilita el test temporalmente
<i>Assertions con AssertJ</i>	
<code>assertThat(x).isEqualTo(y)</code>	Verifica que <code>x</code> es igual a <code>y</code>
<code>assertThat(x).isTrue()</code>	Verifica que <code>x</code> es <code>true</code>
<code>assertThat(x).isFalse()</code>	Verifica que <code>x</code> es <code>false</code>
<code>assertThat(x).isNull()</code>	Verifica que <code>x</code> es <code>null</code>
<code>assertThat(x).isNotNull()</code>	Verifica que <code>x</code> no es <code>null</code>
<code>assertThat(list).hasSize(n)</code>	Verifica que la lista tiene <code>n</code> elementos
<code>assertThat(list).contains(x)</code>	Verifica que la lista contiene <code>x</code>
<code>assertThat(x).assertInstanceOf(T.class)</code>	Verifica que <code>x</code> es de tipo <code>T</code>
<code>.usingComparatorWithPrecision(e)</code>	Compara decimales con tolerancia <code>e</code>
<i>Mockito</i>	
<code>mock(Clase.class)</code>	Crea un doble de <code>Clase</code>
<code>when(m.met()).thenReturn(v)</code>	Define el valor que devuelve el mock
<code>when(m.met()).thenThrow(Ex.class)</code>	Define la excepción que lanza el mock
<code>verify(m).metodo()</code>	Verifica que se llamó al método
<code>verify(m, never()).metodo()</code>	Verifica que nunca se llamó
<code>verify(m, times(n)).metodo()</code>	Verifica que se llamó exactamente <code>n</code> veces
<code>any() / eq(v)</code>	Coincide con cualquier argumento / con <code>v</code> exacto
<code>@Mock</code>	Crea el mock automáticamente (requiere <code>@ExtendWith(MockitoExtension.class)</code>)
<code>@InjectMocks</code>	Injecta los mocks en el objeto bajo prueba

Optimización y documentación

En este capítulo se abordarán las técnicas y herramientas fundamentales para optimizar y documentar programas en el desarrollo de software profesional. Se analizarán los conceptos de refactorización, los beneficios y limitaciones de mejorar la estructura interna del código, así como los patrones más habituales y el papel de las pruebas automatizadas en este proceso. Además, se presentarán los principales analizadores de código y linters, mostrando cómo ayudan a mantener la calidad y coherencia del código fuente mediante la adopción de normas de estilo y buenas prácticas. Finalmente, se explicará la importancia del control de versiones, sus estructuras y funcionalidades clave, y cómo contribuyen a la colaboración y trazabilidad en proyectos de software. El objetivo es proporcionar una visión integral de las estrategias que permiten desarrollar sistemas robustos, mantenibles y seguros.

6.1. Refactorización

6.1.1. Concepto de refactorización

La refactorización puede implicar diversas técnicas, como la reorganización de funciones, la eliminación de duplicidades, la mejora de nombres de variables y métodos, y la simplificación de estructuras de control. Es un proceso incremental que debe realizarse de manera sistemática, preferiblemente con el apoyo de pruebas automatizadas para asegurar que el comportamiento del sistema se mantiene intacto.

Entre los beneficios de la refactorización destacan la reducción de la deuda técnica, la mejora de la comprensión del código por parte de otros desarrolladores y la facilitación de futuras ampliaciones o correcciones. Además, un código bien refactorizado suele ser menos propenso a errores y más eficiente en términos de mantenimiento.

Es importante señalar que la refactorización no busca optimizar el rendimiento, sino la calidad interna del software. Por ello, debe formar parte del ciclo de

desarrollo habitual y no considerarse una tarea secundaria o postergable.

6.1.2. Limitaciones de la refactorización

La refactorización presenta ciertas limitaciones que deben ser consideradas antes de abordar el proceso. En primer lugar, puede resultar difícil en proyectos con plazos ajustados, donde la presión por entregar funcionalidad nueva reduce el tiempo disponible para mejorar la calidad interna del código. Además, la falta de documentación clara o la ausencia de pruebas automatizadas dificulta la comprensión y validación de los cambios, aumentando el riesgo de introducir errores. En sistemas grandes o con dependencias complejas, la refactorización requiere una planificación cuidadosa para evitar efectos secundarios no deseados y garantizar la estabilidad del sistema.

Otra limitación importante es la resistencia al cambio por parte del equipo, especialmente cuando se trabaja con código heredado o «legacy», que puede estar escrito en estilos obsoletos o con tecnologías desactualizadas. Adaptar este tipo de código a estándares modernos puede requerir un esfuerzo considerable y, en ocasiones, la dependencia de librerías externas o APIs restringe las posibilidades de modificación. Por ello, es fundamental evaluar el coste-beneficio de la refactorización y priorizar aquellas áreas que aporten mayor valor al proyecto.

Finalmente, la falta de experiencia en técnicas de refactorización o de herramientas especializadas puede dificultar el proceso y limitar los resultados obtenidos. La formación continua y el uso de asistentes automáticos en los IDEs pueden ayudar a superar estas barreras y mejorar la calidad del código de manera más eficiente.

6.1.3. Patrones de refactorización más usuales

Algunos patrones comunes son: *Extract Method*, *Rename*, *Move Class*, *Inline Variable* y *Replace Temp with Query*. Estos ayudan a organizar y simplificar el código.

Extract Method

Este patrón consiste en tomar un fragmento de código dentro de un método y extraerlo a un nuevo método con un nombre descriptivo. Esto mejora la legibilidad y reutilización.

```
1 public void procesarPersonas(List<Persona> personas) {  
2     for (Persona p : personas) {  
3         if (p.getEdad() > 18) {  
4             System.out.println(p.getNombre() + " es mayor de edad.");  
5         } else {
```



```

6      System.out.println(p.getNombre() + " es menor de edad.");
7    }
8  }
9 }

```

Ejemplo 6.1: Antes de aplicar Extract Method

```

1 public void procesarPersonas(List<Persona> personas) {
2     for (Persona p : personas) {
3         imprimirEdad(p);
4     }
5 }
6
7 private void imprimirEdad(Persona p) {
8     if (p.getEdad() > 18) {
9         System.out.println(p.getNombre() + " es mayor de edad.");
10    } else {
11        System.out.println(p.getNombre() + " es menor de edad.");
12    }
13 }

```

Ejemplo 6.2: Después de aplicar Extract Method

Ejemplo en IntelliJ IDEA: Selecciona el bloque de código que deseas extraer, haz clic derecho y elige **Refactor** → **Extract Method** (o usa el atajo **Ctrl+Alt+M**). IntelliJ sugerirá un nombre y creará el nuevo método automáticamente.

Rename

Permite cambiar el nombre de variables, métodos, clases o archivos para que sean más descriptivos y coherentes.

```

1 public class MainTemp {
2     // ...
3 }

```

Ejemplo 6.3: Antes de aplicar Rename

```

1 public class CalculadoraPreciosMain {
2     // ...
3 }

```

Ejemplo 6.4: Después de aplicar Rename

Ejemplo en IntelliJ IDEA: Haz clic derecho sobre el elemento a renombrar y selecciona **Refactor** → **Rename** (o usa **Shift+F6**). IntelliJ actualizará todas las referencias en el proyecto.

Move Class

Este patrón traslada una clase a otro paquete o archivo para mejorar la organización del proyecto.

Ejemplo en IntelliJ IDEA: Haz clic derecho sobre la clase y selecciona Refactor → Move (o usa F6). Elige el nuevo destino y confirma la operación.

Inline Variable

Consiste en eliminar una variable temporal cuyo nombre no aporta información adicional y usar su valor directamente donde se necesita.

```
1 | boolean esMayorDeEdad = (edad >= 18);
2 | if (esMayorDeEdad) {
3 |     System.out.println("Acceso permitido.");
4 | }
```

Ejemplo 6.5: Antes de aplicar Inline Variable

```
1 | if (edad >= 18) {
2 |     System.out.println("Acceso permitido.");
3 | }
```

Ejemplo 6.6: Después de aplicar Inline Variable

Ejemplo en IntelliJ IDEA: Coloca el cursor sobre la variable y usa Refactor → Inline Variable (o el atajo Ctrl+Alt+N). IntelliJ eliminará la variable y usará su valor directamente en el punto de uso.

Replace Temp with Query

Este patrón reemplaza variables temporales por llamadas a métodos que calculan el valor cuando se necesita, facilitando la reutilización y el mantenimiento.

```
1 | public class CalculadoraPrecios {
2 |     public static void main(String[] args) {
3 |         double precioBase = 100.0;
4 |         double precioConDescuento = precioBase * 0.9; // Aplicar un ↩
5 |         // 10% de descuento
6 |         double precioFinal = precioConDescuento * 1.21; // Añadir un ↩
7 |         // 21% de IVA
8 |         System.out.println("Precio final: " + precioFinal);
9 |     }
10 | }
```

Ejemplo 6.7: Antes de aplicar Replace Temp with Query

```
1 | public class CalculadoraPreciosRefactorizada {
2 |     public static void main(String[] args) {
```

```

3     double precioBase = 100.0;
4     double precioFinal = calcularPrecioConIva(calcularDescuento(↵
precioBase));
5     System.out.println("Precio final: " + precioFinal);
6 }
7
8 private static double calcularDescuento(double precio) {
9     // Ejemplo: aplica un 10% de descuento
10    return precio * 0.9;
11 }
12
13 private static double calcularPrecioConIva(double precio) {
14     // Ejemplo: precio con 21% de IVA incluido
15    return precio * 1.21;
16 }
17 }

```

Ejemplo 6.8: Después de aplicar Replace Temp with Query

Ejemplo en IntelliJ IDEA: Extrae la lógica de cálculo a un método usando Extract Method, elimina la variable temporal y llama al nuevo método donde sea necesario.

6.1.4. Otras técnicas de refactorización

Además de los patrones mencionados, existen otras técnicas útiles para mejorar la calidad interna del código:

Encapsulate Field Consiste en convertir campos públicos en privados y proporcionar métodos de acceso (**getters** y **setters**), mejorando el control sobre los datos y la seguridad.

Introduce Parameter Object Agrupa varios parámetros relacionados en una clase o estructura, simplificando la firma de métodos y facilitando la evolución del código.

Decompose Conditional Divide estructuras condicionales complejas en métodos más pequeños y descriptivos, aumentando la legibilidad y facilitando el mantenimiento.

Remove Dead Code Elimina código que nunca se ejecuta o que ya no es necesario, reduciendo la complejidad y el riesgo de errores.

Replace Magic Number with Symbolic Constant Sustituye números o cadenas «mágicas» por constantes con nombres descriptivos, mejorando la comprensión y facilitando cambios futuros.

Pull Up/Push Down Method Mueve métodos comunes a una clase base (pull up) o específicos a una subclase (push down) en jerarquías de herencia, promoviendo la reutilización y la organización.

Split Variable Divide una variable que se usa para diferentes propósitos en varias variables con nombres claros, evitando confusiones y errores.

Replace Conditional with Polymorphism Sustituye condicionales que dependen del tipo de objeto por el uso de herencia y métodos polimórficos, facilitando la extensión y el mantenimiento.

Estas técnicas permiten abordar diferentes problemas de diseño y contribuyen a crear sistemas más limpios, flexibles y fáciles de mantener.

Ejercicio 6.1



Crea la siguiente clase:

```
1 public class BaseDeDatosDePersonas {
2     final static private List<Persona> personas = new ↵
        ArrayList<>();
3     private int contador = 0;
4     private String ultimaOperacion = "";
5
6     public void agregarPersona(Persona p) {
7         personas.add(p);
8         contador++;
9     }
10
11     public List<Persona> obtenerPersonas() {
12         ultimaOperacion = "obtener";
13         return personas;
14     }
15
16     public Persona buscarPorNombre(String nombre) {
17         for (Persona p : personas) {
18             if (p.getNombre().equals(nombre)) {
19                 return p;
20             }
21         }
22         return null;
23     }
24
25     public void eliminarPersona(Persona p) {
26         personas.remove(p);
27         ultimaOperacion = "eliminar";
28     }
29 }
```

```

30 public int getContador() {
31     return contador;
32 }
33 }

```

Realiza las siguientes refactorizaciones:

- Añade pruebas unitarias para los métodos principales utilizando JUnit, antes de realizar el resto de refactorizaciones. El entorno de desarrollo debe permitir ejecutar las pruebas y verificar que todas pasan correctamente, y cambiar las pruebas si es necesario.
- Cambia el nombre de la clase por `PersonasRepositoryInMemory`.
- Extrae una interfaz `PersonasRepository` que defina los métodos `agregarPersona(Persona p)` y `obtenerPersonas()`.
- Refactoriza el método `buscarPorNombre` para que solo haya un nivel de indentación.
- Elimina el campo `contador` y calcula el número de personas directamente desde la lista cuando sea necesario. Cambia el nombre del método `getContador` por `contarPersonas`.

6.1.5. Refactorización y pruebas

La refactorización debe estar respaldada por pruebas automatizadas que verifiquen el correcto funcionamiento del sistema tras cada cambio. Estas pruebas permiten detectar rápidamente posibles errores introducidos durante el proceso y aseguran que la funcionalidad original se mantiene intacta. Integrar pruebas en el ciclo de refactorización facilita la validación continua y reduce el riesgo de regresiones, contribuyendo a un desarrollo más seguro y eficiente.

Ejemplos de uso de pruebas durante la refactorización

Durante la refactorización, las pruebas automatizadas juegan un papel clave para garantizar que el sistema sigue funcionando correctamente. Algunos ejemplos de uso son:

Pruebas unitarias antes y después de refactorizar Ejecutar la suite de pruebas antes de realizar cambios para asegurar que el código actual es correcto, y repetir la ejecución tras la refactorización para verificar que no se han introducido errores.

Pruebas de regresión Mantener un conjunto de pruebas que cubran funcionalidades críticas, permitiendo detectar rápidamente si algún cambio afecta el comportamiento esperado.

Pruebas de integración Si la refactorización afecta la interacción entre módulos, las pruebas de integración ayudan a validar que la comunicación entre componentes sigue siendo válida.

Pruebas automatizadas en el pipeline de CI/CD Integrar la ejecución de pruebas en el flujo de integración continua para validar automáticamente cada cambio realizado durante la refactorización.

Cobertura de código Utilizar herramientas de cobertura para identificar áreas del código que no están suficientemente probadas antes de refactorizar, y añadir pruebas si es necesario.

En conclusión, las pruebas automatizadas son esenciales durante la refactorización para asegurar que los cambios realizados no afectan negativamente al sistema y que se mantiene la calidad del código.

6.1.6. Herramientas de ayuda a la refactorización

Estas herramientas permiten realizar refactorizaciones de manera segura y eficiente, minimizando el riesgo de errores humanos. Por ejemplo, IntelliJ IDEA ofrece una amplia gama de refactorizaciones automáticas, como extracción de métodos, renombrado de símbolos, movimiento de clases y reorganización de paquetes, todo ello con soporte para deshacer y vista previa de los cambios. VSCode, mediante extensiones como «Java Refactor» y «SonarLint», proporciona funcionalidades similares, permitiendo aplicar patrones de refactorización y recibir sugerencias en tiempo real.

Además, existen herramientas especializadas como ReSharper (para .NET), Eclipse (con su propio conjunto de refactorizaciones para Java) y NetBeans, que facilitan la aplicación de buenas prácticas y la mejora continua del código. Estas herramientas suelen integrarse con sistemas de control de versiones y analizadores de código, permitiendo validar los cambios antes de integrarlos en el proyecto principal.

El uso de asistentes automáticos no solo agiliza el proceso de refactorización, sino que también ayuda a mantener la coherencia y calidad del código en equipos grandes, donde la colaboración y la revisión constante son fundamentales. Por último, muchas de estas herramientas ofrecen integración con sistemas de pruebas automatizadas, permitiendo ejecutar tests tras cada refactorización para asegurar que el comportamiento del sistema se mantiene intacto.

6.2. Analizadores de código

Los analizadores de código son herramientas que revisan el código fuente en busca de errores, malas prácticas, vulnerabilidades y oportunidades de mejora. En Java, los más utilizados son SonarLint, Checkstyle y PMD.

SonarLint se integra en los IDEs y muestra advertencias en tiempo real. Checkstyle permite definir reglas de estilo y formato, asegurando la coherencia en el proyecto. PMD detecta código duplicado, variables no utilizadas y posibles bugs.

Por ejemplo, en IntelliJ IDEA puedes instalar el plugin SonarLint y recibir sugerencias mientras escribes código. En VSCode, la extensión «SonarLint» y «Checkstyle for Java» ofrecen funcionalidades similares, ayudando a mantener la calidad del código desde el editor.

6.2.1. Ejemplo práctico de análisis de código

A continuación se muestra un ejemplo sencillo de cómo utilizar SonarLint y Checkstyle en un proyecto Java:

- **SonarLint:** Instala el plugin en tu IDE (IntelliJ IDEA o VSCode). Al escribir código, SonarLint resaltará posibles problemas, como variables no utilizadas, métodos demasiado largos o posibles vulnerabilidades. Por ejemplo, si declaras una variable que nunca usas, SonarLint mostrará una advertencia y sugerirá eliminarla.
- **Checkstyle:** Configura un archivo `checkstyle.xml` con las reglas de estilo que debe seguir tu proyecto. Al ejecutar Checkstyle, el sistema analizará el código y generará un informe con las infracciones detectadas, como nombres de variables incorrectos, métodos demasiado largos o falta de comentarios.
- **PMD:** Ejecuta PMD sobre tu código fuente para detectar duplicidades, código muerto y posibles errores lógicos. PMD puede integrarse en el proceso de compilación o ejecutarse manualmente para obtener un informe detallado.

Ejemplo de configuración de Checkstyle en Maven:

```

1 <plugin>
2   <groupId>org.apache.maven.plugins</groupId>
3   <artifactId>maven-checkstyle-plugin</artifactId>
4   <version>3.1.2</version>
5   <configuration>
6     <configLocation>checkstyle.xml</configLocation>

```

```
7   </configuration>
8   <executions>
9     <execution>
10      <phase>validate</phase>
11      <goals>
12        <goal>check</goal>
13      </goals>
14    </execution>
15  </executions>
16 </plugin>
```

El uso de estos analizadores permite detectar y corregir problemas antes de que lleguen a producción, mejorando la calidad y seguridad del software. Además, facilitan la adopción de buenas prácticas y la colaboración en equipos grandes, donde la coherencia y el cumplimiento de estándares son esenciales.

6.2.2. Normas de estilo en el código fuente

Las normas de estilo en el código fuente son un conjunto de reglas y convenciones que definen cómo debe escribirse el código para garantizar su legibilidad, coherencia y mantenibilidad. Estas normas pueden abarcar aspectos como la indentación, el uso de espacios y tabulaciones, la longitud máxima de líneas, el formato de los nombres de variables, métodos y clases, la organización de los bloques de código, y la inclusión de comentarios explicativos.

Por ejemplo, en Java es común seguir la convención de nombrar las clases con mayúscula inicial (**Persona**), los métodos y variables con minúscula inicial y notación camelCase (**agregarPersona**, **listaDePersonas**), y utilizar llaves para delimitar los bloques de código. Además, se recomienda limitar la longitud de las líneas a 80 o 120 caracteres, evitar el uso de variables globales innecesarias y escribir comentarios claros y concisos.

El cumplimiento de las normas de estilo facilita la colaboración entre desarrolladores, reduce la probabilidad de errores y permite que el código sea más fácil de entender y mantener. Muchas organizaciones definen sus propias guías de estilo, y existen herramientas como Checkstyle, ESLint (para JavaScript) o PEP8 (para Python) que ayudan a verificar automáticamente que el código cumple con las convenciones establecidas.

Ejemplo de normas de estilo en Java:

- Utilizar nombres descriptivos para variables y métodos.
- Indentar el código de forma consistente (por ejemplo, 4 espacios por nivel).
- Escribir comentarios para explicar la lógica compleja.

- Separar las declaraciones de variables y métodos por líneas en blanco para mejorar la legibilidad.
- Evitar métodos demasiado largos; dividir en funciones más pequeñas si es necesario.
- Seguir la convención de nombres de paquetes en minúsculas (`com.ejemplo.proyecto`).

Adoptar y respetar las normas de estilo es una práctica esencial en el desarrollo profesional de software, ya que contribuye a la calidad y sostenibilidad de los proyectos a largo plazo.

Ejercicio 6.2



Añade el linter de Google (Google Java Style) a un proyecto Maven utilizando el plugin Checkstyle.

1. Descarga el archivo de configuración oficial de Google para Checkstyle desde:
`https://github.com/checkstyle/checkstyle/blob/master/src/main/resources/google\_checks.xml`
2. Coloca el archivo `google_checks.xml` en la raíz de tu proyecto.
3. Añade el siguiente plugin en la sección `<build>` de tu `pom.xml`:

```

1 <plugin>
2   <groupId>org.apache.maven.plugins</groupId>
3   <artifactId>maven-checkstyle-plugin</artifactId>
4   <version>3.1.2</version>
5   <configuration>
6     <configLocation>google_checks.xml</configLocation>
7   </configuration>
8   <executions>
9     <execution>
10      <phase>validate</phase>
11      <goals>
12        <goal>check</goal>
13      </goals>
14    </execution>
15  </executions>
16 </plugin>
```

4. Ejecuta `mvn validate` para analizar tu código y ver los avisos de estilo según la guía de Google.

6.3. Control de versiones

El control de versiones es una práctica esencial en el desarrollo de software que permite gestionar los cambios en el código fuente a lo largo del tiempo. Facilita la colaboración entre desarrolladores, el seguimiento de modificaciones y la recuperación de versiones anteriores del código. Las herramientas más comunes para el control de versiones son Git, Subversion (SVN) y Mercurial.

En el capítulo 7 se profundizará en el uso de Git, incluyendo su instalación, configuración y comandos básicos.

6.3.1. Estructura de las herramientas de control de versiones

Las herramientas de control de versiones suelen estar compuestas por varios elementos clave que permiten gestionar eficazmente los cambios en los proyectos de software:

Repositorio Es el almacén central donde se guarda el historial completo de versiones del proyecto. Puede ser local (en el equipo del desarrollador) o remoto (en servidores como GitHub, GitLab o Bitbucket).

Commit Cada vez que se realiza un cambio significativo en el código, se crea un *commit*, que registra el estado del proyecto en ese momento junto con un mensaje descriptivo.

Ramas (*branches*) Permiten desarrollar nuevas funcionalidades, corregir errores o experimentar sin afectar la versión principal del proyecto. Las ramas facilitan el trabajo colaborativo y la integración de cambios.

Etiquetas (*tags*) Se utilizan para marcar versiones importantes, como lanzamientos (*releases*), facilitando la identificación y recuperación de versiones estables.

Fusión (*merge*) Es el proceso de combinar los cambios realizados en diferentes ramas, resolviendo posibles conflictos y consolidando el desarrollo.

Historial de cambios Permite consultar todas las modificaciones realizadas, quién las hizo y cuándo, facilitando la auditoría y el seguimiento del proyecto.

Colaboración y control de acceso Los sistemas modernos permiten gestionar permisos, revisar cambios mediante *pull requests* y coordinar el trabajo de equipos distribuidos.

Estas estructuras y funcionalidades hacen posible que los equipos de desarrollo trabajen de forma organizada, segura y eficiente, minimizando el riesgo de pérdida de información y facilitando la integración continua y la entrega de software de calidad.

Resumen

En este capítulo se han presentado las principales técnicas y herramientas para optimizar y documentar programas, destacando la importancia de la refactorización, el análisis de código y el control de versiones en el desarrollo profesional de software.

La refactorización permite mejorar la estructura interna del código sin alterar su funcionalidad, facilitando el mantenimiento, la comprensión y la evolución de los proyectos. Se han revisado los patrones de refactorización más habituales y las limitaciones que pueden surgir, así como la relevancia de contar con pruebas automatizadas para garantizar la calidad tras cada cambio.

El análisis de código, mediante herramientas como SonarLint, Checkstyle y PMD, ayuda a detectar errores, malas prácticas y vulnerabilidades, promoviendo la adopción de normas de estilo y buenas prácticas que mejoran la legibilidad y coherencia del código fuente. La integración de linters y analizadores en el flujo de trabajo contribuye a la prevención de problemas antes de que lleguen a producción.

Por último, el control de versiones es fundamental para gestionar los cambios en el código, facilitar la colaboración entre desarrolladores y asegurar la trazabilidad de las modificaciones. El uso de sistemas como Git permite trabajar de forma organizada, recuperar versiones anteriores y coordinar equipos distribuidos.

En conjunto, la aplicación de estas técnicas y herramientas es esencial para desarrollar software robusto, seguro y fácil de mantener, asegurando la calidad y sostenibilidad de los proyectos a largo plazo.

Herramientas de control de versiones y documentación

En el desarrollo de software moderno, la gestión eficiente del código fuente y la documentación es fundamental para garantizar la calidad, la colaboración y la evolución de los proyectos. Las herramientas de control de versiones permiten registrar, organizar y compartir los cambios realizados en el código, facilitando el trabajo en equipo y la trazabilidad de las modificaciones. Además, la integración continua automatiza la construcción y prueba de aplicaciones, mientras que la documentación y la observabilidad aseguran que el conocimiento y el funcionamiento del sistema sean accesibles y comprensibles para todos los miembros del equipo. En este capítulo se exploran las principales herramientas y buenas prácticas que conforman el entorno profesional de desarrollo, desde el uso de repositorios remotos hasta la generación de documentación y la monitorización de aplicaciones.

7.1. Repositorios remotos

Los repositorios remotos permiten almacenar el código fuente y otros archivos de un proyecto en servidores externos, facilitando la colaboración entre varios desarrolladores y el acceso desde diferentes ubicaciones. Dos de las herramientas más utilizadas para la gestión de repositorios remotos son Subversion (SVN) y Git.

7.1.1. Subversion (SVN)

Subversion es un sistema de control de versiones centralizado. En SVN, existe un único repositorio central al que los usuarios se conectan para obtener o enviar cambios. Esto facilita la administración y el control de acceso, pero puede limitar la flexibilidad en proyectos distribuidos. SVN es conocido por su simplicidad y por ser adecuado en entornos donde se requiere un flujo de trabajo más

controlado.

7.1.2. Git

Git es un sistema de control de versiones distribuido. Cada usuario tiene una copia completa del repositorio, lo que permite trabajar de manera independiente y realizar cambios localmente antes de compartirlos. Git es muy popular en proyectos de código abierto y en equipos que requieren flexibilidad y velocidad en el desarrollo.

En los siguientes puntos, entraremos en detalle sobre los aspectos más importantes de Git.

Ejercicio 7.1



Inicializa un repositorio localmente. Para ello, ejecuta el siguiente comando:

```
1|git init
```

Durante esta sección, realizarás varios pasos sobre este repositorio.

Commits

En Git, los **commits** representan puntos en el historial del proyecto donde se han registrado cambios. Cada commit incluye un mensaje descriptivo, la fecha y el autor, lo que facilita el seguimiento de las modificaciones realizadas. Los commits permiten revertir el proyecto a estados anteriores y entender la evolución del código.

Ejercicio 7.2



Crea un fichero de texto llamado `hola.txt` y añade tres líneas de texto. Después, crea un **commit** de los cambios realizados:

```
1|git add hola.txt
2|git commit -m "Añadir archivo hola.txt"
```

Ramas (branches)

Las ramas en Git permiten desarrollar nuevas funcionalidades, corregir errores o experimentar sin afectar la rama principal. Cada rama es una línea de desarrollo independiente que puede fusionarse con otras mediante operaciones de **merge**. Esto fomenta el trabajo colaborativo y la organización del desarrollo.

Ejercicio 7.3



Crea una nueva rama llamada `nueva-funcionalidad` y cámbiate a ella:

```
1| git switch -c nueva-funcionalidad
```

Edita la segunda línea del archivo `hola.txt` para que diga «Hola, mundo desde la nueva funcionalidad», y haz un `commit` de los cambios realizados.

Merge

La operación de `merge` en Git combina los cambios realizados en diferentes ramas. Es fundamental para integrar nuevas funcionalidades o correcciones en la rama principal. Git ayuda a resolver conflictos que puedan surgir cuando los mismos archivos han sido modificados en distintas ramas.

Ejercicio 7.4



Cambia a la rama principal (usualmente llamada `main`) y realiza un `merge` de la rama `nueva-funcionalidad`:

```
1| git switch main
2| git merge nueva-funcionalidad
```

Historial

Git mantiene un historial completo de todos los cambios realizados en el proyecto. Este historial permite revisar, comparar y revertir modificaciones, lo que resulta esencial para la trazabilidad y la recuperación ante errores.

Ejercicio 7.5



Visualiza el historial de commits en la rama principal:

```
1| git log
```

Observa los mensajes de commit, las fechas y los autores. Practica con los comandos `git diff` para ver las diferencias entre commits y `git checkout` para explorar diferentes estados del proyecto.

Ahora, borra una línea del archivo `hola.txt` y ejecuta el siguiente comando para saber qué cambios están pendientes:

```
1| git status
```

Podrás guardar los cambios temporalmente en el `stash`, lo cual te permitirá limpiar el área de trabajo sin perder los cambios pendientes. Para ello, ejecuta el siguiente comando:

```
1|git stash
```

Si quieres recuperar los cambios guardados en el *stash*, puedes usar el comando:

```
1|git stash pop
```

Repositorio remoto

Un repositorio remoto es una copia del proyecto alojada en servidores externos, como GitHub, GitLab o Bitbucket. Los desarrolladores sincronizan sus cambios locales con el repositorio remoto usando comandos como **push** (para enviar cambios) y **pull** (para recibir actualizaciones).

Ejercicio 7.6



Crea un repositorio remoto en GitHub y vincúlalo con tu repositorio local. Para ello, sigue estos pasos:

1. Accede a GitHub y crea un nuevo repositorio (sin inicializar con README).
2. Copia la URL del repositorio remoto (por ejemplo, `https://github.com/usuario/nombre-repo.git`).
3. En tu terminal, vincula el repositorio local con el remoto usando el siguiente comando:

```
1|git remote add origin https://github.com/usuario/↵  
    nombre-repo.git
```

4. Envía tus cambios locales al repositorio remoto con:

```
1|git push -u origin main
```

Comprueba en GitHub que los archivos se han subido correctamente. Si quieres obtener los cambios del repositorio remoto, utiliza el comando:

```
1|git pull origin main
```

Ahora, borra el repositorio en tu máquina, y clónalo remotamente mediante:

```
1|git clone https://github.com/usuario/nombre-repo.git
```


Colaboración

Git facilita la colaboración entre desarrolladores mediante la gestión de ramas, revisiones de código y resolución de conflictos. Las herramientas de alojamiento de repositorios remotos ofrecen funcionalidades adicionales como solicitudes de extracción (*pull requests*), comentarios y seguimiento de incidencias.

Ejercicio 7.7



Crea una nueva rama llamada `colaboracion` y realiza un `push` de esta rama al repositorio remoto:

```
1|git switch -c colaboracion
2|git push -u origin colaboracion
```

Dentro de GitHub, crea una solicitud de extracción (*pull request*) desde la rama `colaboracion` hacia la rama `main`. Esto permitirá revisar los cambios antes de fusionarlos. Comprueba que cambios se han realizado y discútelos con tu equipo antes de proceder a la fusión. Una vez revisado, *mergea* la solicitud.

Desde tu máquina, vuelve a elegir la rama `main` y actualízala con los cambios de la rama `colaboracion`:

```
1|git switch main
2|git pull origin main
```

Importante



En proyectos de código abierto, es habitual que los colaboradores externos trabajen sobre un *fork* del repositorio original. Esto permite realizar cambios sin afectar directamente al proyecto principal. Al crear una *pull request*, se pueden revisar y discutir los cambios antes de fusionarlos en el repositorio original. Este flujo de trabajo es común en proyectos de código abierto y fomenta la colaboración efectiva entre desarrolladores.

Consejo



Si quieres más información, puedes consultar la documentación oficial de Git en <https://git-scm.com/doc> y la guía de GitHub en <https://docs.github.com/es>.

7.1.3. Uso integrado en el entorno de desarrollo

Muchos entornos de desarrollo integrados (IDE) ofrecen soporte nativo para sistemas de control de versiones como Git y Subversion. Esto permite realizar operaciones habituales (commit, push, pull, merge, gestión de ramas) directamente desde la interfaz gráfica, sin necesidad de utilizar la terminal.

Por ejemplo, Visual Studio Code, IntelliJ IDEA, Eclipse y NetBeans incluyen paneles para visualizar el historial de cambios, comparar versiones, resolver conflictos y gestionar repositorios remotos. Además, suelen mostrar notificaciones sobre archivos modificados y permiten realizar commits selectivos.

El uso integrado facilita la adopción de buenas prácticas de control de versiones, mejora la productividad y reduce errores, ya que el IDE puede guiar al usuario en cada paso y ofrecer herramientas visuales para la gestión de cambios.

Ejercicio 7.8



Instala la extensión de Git en tu entorno de desarrollo (por ejemplo, Visual Studio Code).

Abre el proyecto en el IDE y realiza las siguientes acciones usando la interfaz gráfica:

1. Realiza un commit de los cambios en el archivo `hola.txt`.
2. Crea una nueva rama llamada `rama-ide` desde la interfaz.
3. Cambia a la rama `rama-ide` y edita el archivo `hola.txt`.
4. Realiza un commit de los cambios desde el IDE.
5. Fusiona la rama `rama-ide` con la rama principal usando la herramienta de merge del IDE.
6. Sincroniza los cambios con el repositorio remoto usando la opción de `push`.

Describe brevemente las ventajas de usar el control de versiones integrado en el IDE frente al uso exclusivo de la terminal.

Consejo



Si quieres simplificar el uso de Git en tu ordenador, puedes usar herramientas visuales como GitHub Desktop, Sourcetree o GitKraken. Estas aplicaciones proporcionan una interfaz gráfica para realizar operaciones

de control de versiones, facilitando la gestión de repositorios y la colaboración en proyectos.

7.2. Integración continua. Herramientas

La integración continua (CI) es una práctica fundamental en el desarrollo moderno de software. Consiste en automatizar la construcción y las pruebas del proyecto cada vez que se realizan cambios en el repositorio, detectando errores rápidamente y manteniendo la calidad del código. A menudo se combina con la Entrega o Despliegue Continuos (CD), que añade la automatización del despliegue a entornos de pruebas o producción.

Existen diversas herramientas que facilitan la integración continua, entre las más populares destacan:

7.2.1. GitHub Actions

GitHub Actions es una plataforma integrada en GitHub que permite automatizar flujos de trabajo directamente desde el repositorio. Mediante archivos de configuración (`.github/workflows/`), se pueden definir acciones como ejecutar pruebas, compilar el proyecto, analizar la calidad del código o desplegar aplicaciones. GitHub Actions se activa automáticamente ante eventos como `push`, `pull request` o la creación de nuevas ramas.

- **Ventajas:** Integración nativa con GitHub, fácil configuración, amplia comunidad y marketplace de acciones reutilizables.
- **Ejemplo:** Un flujo de trabajo típico ejecuta pruebas unitarias cada vez que se realiza un `push` al repositorio.

7.2.2. Jenkins

Jenkins es una herramienta de integración continua de código abierto ampliamente utilizada en la industria. Permite automatizar tareas de construcción, prueba y despliegue mediante la configuración de *pipelines*. Jenkins se instala en servidores propios y ofrece gran flexibilidad gracias a su sistema de plugins.

- **Ventajas:** Altamente configurable, compatible con múltiples lenguajes y sistemas, integración con otras herramientas de desarrollo.

- **Ejemplo:** Un pipeline en Jenkins puede compilar el código, ejecutar pruebas y desplegar la aplicación en un entorno de pruebas cada vez que se actualiza el repositorio.

7.2.3. Otras herramientas

Además de GitHub Actions y Jenkins, existen otras soluciones como GitLab CI/CD, Travis CI, CircleCI y Azure DevOps, que ofrecen funcionalidades similares y se adaptan a diferentes necesidades y entornos.

Importante

La integración continua ayuda a detectar errores de manera temprana, mejora la colaboración entre equipos y acelera el ciclo de desarrollo. Automatizar pruebas y despliegues reduce el riesgo de introducir fallos en producción y permite entregar software de mayor calidad.

Ejercicio 7.9

Accede al repositorio público <https://github.com/cavefish-dev/jupyter-student-stack> en GitHub.

1. Haz clic en la pestaña **Actions** en la parte superior del repositorio. Observa la lista de flujos de trabajo (*workflows*) configurados.
2. Selecciona una acción reciente y revisa las distintas partes:
 - **Resumen:** Consulta el estado general (éxito, fallo, en progreso) y el evento que la activó (*push*, *pull request*, etc.).
 - **Jobs:** Observa los distintos trabajos que componen la acción. Haz clic en cada uno para ver los pasos ejecutados.
 - **Logs:** Analiza los registros de cada paso para entender qué tareas realiza (instalación de dependencias, ejecución de pruebas, despliegue, etc.).
 - **Archivos de configuración:** Ve a la carpeta `.github/workflows` en el repositorio y abre el archivo YAML correspondiente para ver cómo está definida la acción.
3. Describe brevemente qué tareas automatiza la acción seleccionada y cómo ayuda al desarrollo del proyecto.

7.3. Documentación y observabilidad

La documentación es esencial para garantizar la comprensión, el mantenimiento y la evolución de cualquier proyecto de software. Un código bien documentado facilita la incorporación de nuevos desarrolladores, la detección de errores y la reutilización de componentes.

7.3.1. Uso de comentarios

Los comentarios en el código permiten explicar el propósito de funciones, clases, variables y bloques de lógica compleja. Es recomendable escribir comentarios claros y concisos, evitando redundancias y asegurando que aporten valor. Los comentarios pueden ser de distintos tipos:

Comentarios de línea Explican instrucciones específicas.

Comentarios de bloque Describen secciones completas de código.

Comentarios de documentación Utilizados por herramientas como Javadoc, Doxygen o Sphinx para generar documentación automática.

7.3.2. Herramientas de documentación

Las herramientas de documentación permiten crear, mantener y publicar información técnica sobre el proyecto de manera estructurada y accesible. Algunas de las más utilizadas son:

Javadoc Utilizada en proyectos Java, genera documentación HTML a partir de comentarios especiales en el código fuente. Facilita la consulta de clases, métodos y atributos.

Doxygen Compatible con C, C++, Python y otros lenguajes. Permite generar documentación en varios formatos (HTML, PDF, LaTeX) a partir de anotaciones en el código.

Sphinx Popular en proyectos Python, convierte archivos reStructuredText y comentarios en documentación web navegable. Se integra con Read the Docs para publicación automática.

MkDocs Herramienta sencilla para crear documentación estática usando Markdown. Es ideal para proyectos que requieren documentación clara y rápida de mantener.

Swagger/OpenAPI Para documentar APIs REST, genera documentación interactiva que permite probar los endpoints directamente desde el navegador.

PlantUML Permite crear diagramas UML a partir de texto, útil para documentar arquitecturas y flujos de trabajo.

Estas herramientas suelen integrarse en el flujo de desarrollo y pueden ejecutarse automáticamente mediante scripts o sistemas de integración continua, asegurando que la documentación esté siempre actualizada y disponible para el equipo y los usuarios finales.

7.3.3. Distribución de la documentación

La documentación de un proyecto puede distribuirse de diversas formas para facilitar el acceso y la colaboración entre desarrolladores y usuarios:

README El archivo `README.md` es el punto de entrada principal para la documentación de un repositorio. Suele incluir una descripción general del proyecto, instrucciones de instalación, uso básico y enlaces a recursos adicionales. Es recomendable mantenerlo actualizado y claro.

Wikis Muchas plataformas como GitHub y GitLab ofrecen sistemas de *wiki* integrados, donde se puede crear documentación más extensa y estructurada. Los wikis permiten organizar la información en páginas, facilitando la navegación y la colaboración.

Documentación colaborativa Herramientas como Confluence, Notion o Google Docs permiten que varios miembros del equipo editen y comenten la documentación en tiempo real. Esto fomenta la participación y asegura que el conocimiento se mantenga centralizado y accesible.

Sitios web de documentación Generadores como Sphinx, MkDocs o Docusaurus permiten publicar la documentación en sitios web estáticos, facilitando la consulta por parte de usuarios externos y la integración con sistemas de control de versiones.

Integración con el repositorio Es buena práctica incluir la documentación técnica junto al código fuente, en carpetas como `docs/`, para asegurar que evolucione junto al proyecto y esté disponible para todos los colaboradores.

Distribuir la documentación de manera accesible y colaborativa mejora la comunicación, reduce errores y facilita la incorporación de nuevos miembros al equipo.

Ejercicio 7.10



Crea un fichero `README.md` en el directorio raíz de tu repositorio. Incluye la siguiente información (Rellena con la información de un proyecto ficticio, pero asegúrate de que sea coherente y completa):

- Nombre del proyecto
- Descripción breve
- Instrucciones de instalación
- Ejemplos de uso
- Enlaces a la documentación adicional

Mergea los cambios en la rama principal y sube el fichero al repositorio remoto. Visita la página del repositorio en GitHub y verifica que el fichero `README.md` se muestra correctamente.

Consejo: visita <https://daringfireball.net/projects/markdown/basics> para obtener más información sobre cómo escribir un fichero `Markdown`.

7.3.4. Buenas prácticas

Las siguientes buenas prácticas ayudan a mantener la documentación útil y sostenible en proyectos de software:

Escribe para el lector Piensa en quién consultará la documentación (desarrolladores, usuarios finales, nuevos miembros del equipo) y adapta el contenido y el nivel de detalle.

Sé claro y conciso Evita explicaciones innecesarias o ambiguas. Utiliza ejemplos prácticos y lenguaje sencillo.

Organiza la información Estructura la documentación en secciones lógicas (instalación, uso, API, contribución, preguntas frecuentes) y utiliza índices o tablas de contenido.

Automatiza la generación Usa herramientas que extraigan documentación directamente del código fuente para evitar desincronización y facilitar actualizaciones.

Incluye diagramas y visualizaciones Apoya las explicaciones con diagramas de arquitectura, flujos de trabajo o ejemplos visuales cuando sea relevante.

Revisa y actualiza periódicamente Establece revisiones regulares para asegurar que la documentación refleje el estado actual del proyecto.

Facilita la colaboración Permite que otros miembros del equipo propongan mejoras o corrijan errores en la documentación.

Documenta dependencias y requisitos Especifica claramente las dependencias, versiones y configuraciones necesarias para ejecutar o desarrollar el proyecto.

Incluye guías de contribución Si el proyecto es colaborativo, proporciona instrucciones claras para contribuir, reportar errores o sugerir mejoras.

Consejo



La documentación también puede ser considerada código: archivos como scripts, notebooks, ejemplos de configuración o documentación generada automáticamente forman parte integral del proyecto. Es recomendable gestionar y versionar la documentación junto al código fuente mediante sistemas de control de versiones como Git, lo que permite mantener su historial, facilitar la colaboración y asegurar que evolucione coordinadamente con el desarrollo.

7.3.5. Observabilidad y herramientas

La observabilidad en el desarrollo de software implica la capacidad de monitorear, rastrear y comprender el comportamiento de una aplicación en tiempo real. Documentar cómo se realiza la observabilidad es clave para el mantenimiento y la resolución de incidencias. Las principales herramientas de la observabilidad son:

Sistemas de logging Permiten registrar eventos y errores (por ejemplo, Log4j, Winston, Serilog).

Monitorización Herramientas como Prometheus, Grafana, Datadog o New Relic ayudan a visualizar métricas y alertas.

Tracing Soluciones como Jaeger y Zipkin permiten rastrear peticiones a través de servicios distribuidos.

APM (Application Performance Monitoring) Plataformas como Elastic APM y Dynatrace ofrecen análisis detallado del rendimiento y la salud de las aplicaciones.

Integrar estas herramientas y documentar su uso facilita la detección de problemas, la optimización del rendimiento y la mejora continua del software.

Ejercicio 7.11



Realiza una comparativa entre al menos tres sistemas de monitorización (por ejemplo, Prometheus, Grafana, Datadog, New Relic, Zabbix, etc.). Para cada sistema, investiga y completa una tabla con los siguientes atributos:

- Facilidad de instalación y configuración
- Tipos de métricas soportadas (sistemas, aplicaciones, bases de datos, etc.)
- Integraciones disponibles (con lenguajes, frameworks, servicios en la nube)
- Visualización de datos (paneles, alertas, gráficos)
- Escalabilidad y rendimiento
- Coste (licencia, gratuito, pago por uso)
- Comunidad y soporte

Presenta la tabla comparativa y redacta un breve análisis sobre cuál sería el sistema más adecuado para un proyecto de tamaño medio en una empresa, justificando tu elección.

Resumen

En este capítulo se han presentado las principales herramientas y prácticas relacionadas con el control de versiones, la integración continua, la documentación y la observabilidad en proyectos de software. El uso de sistemas como Git y Subversion permite gestionar el historial de cambios, facilitar la colaboración y mantener la trazabilidad del desarrollo. La integración continua, mediante plataformas como GitHub Actions y Jenkins, automatiza la construcción, prueba y despliegue, mejorando la calidad y la velocidad de entrega del software.

La documentación, tanto en el código como en archivos y sitios web dedicados, es esencial para la comprensión y el mantenimiento del proyecto. Herramientas como Javadoc, Doxygen, Sphinx y MkDocs ayudan a mantener la información técnica organizada y accesible. Además, la observabilidad mediante sistemas

de monitorización y logging permite detectar y resolver problemas de manera eficiente, asegurando el correcto funcionamiento de las aplicaciones.

Adoptar estas prácticas y herramientas contribuye a la profesionalización del desarrollo de software, fomenta la colaboración y facilita la evolución y el éxito de los proyectos.

Chuletarío

Comando	Descripción
<i>Estado y staging</i>	
<code>git status</code>	Muestra ficheros modificados, en staging y sin seguimiento
<code>git diff</code>	Diferencias entre directorio de trabajo y staging
<code>git diff --staged</code>	Diferencias entre staging y último commit
<code>git add <fichero></code>	Añade un fichero al área de staging
<code>git add .</code>	Añade todos los cambios al staging
<i>Commits</i>	
<code>git commit -m «msg»</code>	Crea un commit con el mensaje indicado
<code>git log --oneline</code>	Historial de commits en formato compacto
<code>git show <sha></code>	Muestra los cambios de un commit concreto
<i>Ramas</i>	
<code>git branch</code>	Lista las ramas locales
<code>git branch <nombre></code>	Crea una nueva rama
<code>git switch <rama></code>	Cambia a la rama indicada
<code>git switch -c <nombre></code>	Crea una rama y cambia a ella
<code>git merge <rama></code>	Fusiona la rama indicada en la actual
<code>git branch -d <nombre></code>	Elimina una rama (solo si está fusionada)
<i>Repositorio remoto</i>	
<code>git clone <url></code>	Clona un repositorio remoto
<code>git remote -v</code>	Lista los remotos configurados
<code>git fetch</code>	Descarga cambios remotos sin aplicarlos
<code>git pull</code>	Descarga y aplica cambios de la rama remota
<code>git push</code>	Publica commits locales en el remoto
<code>git push -u origin <rama></code>	Publica una rama nueva y la vincula al remoto
<i>Deshacer cambios</i>	
<code>git restore <fichero></code>	Descarta cambios en el directorio de trabajo

Comando	Descripción
<code>git restore --staged <fichero></code>	Saca un fichero del staging sin perder cambios
<code>git revert <sha></code>	Crea un commit que deshace el commit indicado

Elaboración de diagramas de clases

En este capítulo se estudia el proceso de elaboración de diagramas de clases, una de las herramientas fundamentales en el diseño orientado a objetos. Los diagramas de clases permiten representar de forma gráfica la estructura estática de un sistema, mostrando las clases, sus atributos y métodos, así como las relaciones entre ellas. Se abordan los conceptos clave como la definición de clases, la instanciación de objetos, los distintos tipos de relaciones (asociación, herencia, composición, agregación, realización y dependencia), y la notación utilizada en los diagramas. Además, se presentan algunas herramientas para la creación de diagramas de clases y se introduce la generación automática de código y la ingeniería inversa a partir de estos diagramas.

Consejo



A lo largo de este capítulo, encontrarás varios diagramas, y junto a ellos, el código PlantUML utilizado para generarlos. En la sección de herramientas, se mencionan algunas opciones para crear diagramas de clases.

Los diagramas mostrados no siguen estrictamente las convenciones de UML, pero son una representación útil de los conceptos que se están tratando.

Consejo: Tanto en el desarrollo de software, como en la vida misma, es importante encontrar un equilibrio entre la teoría y la práctica.

8.1. Clases

Una clase es una plantilla que define las características y comportamientos de los objetos. Los atributos representan el estado de la clase, mientras que los métodos definen su comportamiento.

Cada uno de sus componentes se representa en el diagrama de clases de la siguiente manera:

Clases Se representan como rectángulos divididos en tres secciones. La sección

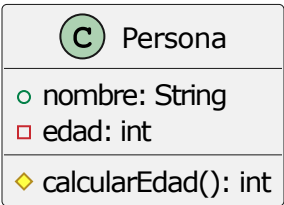
superior contiene el nombre de la clase.

Atributos Se muestran en la sección central del rectángulo de la clase, cada uno en una línea, precedido por el símbolo de visibilidad.

Métodos Se colocan en la sección inferior del rectángulo, cada uno en una línea, también precedidos por el símbolo de visibilidad.

Visibilidad Se indica mediante símbolos antes del nombre del atributo o método: + (público), - (privado), # (protegido).

```
1 @startuml
2 class Persona {
3   + nombre: String
4   - edad: int
5   # calcularEdad(): int
6 }
7 @enduml
```



8.2. Objetos

Un objeto es una instancia de una clase, es decir, una entidad concreta que posee los atributos y métodos definidos en la clase. La instanciación es el proceso de crear un objeto a partir de una clase. En los diagramas de clases, los objetos se representan como instancias de las clases, generalmente en diagramas de objetos.

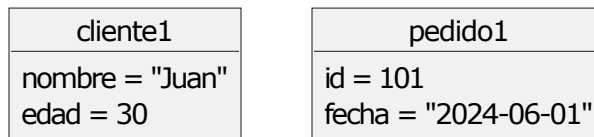
Objetos En UML, los objetos se representan como rectángulos con el nombre del objeto subrayado, seguido de dos puntos y el nombre de la clase a la que pertenecen (por ejemplo, miObjeto:MiClase). Los atributos pueden mostrarse con sus valores actuales.

Instanciación El proceso de instanciación no se representa explícitamente en los diagramas de clases UML, pero se puede mostrar en diagramas de objetos, donde se visualizan instancias concretas de las clases y sus relaciones.

```

1 @startuml
2
3 object cliente1 {
4     nombre = "Juan"
5     edad = 30
6 }
7
8 object pedido1 {
9     id = 101
10    fecha = "2024-06-01"
11 }
12
13 @enduml

```



8.3. Relaciones

Las relaciones entre clases pueden ser de varios tipos, cada una con un propósito y una representación específica en los diagramas de clases UML. Estas relaciones permiten modelar cómo interactúan y se vinculan las distintas clases dentro de un sistema, facilitando la comprensión de la arquitectura y el diseño del software. Entre los tipos más comunes de relaciones se encuentran la asociación, la herencia, la composición, la agregación, la realización y la dependencia. A continuación se describen brevemente cada una de ellas y su notación en UML.

Asociación Representa una conexión entre dos clases. En UML, se dibuja como una línea sólida entre las clases, opcionalmente con un nombre de la relación y roles en los extremos.

Navegabilidad Indica la dirección en la que una clase conoce a la otra. Se representa con una flecha en uno de los extremos de la línea de asociación, apuntando hacia la clase conocida.

Multiplicidad Especifica cuántos objetos de una clase pueden estar relacionados con objetos de otra clase. Se indica con números o rangos (por ejemplo, 1, 0..*, *, 1..n) cerca de los extremos de la línea de asociación.

Herencia Permite que una clase herede atributos y métodos de otra. En UML, se representa con una línea sólida con una flecha vacía (triángulo) apuntando hacia la clase base (superclase).

Composición Relación fuerte donde una clase contiene a otra y el ciclo de vida del objeto contenido depende de la clase contenedora. Se dibuja como una línea sólida con un rombo negro en el extremo de la clase contenedora.

Agregación Relación débil de contención. Se representa con una línea sólida y un rombo blanco en el extremo de la clase agregadora.

Realización Una clase implementa una interfaz. En UML, se muestra con una línea discontinua y una flecha vacía (triángulo) apuntando hacia la interfaz.

Dependencia Una clase utiliza a otra de manera temporal. Se representa con una línea discontinua con una flecha normal apuntando hacia la clase de la que depende.

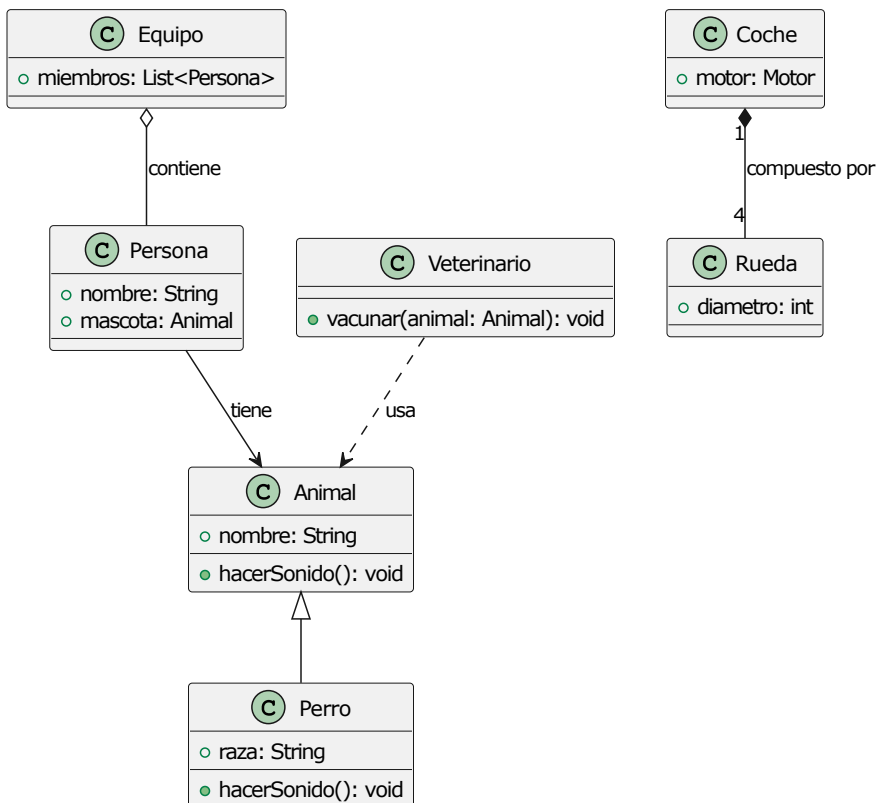
```
1 @startuml
2
3 ' Clase base para herencia
4 class Animal {
5     +nombre: String
6     +hacerSonido(): void
7 }
8
9 ' Herencia (generalización)
10 class Perro extends Animal {
11     +raza: String
12     +hacerSonido(): void
13 }
14
15 ' Asociación
16 class Persona {
17     +nombre: String
18     +mascota: Animal
19 }
20
21 Persona --> Animal : tiene
22
23 ' Agregación
24 class Equipo {
25     +miembros: List<Persona>
26 }
27
28 Equipo o-- Persona : contiene
29
30 ' Composición
31
32 class Coche {
33     +motor: Motor
34 }
```



```

35
36 class Rueda {
37     +diametro: int
38 }
39
40 Coche "1" *-- "4" Rueda : compuesto por
41
42 ' Dependencia
43 class Veterinario {
44     +vacunar(animal: Animal): void
45 }
46
47 Veterinario ..> Animal : usa
48
49 @enduml

```



Ejercicio 8.1

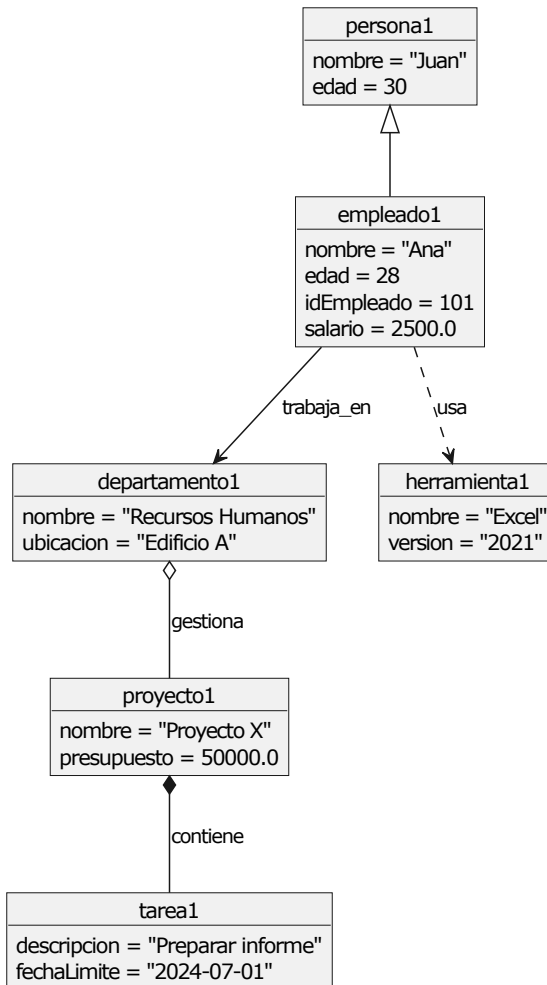


Crea el código Java representado por el diagrama de clases anterior.

Estas relaciones también aplican a los diagramas de objetos:

```
1 @startuml
2
3 ' Objetos con atributos
4 object persona1 {
5     nombre = "Juan"
6     edad = 30
7 }
8
9 object empleado1 {
10     nombre = "Ana"
11     edad = 28
12     idEmpleado = 101
13     salario = 2500.0
14 }
15
16 object departamento1 {
17     nombre = "Recursos Humanos"
18     ubicacion = "Edificio A"
19 }
20
21 object proyecto1 {
22     nombre = "Proyecto X"
23     presupuesto = 50000.0
24 }
25
26 object tarea1 {
27     descripcion = "Preparar informe"
28     fechaLimite = "2024-07-01"
29 }
30
31 object herramienta1 {
32     nombre = "Excel"
33     version = "2021"
34 }
35
36 ' Herencia (generalización) - solo conceptual en objetos
37 persona1 <|-- empleado1
38
39 ' Asociación
40 empleado1 --> departamento1 : trabaja_en
41
42 ' Agregación
43 departamento1 o-- proyecto1 : gestiona
44
45 ' Composición
46 proyecto1 *-- tarea1 : contiene
47
48 ' Dependencia
49 empleado1 ..> herramienta1 : usa
```

50 |
51 | @enduml



8.3.1. Recomendaciones para la elaboración de diagramas de clases

Al crear diagramas de clases, es recomendable seguir ciertas buenas prácticas para asegurar que sean útiles, claros y mantenibles:

Simplicidad y claridad Evita sobrecargar el diagrama con detalles innecesarios. Representa únicamente las clases, atributos y métodos relevantes para el contexto del sistema.

Consistencia en la notación Utiliza símbolos y convenciones estándar de UML para facilitar la comprensión por parte de otros desarrolladores y equipos.

Nombres descriptivos Asigna nombres claros y significativos a las clases, atributos y métodos, reflejando su propósito y funcionalidad.

Organización jerárquica Agrupa las clases relacionadas y utiliza paquetes o módulos para organizar el diagrama, especialmente en sistemas grandes.

Documentación complementaria Acompaña el diagrama con descripciones breves de las clases y relaciones, explicando su función en el sistema.

Actualización continua Mantén los diagramas actualizados conforme evoluciona el diseño y la implementación del sistema, evitando que se desincronicen del código fuente.

Uso de herramientas adecuadas Selecciona herramientas que se integren con tu flujo de trabajo y permitan la colaboración entre miembros del equipo.

Aplicar estas recomendaciones contribuye a que los diagramas de clases sean una herramienta efectiva para el diseño, la comunicación y la documentación de sistemas orientados a objetos.

Ejercicio 8.2



Crea el diagrama UML de un juego de cartas, incluyendo las clases principales como **Carta**, **Mazo**, **Jugador** y **Partida**. Define los atributos y métodos relevantes para cada clase, y establece las relaciones entre ellas, como la asociación entre **Jugador** y **Partida**, o la composición entre **Mazo** y **Carta**. Utiliza la notación UML adecuada para representar las clases, atributos, métodos y relaciones.

Además, crea un diagrama de objetos que muestre una instancia concreta de una partida en curso, con jugadores específicos y un mazo de cartas inicializado. Asegúrate de incluir los valores actuales de los atributos en el diagrama de objetos para reflejar el estado del juego en ese momento.

8.4. Herramientas

A continuación se describen brevemente algunas de las herramientas más utilizadas para la elaboración de diagramas de clases:

StarUML Es una plataforma avanzada de modelado UML que soporta múltiples tipos de diagramas, incluyendo diagramas de clases. Permite la generación de código y la ingeniería inversa, así como la colaboración en equipo y la personalización mediante extensiones.

Visual Paradigm Ofrece una amplia gama de funcionalidades para modelado UML y otros estándares, integración con IDEs, generación de documentación y soporte para metodologías ágiles. Es adecuada tanto para proyectos educativos como empresariales.

Enterprise Architect Herramienta profesional orientada a grandes proyectos, con soporte para modelado UML, BPMN y otros lenguajes. Permite la gestión de requisitos, trazabilidad y generación de código en varios lenguajes de programación.

PlantUML Permite crear diagramas a partir de texto plano, lo que facilita la integración en sistemas de control de versiones y documentación técnica. Es especialmente útil para desarrolladores que prefieren trabajar con archivos de texto y automatizar la generación de diagramas. Puedes crear dichos diagramas usando plugins de PlantUML en tu editor de texto favorito, o desde su página web oficial: <https://plantuml.com/>.

Lucidchart Es una herramienta en línea que permite crear diagramas de manera colaborativa. Ofrece plantillas y una interfaz intuitiva, facilitando el diseño y la exportación de diagramas en diversos formatos.

La elección de la herramienta depende de las necesidades del proyecto, el presupuesto y las preferencias del equipo de desarrollo. Para proyectos colaborativos y documentación integrada, PlantUML y Lucidchart son opciones populares. Para proyectos empresariales con requerimientos avanzados, Enterprise Architect y Visual Paradigm ofrecen funcionalidades adicionales.

8.5. Generación automática de código. Ingeniería inversa.

Algunas herramientas permiten generar automáticamente el código fuente a partir de los diagramas de clases (ingeniería directa), facilitando la implementación del sistema. También es posible realizar ingeniería inversa, es decir, generar diagramas de clases a partir del código existente, lo que ayuda a documentar y comprender sistemas ya desarrollados.

8.5.1. Generación automática de código

En el contexto de la generación automática de código, muchas herramientas de modelado UML permiten exportar diagramas de clases directamente a código fuente en diversos lenguajes de programación. Por ejemplo, StarUML y Visual

Paradigm ofrecen funciones para transformar diagramas UML en clases Java, C#, Python, entre otros. Este proceso consiste en definir las clases, atributos y métodos en el diagrama, y luego utilizar la opción de exportación o generación de código que provee la herramienta.

El código generado suele incluir la estructura básica de las clases, sus atributos y métodos, así como las relaciones de herencia y asociación. Sin embargo, es importante revisar y adaptar el código generado, ya que normalmente requiere ajustes para cumplir con los requisitos específicos del proyecto, como la implementación de lógica de negocio, validaciones o integración con otros componentes.

A continuación se muestran ejemplos de código generado desde la definición de dos interfaces API distintas.

Ejemplo: Generación de código a partir de un modelo OpenAPI

Supongamos que tienes un archivo de especificación OpenAPI (por ejemplo, `api.yaml`) que describe los modelos de datos y las operaciones de una API. Herramientas como `openapi-generator` permiten generar automáticamente clases en distintos lenguajes a partir de esta especificación.

```
1 openapi: 3.0.0
2 info:
3   title: Pet API
4   version: 1.0.0
5 paths:
6   /pets:
7     get:
8       summary: Listar mascotas
9       responses:
10        '200':
11          description: Lista de mascotas
12          content:
13            application/json:
14              schema:
15                type: array
16                items:
17                  $ref: '#/components/schemas/Pet '
18 components:
19   schemas:
20     Pet:
21       type: object
22       properties:
23         id:
24           type: integer
25           format: int64
26         name:
27           type: string
```

```

28     tag:
29     type: string

```

Ejemplo 8.1: Ejemplo de archivo api.yaml

Por ejemplo, para generar clases en Java, puedes ejecutar:

```

1 openapi-generator generate -i api.yaml -g java -o ./client

```

Esto creará una estructura de carpetas con las clases correspondientes a los modelos definidos en el archivo OpenAPI, así como los métodos para interactuar con la API.

```

1 public class Pet {
2     private Long id;
3     private String name;
4     private String tag;
5
6     public Pet(Long id, String name, String tag) {
7         this.id = id;
8         this.name = name;
9         this.tag = tag;
10    }
11
12    // Getters y setters
13    public Long getId() { return id; }
14    public void setId(Long id) { this.id = id; }
15
16    public String getName() { return name; }
17    public void setName(String name) { this.name = name; }
18
19    public String getTag() { return tag; }
20    public void setTag(String tag) { this.tag = tag; }
21 }

```

Ejemplo 8.2: Ejemplo de clase generada

Este proceso puede aplicarse a otros lenguajes (Python, C#, TypeScript, etc.) simplemente cambiando el parámetro `-g`. Así, la generación automática de código a partir de modelos facilita la implementación y reduce errores de transcripción.

Ejemplo: Generación de código a partir de un modelo Protobuf

Protocol Buffers (protobuf) es un sistema de serialización de datos desarrollado por Google, ampliamente utilizado para definir estructuras de datos y servicios en aplicaciones distribuidas. Para generar código, primero se define el modelo en un archivo `.proto`:

```

1 syntax = "proto3";

```

```
2
3 message Person {
4   int32 id = 1;
5   string name = 2;
6   string email = 3;
7 }
```

Ejemplo 8.3: Ejemplo de archivo .proto

Luego, se utiliza el compilador `protoc` para generar clases en el lenguaje deseado. Por ejemplo, para generar código Java:

```
1 | protoc --java_out=. person.proto
```

Esto crea una clase `Person` en Java con los atributos definidos en el archivo .proto, lista para ser utilizada en la aplicación. El código generado por `protoc` suele tener una estructura similar a la siguiente:

```
1 public final class Person extends
2   com.google.protobuf.GeneratedMessageV3 implements
3   // Use Java interfaces if needed
4   PersonOrBuilder {
5   private int id;
6   private String name;
7   private String email;
8
9   // Métodos de acceso, constructores y lógica generada por ↪
   Protobuf...
10 }
```

Ejemplo 8.4: Ejemplo de clase generada por Protobuf

Este proceso es similar para otros lenguajes como Python, C#, Go, etc., cambiando el parámetro de salida (`--python_out`, `--csharp_out`, etc.).

8.5.2. Ingeniería inversa

La ingeniería inversa es el proceso de analizar el código fuente existente para generar automáticamente los diagramas de clases que representan su estructura. Esto resulta útil para comprender sistemas heredados, documentar aplicaciones y facilitar la comunicación entre desarrolladores. Herramientas como Enterprise Architect, Visual Paradigm y StarUML ofrecen funciones de ingeniería inversa, permitiendo importar proyectos y visualizar las clases, relaciones y dependencias detectadas en el código.

En el contexto de la ingeniería inversa, una práctica común es analizar archivos compilados, como los ficheros `.jar` de Java, para comprender la estructura y el funcionamiento del código fuente original. Herramientas como IntelliJ IDEA permiten abrir y explorar archivos `.jar`, mostrando las clases y métodos

contenidos en ellos. IntelliJ incluye un decompilador integrado que traduce el bytecode de Java de vuelta a un código fuente legible, facilitando la revisión y documentación de bibliotecas de terceros o componentes sin acceso al código original.

Además de IntelliJ, existen decompiladores especializados como JD-GUI, CFR, Procyon y FernFlower, que permiten analizar archivos `.class` o `.jar` y obtener una representación aproximada del código fuente original. Estos decompiladores son útiles para estudiar el diseño de sistemas, realizar auditorías de seguridad o recuperar código perdido. Sin embargo, el código generado por los decompiladores puede no ser idéntico al original, especialmente si se han aplicado técnicas de ofuscación o si el código utiliza características avanzadas del lenguaje.

El uso de decompiladores y herramientas de ingeniería inversa debe realizarse respetando las licencias y derechos de autor del software analizado.

Consejo



El uso de técnicas como el Desarrollo Guiado por Pruebas (TDD, por sus siglas en inglés) permite realizar ingeniería inversa funcional de librerías externas sin necesidad de decompilar su código. Mediante TDD, se diseñan pruebas que verifican el comportamiento esperado de la librería, observando sus entradas y salidas públicas. A partir de estas pruebas, es posible implementar una versión propia que cumpla con la misma interfaz y funcionalidad, asegurando la compatibilidad con el sistema original. Este enfoque facilita la sustitución de dependencias externas por desarrollos propios, respetando la legalidad y evitando el acceso al código fuente original. Además, ayuda a comprender el diseño y los requisitos funcionales de la librería, promoviendo un desarrollo más seguro y controlado.

8.5.3. Limitaciones

Aunque estas técnicas son muy útiles, es importante tener en cuenta que la generación automática no siempre produce modelos o código completamente óptimos. Los diagramas generados por ingeniería inversa pueden ser complejos o contener detalles innecesarios, y el código generado desde diagramas suele requerir ajustes manuales. Se recomienda utilizar estas funciones como apoyo en el desarrollo y documentación, pero no como sustituto del diseño y la programación cuidadosa.

Resumen

En este capítulo se han revisado los conceptos clave de las clases, objetos y relaciones, así como la notación estándar utilizada en los diagramas UML. Se han presentado ejemplos prácticos de generación automática de código y de ingeniería inversa, mostrando cómo estas técnicas pueden acelerar el desarrollo y mejorar la documentación de los sistemas.

El uso de herramientas especializadas, como PlantUML, StarUML o Visual Paradigm, permite crear diagramas precisos y mantenerlos sincronizados con el código fuente. Además, la aplicación de buenas prácticas en la elaboración de diagramas contribuye a que sean claros, útiles y mantenibles a lo largo del ciclo de vida del software.

En resumen, dominar la elaboración y el análisis de diagramas de clases es esencial para diseñar sistemas robustos, facilitar la colaboración entre equipos y asegurar la calidad del software desarrollado.

Lecturas complementarias

A continuación se sugieren algunas lecturas y recursos adicionales para profundizar en el diseño y elaboración de diagramas de clases, así como en las técnicas de generación automática de código e ingeniería inversa:

- **UML Distilled: A Brief Guide to the Standard Object Modeling Language (Martin Fowler)**. Un libro esencial para comprender los fundamentos de UML y sus diagramas, con explicaciones claras y ejemplos prácticos.
- **Object-Oriented Analysis and Design with Applications (Grady Booch)**. Referencia clásica sobre análisis y diseño orientado a objetos, incluyendo el uso de diagramas de clases en el proceso de desarrollo.
- **Decompilers for Java**. Artículos y documentación sobre herramientas de decompilación como JD-GUI, CFR y Procyon: <https://github.com/java-decompiler/jd-gui>, <https://www.benf.org/other/cfr/>, <https://github.com/mstrobels/procyon>.
- **UML Diagrams**. Recurso en línea con ejemplos, explicaciones y plantillas de todos los tipos de diagramas UML: <https://www.uml-diagrams.org/>

Estas lecturas y recursos proporcionan una base sólida para ampliar conocimientos sobre el diseño orientado a objetos, el uso de diagramas UML y las técnicas de automatización en el desarrollo de software.

Elaboración de diagramas de comportamiento

Los diagramas de comportamiento en UML permiten modelar la dinámica de un sistema, es decir, cómo interactúan los elementos a lo largo del tiempo y ante diferentes estímulos. Los principales tipos de diagramas de comportamiento son:

- **Diagrama de casos de uso:** Representa las funcionalidades principales del sistema desde el punto de vista del usuario o actor externo. Es útil para identificar los requisitos y delimitar el alcance del sistema.
- **Diagrama de secuencia:** Muestra cómo los objetos interactúan entre sí mediante el envío de mensajes en un orden temporal específico. Es ideal para describir escenarios detallados de interacción.
- **Diagrama de comunicación:** Similar al diagrama de secuencia, pero enfatiza las relaciones entre los objetos y el flujo de mensajes, en vez del orden temporal.
- **Diagrama de actividad:** Modela los flujos de trabajo o procesos internos, mostrando actividades, decisiones y concurrencia. Es útil para representar algoritmos o procesos de negocio.
- **Diagrama de estados:** Describe los diferentes estados que puede tener un objeto y las transiciones entre ellos en respuesta a eventos. Se utiliza para modelar el comportamiento reactivo de componentes.

El campo de aplicación de estos diagramas abarca desde el análisis de requisitos hasta el diseño detallado y la documentación de sistemas software, permitiendo una mejor comunicación entre los miembros del equipo y con los clientes. Estos diagramas facilitan la identificación de posibles problemas en las fases tempranas del desarrollo, ayudan a clarificar y validar los requisitos funcionales y no funcionales, y sirven como referencia para la implementación y las pruebas.

Además, son útiles para la formación de nuevos integrantes del equipo, ya que proporcionan una visión clara y estructurada del funcionamiento del sistema. En proyectos complejos, los diagramas de comportamiento contribuyen a mantener la coherencia entre los distintos componentes y a gestionar los cambios de manera controlada.

Consejo



A lo largo de este capítulo, encontrarás varios diagramas, y junto a ellos, el código PlantUML utilizado para generarlos.

9.1. Diagrama de casos de uso

En el diagrama de casos de uso se representan las funcionalidades principales del sistema desde el punto de vista de los usuarios o actores externos. Este tipo de diagrama es fundamental para identificar los requisitos y delimitar el alcance del sistema.

9.1.1. Caso de uso

Un caso de uso representa una funcionalidad o servicio que el sistema proporciona a uno o más actores. Cada caso de uso describe una interacción completa entre los actores y el sistema para alcanzar un objetivo específico. Los casos de uso se identifican a partir de los requisitos del sistema y se documentan con una breve descripción, actores involucrados, escenarios principales y alternativos, y condiciones de inicio y fin.

En el diagrama, los casos de uso se representan mediante óvalos que contienen el nombre de la funcionalidad. Los actores se conectan a los casos de uso mediante líneas, indicando la participación en la funcionalidad. Es recomendable que los nombres de los casos de uso sean claros y reflejen acciones concretas, como «Registrar usuario», «Realizar pago» o «Consultar saldo».

9.1.2. Actores

Los actores son entidades externas al sistema que interactúan con él. Pueden ser personas, otros sistemas o dispositivos. Cada actor representa un rol específico en la interacción con el sistema, y no necesariamente corresponde a un usuario individual. Por ejemplo, en un sistema bancario, los actores pueden ser el cliente, el cajero y el sistema de verificación externa.

Los actores humanos se representan con un ícono de persona (*stick figure*). Los actores que son sistemas externos o dispositivos se representan con un rectángulo etiquetado con el estereotipo «actor».

9.1.3. Escenario

Un escenario describe una secuencia específica de acciones e interacciones entre los actores y el sistema para lograr un objetivo particular. Cada caso de uso puede tener uno o varios escenarios, incluyendo escenarios principales y alternativos. Los escenarios ayudan a detallar cómo se lleva a cabo una funcionalidad y permiten identificar excepciones o variantes en el flujo de trabajo. Los escenarios son descripciones narrativas y no tienen representación gráfica propia en el diagrama.

El **límite del sistema** (*subject o system boundary*) es el rectángulo que delimita el alcance del sistema y agrupa todos sus casos de uso. Muestra el nombre del sistema y separa visualmente el interior (el sistema) de los actores externos.

9.1.4. Relación de comunicación

Las relaciones de comunicación muestran cómo los actores interactúan con los casos de uso. Se representan mediante líneas que conectan a los actores con los casos de uso, indicando que existe una interacción o intercambio de información. Además, pueden existir relaciones entre casos de uso, como *include* (inclusión de funcionalidad común) y *extend* (extensión de comportamiento bajo ciertas condiciones).

Se representan mediante líneas sólidas que conectan actores y casos de uso, o entre casos de uso.

Conclusión

Estos elementos permiten construir diagramas claros y comprensibles que facilitan la comunicación entre los desarrolladores, analistas y clientes, asegurando que todos tengan una visión común de las funcionalidades del sistema.

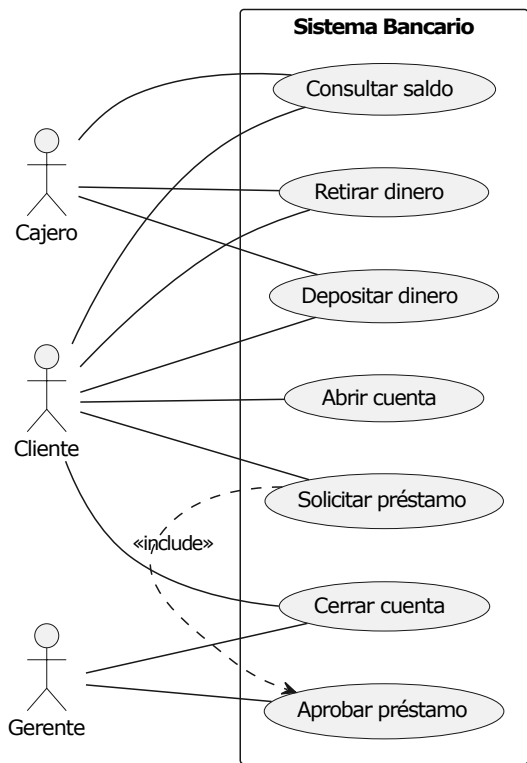
A continuación, se presenta un ejemplo de diagrama de casos de uso de un sistema bancario.

```

1 @startuml
2
3 left to right direction
4
5 actor Cliente
6 actor Cajero
7 actor Gerente

```

```
8
9 rectangle "Sistema Bancario" {
10     usecase "Abrir cuenta" as UC1
11     usecase "Depositar dinero" as UC2
12     usecase "Retirar dinero" as UC3
13     usecase "Consultar saldo" as UC4
14     usecase "Solicitar préstamo" as UC5
15     usecase "Aprobar préstamo" as UC6
16     usecase "Cerrar cuenta" as UC7
17 }
18
19 Cliente -- UC1
20 Cliente -- UC2
21 Cliente -- UC3
22 Cliente -- UC4
23 Cliente -- UC5
24 Cliente -- UC7
25
26 Cajero -- UC2
27 Cajero -- UC3
28 Cajero -- UC4
29
30 Gerente -- UC6
31 Gerente -- UC7
32
33 UC5 .> UC6 : <<include>>
34
35 @enduml
```



Ejercicio 9.1



Identifica al menos tres casos de uso para un sistema de gestión de biblioteca y representa su diagrama de casos de uso. Debe haber al menos un caso de uso que involucre a un administrador y otro que involucre a un usuario.

9.2. Diagrama de secuencia

En el diagrama de secuencia se representa cómo los objetos de un sistema interactúan entre sí a través del envío de mensajes en un orden temporal determinado. Este tipo de diagrama es útil para describir escenarios detallados de interacción, como el proceso de autenticación de un usuario o la realización de una transacción.

9.2.1. Línea de vida de un objeto

La línea de vida representa la existencia de un objeto durante la interacción. Se dibuja como una línea vertical bajo el nombre del objeto, y muestra el tiempo durante el cual el objeto participa en el escenario.

9.2.2. Activación

La activación indica el periodo en el que un objeto está realizando una acción o procesando un mensaje. Se representa mediante un rectángulo estrecho sobre la línea de vida del objeto.

9.2.3. Envío de mensajes

Los mensajes son las comunicaciones entre objetos. Se representan como flechas horizontales entre las líneas de vida, indicando el orden y el tipo de interacción (llamada de método, respuesta, etc.).

Conclusión

Los diagramas de secuencia permiten visualizar claramente el flujo de mensajes entre los objetos y el orden en que ocurren las interacciones. Son especialmente útiles para identificar responsabilidades, dependencias y posibles puntos de mejora en la lógica de negocio. Además, ayudan a detectar situaciones como llamadas recursivas, concurrencia o posibles cuellos de botella en el procesamiento. Utilizar diagramas de secuencia facilita la validación de requisitos y la comunicación entre los miembros del equipo de desarrollo.

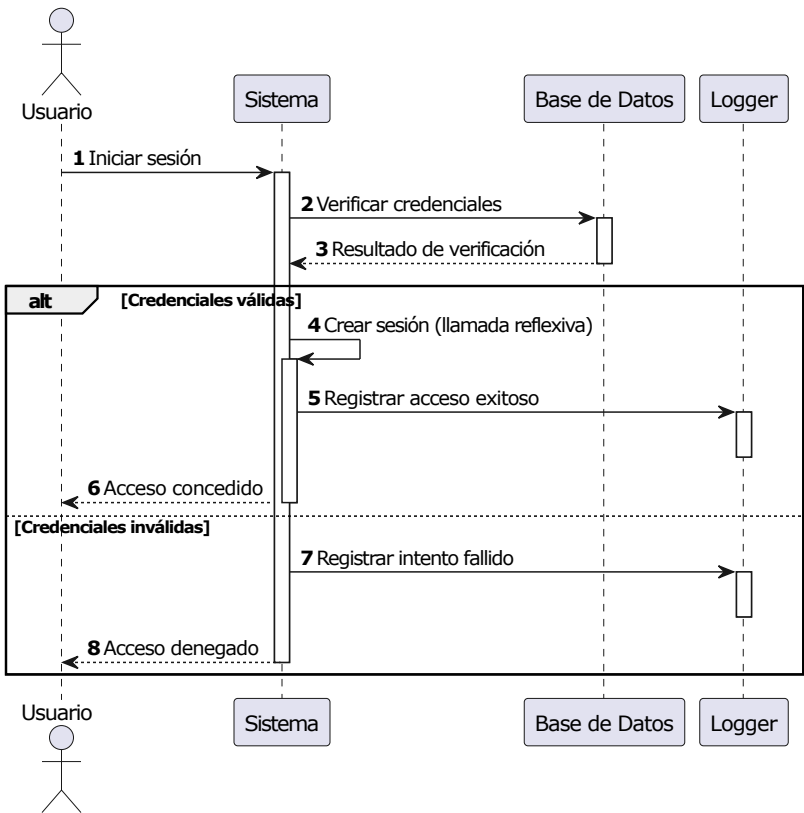
En este apartado hemos visto solo sus elementos principales, sin embargo, existen otros aspectos a considerar, como la gestión de excepciones, la sincronización entre objetos y la representación de condiciones temporales.

A continuación se muestra un ejemplo de diagrama de secuencia para el proceso de autenticación en un sistema bancario.

```
1 @startuml diagrama_secuencia
2
3 ' Muestra automáticamente la activación de participantes
4 autoactivate on
5 ' Muestra automáticamente los números de las interacciones
6 autonumber
7
8 actor Usuario
9 participant "Sistema" as S
10 participant "Base de Datos" as BD
11 participant "Logger" as L
12
13 ' Las llamadas se indican por un único guión
14 Usuario -> S: Iniciar sesión
15 S -> BD: Verificar credenciales
16 ' El retorno de una llamada se indica por un doble guión
17 BD --> S: Resultado de verificación
18 ' Esto representa un camino alternativo
```



```
19 alt Credenciales válidas
20   S -> S: Crear sesión (llamada reflexiva)
21   S -> L: Registrar acceso exitoso
22   deactivate L
23   S --> Usuario: Acceso concedido
24 else Credenciales inválidas
25   S -> L: Registrar intento fallido
26   deactivate L
27   S --> Usuario: Acceso denegado
28 end
29
30 @enduml
```



Ejercicio 9.2



Elabora un diagrama de secuencia para el proceso de préstamo de un libro en un sistema de gestión de biblioteca. Incluye al menos tres objetos y muestra el envío de mensajes entre ellos.

9.3. Diagrama de comunicación. Objetos, mensajes

El diagrama de comunicación (denominado diagrama de colaboración en versiones anteriores de UML) muestra cómo los objetos interactúan entre sí para llevar a cabo una funcionalidad. A diferencia del diagrama de secuencia, no se centra en el orden temporal de los mensajes, sino en las relaciones estructurales entre los objetos participantes.

9.3.1. Objetos y enlaces

Los objetos son instancias de clases que participan en la interacción. Se representan como rectángulos con el nombre del objeto subrayado, siguiendo la notación **nombre:Clase**. Los **enlaces** son las conexiones entre objetos a través de las cuales se envían los mensajes; se dibujan como líneas que unen los objetos.

9.3.2. Mensajes y numeración de secuencia

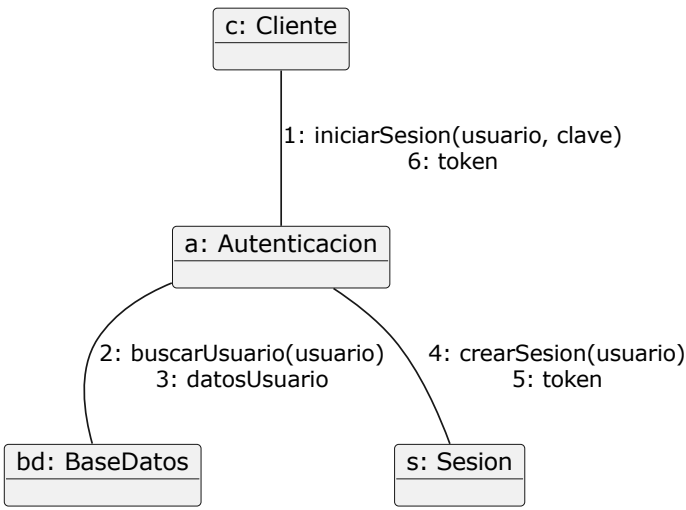
Los mensajes son las comunicaciones entre objetos y se representan como etiquetas sobre los enlaces, con una flecha que indica la dirección del envío. Cada mensaje lleva un **número de secuencia** que establece el orden de ejecución dentro de la colaboración. Se utiliza notación numérica simple (1, 2, 3...) para el flujo principal, y jerárquica (1.1, 1.2...) para mensajes anidados dentro de otro.

Conclusión

El diagrama de comunicación y el diagrama de secuencia son equivalentes: ambos representan la misma interacción, pero desde perspectivas distintas. El de secuencia destaca el orden temporal; el de comunicación destaca las relaciones entre objetos. La elección depende de qué aspecto se quiera comunicar: si el orden de los pasos o la red de dependencias entre objetos.

A continuación se muestra el mismo proceso de autenticación del ejemplo anterior, ahora representado como diagrama de comunicación.

```
1 @startuml
2 object "c: Cliente" as c
3 object "a: Autenticacion" as a
4 object "bd: BaseDatos" as bd
5 object "s: Sesion" as s
6
7 c -- a : "1: iniciarSesion(usuario, clave)\n6: token"
8 a -- bd : "2: buscarUsuario(usuario)\n3: datosUsuario"
9 a -- s : "4: crearSesion(usuario)\n5: token"
```



Ejercicio 9.3



Representa el proceso de préstamo de un libro en un sistema de gestión de biblioteca mediante un diagrama de comunicación. Incluye al menos cuatro objetos (por ejemplo, **usuario**, **biblioteca**, **catalogo** y **prestamo**) e indica la numeración de secuencia en cada mensaje.

9.4. Diagrama de actividad

El diagrama de actividad modela los flujos de trabajo o procesos de un sistema, mostrando la secuencia de acciones, las decisiones y la concurrencia. Es especialmente útil para representar algoritmos, procesos de negocio o casos de uso complejos.

9.4.1. Nodo de inicio y fin

El **nodo de inicio** marca el comienzo del flujo y se representa mediante un círculo relleno. El **nodo de fin** indica la terminación del proceso y se representa como un círculo relleno dentro de otro círculo. Un diagrama puede tener múltiples nodos de fin si el proceso puede terminar en distintos puntos.

9.4.2. Acción

Una **acción** representa una tarea o paso dentro del proceso. Se dibuja como un rectángulo con esquinas redondeadas que contiene una descripción breve de la actividad realizada.

9.4.3. Decisión

Un **nodo de decisión** representa un punto de bifurcación condicional. Se dibuja como un rombo del que parten varias transiciones, cada una etiquetada con la condición que debe cumplirse para seguir ese camino.

9.4.4. Bifurcación y unión

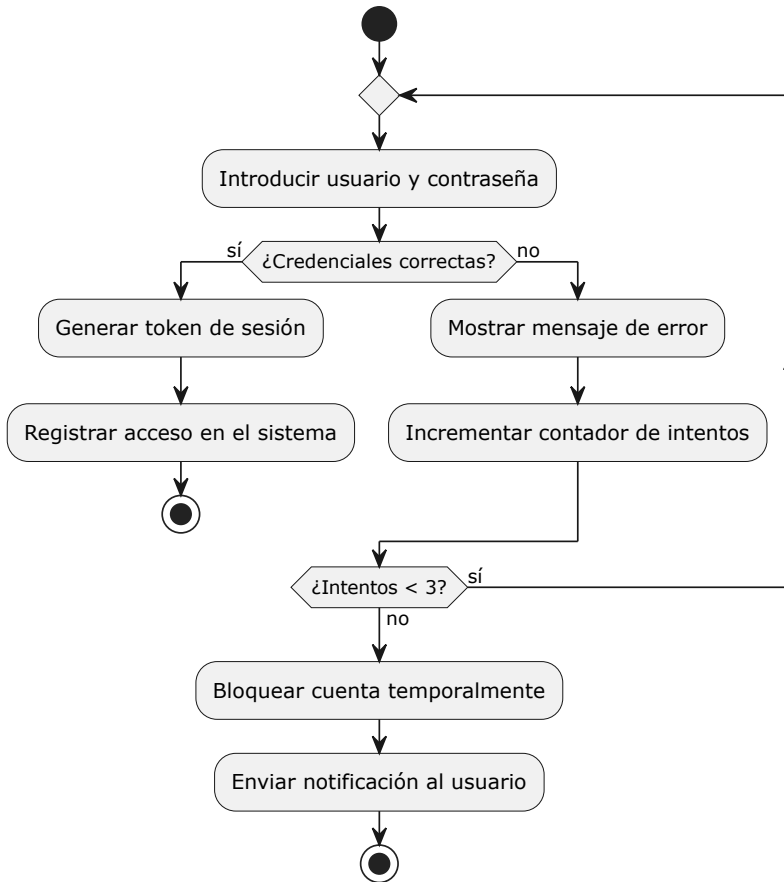
La **bifurcación** (*fork*) divide el flujo en varios hilos de ejecución paralelos y se representa con una barra horizontal gruesa. La **unión** (*join*) sincroniza varios hilos paralelos esperando a que todos completen antes de continuar, y se representa con la misma notación.

Conclusión

Los diagramas de actividad son muy versátiles: sirven tanto para modelar procesos de negocio de alto nivel como para detallar el flujo interno de un algoritmo. Frente a los diagramas de secuencia, son más adecuados cuando el flujo de control —decisiones, bucles y paralelismo— es el aspecto central que se quiere comunicar.

A continuación se muestra el flujo de actividad del proceso de inicio de sesión en un sistema bancario.

```
1 @startuml
2 start
3 repeat
4   :Introducir usuario y contraseña;
5   if (¿Credenciales correctas?) then (sí)
6     :Generar token de sesión;
7     :Registrar acceso en el sistema;
8     stop
9   else (no)
10    :Mostrar mensaje de error;
11    :Incrementar contador de intentos;
12  endif
13 repeat while (¿Intentos < 3?) is (sí) not (no)
14 :Bloquear cuenta temporalmente;
15 :Enviar notificación al usuario;
16 stop
```



Ejercicio 9.4



Elabora un diagrama de actividad para el proceso de reserva de un libro en una biblioteca. El proceso debe incluir al menos una decisión (por ejemplo, si el libro está disponible), una acción de notificación y un nodo de fin diferente según el resultado.

9.5. Diagramas de estados

El diagrama de estados (o diagrama de máquina de estados) describe los distintos estados en los que puede encontrarse un objeto a lo largo de su ciclo de vida y las transiciones entre esos estados provocadas por eventos. Se utiliza para modelar objetos cuyo comportamiento depende de su historial, como pedidos, cuentas de usuario o reservas.

9.5.1. Estado

Un **estado** representa una situación estable en la que se encuentra un objeto durante un período de tiempo. Se representa como un rectángulo con esquinas redondeadas que contiene el nombre del estado. Un objeto puede tener acciones asociadas a la entrada (*entry*), salida (*exit*) o permanencia en el estado (*do*).

9.5.2. Transición

Una **transición** es el cambio de un estado a otro. Se representa como una flecha entre dos estados, etiquetada con el evento que la desencadena. Opcionalmente puede incluir una condición de guarda entre corchetes y una acción que se ejecuta durante la transición.

9.5.3. Evento

Un **evento** es un estímulo que puede provocar una transición. Puede ser una llamada a un método, el paso del tiempo, una condición que se cumple o una señal recibida del entorno.

9.5.4. Estado inicial y final

El **estado inicial** se representa con un círculo relleno y marca el punto de partida del objeto al ser creado. El **estado final** se representa con un círculo relleno dentro de otro círculo e indica que el objeto ha concluido su ciclo de vida.

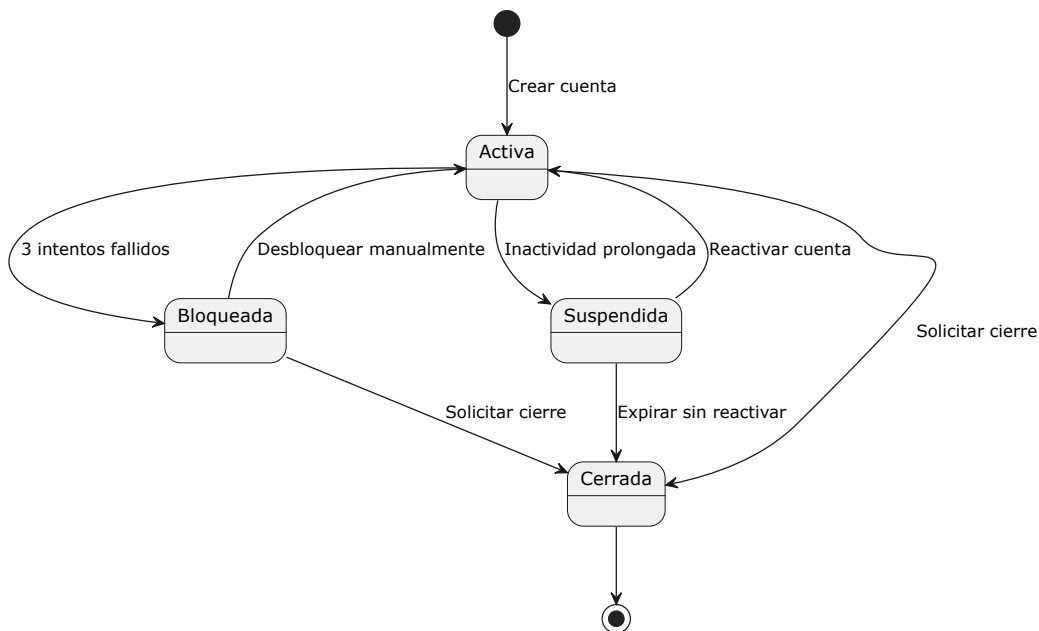
Conclusión

Los diagramas de estados son especialmente útiles para modelar objetos con comportamiento reactivo: cualquier entidad que responda de forma diferente a los mismos eventos según el estado en que se encuentre. Identificar correctamente los estados y las transiciones ayuda a evitar comportamientos inesperados en la implementación.

A continuación se muestra el diagrama de estados de una cuenta bancaria, con las transiciones entre sus posibles estados.

```
1 @startuml
2 [*] --> Activa : Crear cuenta
3
4 Activa --> Bloqueada : 3 intentos fallidos
5 Bloqueada --> Activa : Desbloquear manualmente
6 Activa --> Suspendida : Inactividad prolongada
7 Suspendida --> Activa : Reactivar cuenta
8 Activa --> Cerrada : Solicitar cierre
```

```
9 Bloqueada --> Cerrada : Solicitar cierre
10 Suspendida --> Cerrada : Expirar sin reactivar
11 Cerrada --> [*]
12 @enduml
```



Ejercicio 9.5



Elabora un diagrama de estados para un libro en un sistema de gestión de biblioteca. El libro puede estar disponible, reservado, prestado, en reparación o dado de baja. Define al menos cinco transiciones con sus eventos correspondientes.

Resumen

En este capítulo se han estudiado los principales diagramas de comportamiento de UML, que permiten modelar la dinámica de un sistema mostrando cómo interactúan sus elementos en respuesta a eventos y a lo largo del tiempo.

El **diagrama de casos de uso** permite identificar las funcionalidades del sistema desde el punto de vista de los actores externos, siendo el punto de partida habitual del análisis de requisitos. El **diagrama de secuencia** detalla cómo los objetos se comunican en un orden temporal preciso, lo que lo hace ideal para describir escenarios concretos. El **diagrama de comunicación** representa la misma información pero haciendo énfasis en las relaciones estructurales entre los objetos en lugar del orden de los mensajes.

El **diagrama de actividad** modela los flujos de trabajo mediante acciones, decisiones y concurrencia, siendo especialmente útil para representar procesos de negocio o algoritmos complejos. Por último, el **diagrama de estados** describe el ciclo de vida de un objeto a través de sus estados y las transiciones entre ellos provocadas por eventos.

Diagrama	Pregunta que responde	Uso principal
Casos de uso	¿Qué hace el sistema?	Análisis de requisitos
Secuencia	¿Cómo interactúan los objetos en el tiempo?	Diseño de escenarios
Comunicación	¿Qué objetos se relacionan?	Diseño de la red de objetos
Actividad	¿Cómo fluye el proceso?	Modelado de algoritmos y flujos
Estados	¿Cómo cambia el comportamiento con el estado?	Objetos con ciclo de vida

Cuadro 9.1: Comparativa de los diagramas de comportamiento UML

Chuleterio

Sintaxis PlantUML	Resultado
<i>Diagrama de clases</i>	
class Nombre	Clase
abstract class Nombre	Clase abstracta
interface Nombre	Interfaz
Nombre : +atributo : Tipo	Atributo público
Nombre : -atributo : Tipo	Atributo privado
Nombre : #atributo : Tipo	Atributo protegido
Nombre : +metodo() : Tipo	Método público
A < -- B	Herencia (B extiende A)
A < .. B	Realización (B implementa A)
A -->B	Asociación
A o-- B	Agregación
A *-- B	Composición
A «1» -->«0..*» B	Multiplicidad en relación
A -->B : etiqueta	Relación con etiqueta
<i>Diagrama de actividad</i>	
start / stop	Inicio y fin del flujo

Sintaxis PlantUML	Resultado
:acción;	Acción (nodo de actividad)
if (condición) then (sí)	Inicio de decisión
else (no) / endif	Rama alternativa y cierre
fork / fork again / end fork	Bifurcación paralela
<i>Diagrama de estados</i>	
[*] -->Estado	Transición inicial
Estado -->[*]	Transición final
A -->B : evento	Transición con evento
A -->B : evento [guarda] / acción	Transición completa
state «Nombre largo» as E	Alias para nombres con espacios
state E {...}	Estado compuesto
<i>Elementos comunes</i>	
title Mi diagrama	Título del diagrama
note right of X : texto	Nota a la derecha de X
' comentario	Comentario (no se muestra)
skinparam classBackgroundColor LightBlue	Personalización de color



Contenidos avanzados

TDD: Desarrollo guiado por pruebas

En el capítulo 5 aprendimos a escribir pruebas unitarias con JUnit 5 y a usar Mockito para crear dobles de prueba. En este capítulo damos un paso más: en lugar de escribir el código primero y las pruebas después, invertimos el orden. Escribiremos la prueba *antes* de que exista ningún código. Esta técnica se llama **desarrollo guiado por pruebas** (*Test-Driven Development*, TDD).

El resultado no es solo código probado — es código en el que confías al modificarlo, porque cada cambio queda respaldado por una batería de pruebas que escribiste tú mismo.

10.1. El problema

Vamos a implementar el método `esBalanceada(String)` de la clase `BalanceadorParentesis`. El método recibe una cadena y devuelve `true` si los paréntesis, corchetes y llaves están correctamente balanceados.

Entrada	Resultado esperado
«»	true
«(»	false
«()»	true
«(())»	true
«() [] {}»	true
«{ [] }»	true
«([)]»	false
«) (»	false

Cuadro 10.1: Casos de prueba para `esBalanceada`

Antes de escribir una sola línea de código de producción, necesitamos un test. Empezamos con dos ficheros vacíos en nuestro proyecto Java:

```
1 public class BalanceadorParentesis {  
2 }
```

Ejemplo 10.1: Punto de partida: clase de producción vacía

```
1 import org.junit.jupiter.api.Test;  
2 import static org.assertj.core.api.Assertions.*;  
3  
4 class BalanceadorParentesisTest {  
5 }
```

Ejemplo 10.2: Punto de partida: clase de test vacía

10.2. Iteración 1: el primer test rojo

¿Por dónde empezamos? Por el caso más sencillo de la tabla: la cadena vacía debe devolver `true`. Escribimos el test:

```
1 @Test  
2 void deberia_devolver_true_con_cadena_vacia() {  
3     assertThat(BalanceadorParentesis.esBalanceada("")).isTrue();  
4 }
```

Ejemplo 10.3: Iteración 1 — test rojo

Este código *no compila*: el método `esBalanceada` no existe todavía. Eso ya es un fallo — ya estamos en rojo. Creamos el método con el mínimo necesario para que compile:

```
1 public class BalanceadorParentesis {  
2     public static boolean esBalanceada(String cadena) {  
3         return false;  
4     }  
5 }
```

Ejemplo 10.4: Iteración 1 — código mínimo que compila y falla

Ahora compila, pero el test falla: esperábamos `true` y obtenemos `false`. Seguimos en rojo. Para pasar a verde, hacemos el cambio mínimo posible:

```
1 public class BalanceadorParentesis {  
2     public static boolean esBalanceada(String cadena) {  
3         return true;  
4     }  
5 }
```

Ejemplo 10.5: Iteración 1 — verde

El test pasa. Pasamos a verde con el mínimo esfuerzo posible, sin pensar en todos los casos futuros.

Importante

Las tres leyes de TDD (Robert C. Martin, *Uncle Bob*):

1. No escribas código de producción sin tener un test que falle.
2. No escribas más test de lo necesario para que falle (que no compile también es un fallo).
3. No escribas más código de producción de lo necesario para hacer pasar el test.

Estas tres reglas generan el ciclo **Rojo** → **Verde** → **Refactor**, que se repite en bucle durante todo el desarrollo.

10.3. Iteración 2: un paréntesis sin cerrar

Añadimos el siguiente caso: una cadena con un solo paréntesis abierto debe devolver `false`.

```

1 @Test
2 void deberia_devolver_false_con_parentesis_sin_cerrar() {
3     assertThat(BalanceadorParentesis.esBalanceada("(")).isFalse();
4 }

```

Ejemplo 10.6: Iteración 2 — test rojo

Ejecutamos. El test falla: nuestra implementación devuelve `true` para cualquier cosa. Necesitamos algo más de lógica. La solución más sencilla que se nos ocurre: contar las aperturas y cierres.

```

1 public class BalanceadorParentesis {
2     public static boolean esBalanceada(String cadena) {
3         int contador = 0;
4         for (char c : cadena.toCharArray()) {
5             if (c == '(') contador++;
6             if (c == ')') contador--;
7         }
8         return contador == 0;
9     }
10 }

```

Ejemplo 10.7: Iteración 2 — verde con contador

Ambos tests pasan: la cadena vacía devuelve `true` (contador permanece en 0) y «(» devuelve `false` (contador queda en 1). Verde.

Consejo

En TDD, la regla es escribir *el mínimo código* para pasar el test actual, no el código que creemos que necesitaremos en el futuro. Si en la siguiente iteración ese código se queda corto, lo ampliaremos entonces.

10.4. Iteración 3: el primer par y el paso Refactor

Añadimos el caso «()», que debe devolver true.

```

1 @Test
2 void deberia_devolver_true_con_un_par() {
3     assertThat(BalanceadorParentesis.esBalanceada("(")).isTrue();
4 }

```

Ejemplo 10.8: Iteración 3 — test

Ejecutamos: el test ya pasa con la implementación actual (el contador sube a 1 y baja a 0). Seguimos en verde sin tocar nada.

Es el momento del paso **Refactor**. Miramos el código y pensamos: el contador funciona para paréntesis, pero en las próximas iteraciones necesitaremos manejar también corchetes y llaves. Un contador simple no distingue entre `[]` — que debería ser inválido — y `()` — que es válido. La estructura adecuada para este problema es una **pila** (*stack*).

Refactorizamos la implementación para usar `Deque<Character>`. Todos los tests existentes deben seguir pasando:

```

1 import java.util.ArrayDeque;
2 import java.util.Deque;
3
4 public class BalanceadorParentesis {
5
6     public static boolean esBalanceada(String cadena) {
7         Deque<Character> pila = new ArrayDeque<>();
8         for (char c : cadena.toCharArray()) {
9             if (c == '(') {
10                 pila.push(c);
11             } else if (c == ')') {
12                 if (pila.isEmpty()) return false;
13                 pila.pop();
14             }
15         }
16         return pila.isEmpty();
17     }
18 }

```

Ejemplo 10.9: Iteración 3 — refactor a pila

Los tres tests siguen en verde. El comportamiento observable es idéntico; el diseño interno es mejor.

Importante

El paso **Refactor** no añade funcionalidad nueva. Su único objetivo es mejorar la estructura interna del código: legibilidad, claridad, eliminación de duplicación. La condición de éxito es que todos los tests sigan pasando al terminar.

10.5. Iteración 4: anidamiento y nombres expresivos

Añadimos el caso de paréntesis anidados: «`(( ))`» debe devolver `true`.

```
1 @Test
2 void deberia_devolver_true_con_parentesis_anidados() {
3     assertThat(BalanceadorParentesis.esBalanceada("(( ))").isTrue()
4     );
5 }
```

Ejemplo 10.10: Iteración 4 — test

El test pasa de inmediato: la pila maneja el anidamiento sin necesidad de cambios. Verde sin modificar la implementación.

Esta iteración nos da la oportunidad de hablar del **nombre del test**. Fíjate en `deberia_devolver_true_con_parentesis_anidados`: es largo, pero explica exactamente qué verifica. Cuando este test falle en el futuro, sabremos inmediatamente qué parte del comportamiento está rota, sin leer el cuerpo del método.

Consejo

Los nombres de los tests son documentación ejecutable. Escríbelos como frases: *debería devolver X cuando Y*. Cuando el test falle, el nombre del método es el primer mensaje que verás en el informe de errores.

10.6. Iteración 5: corchetes y llaves

Ahora incorporamos corchetes y llaves. Añadimos varios casos válidos. Para no repetir el mismo test tres veces con valores distintos, usamos `@ParameterizedTest`:

```
1 @ParameterizedTest
2 @ValueSource(strings = {"() [] {} ", "{ [] }", "({ }) "})
```

```

3 void deberia_devolver_true_con_tipos_mixtos_correctos(String ↩
  cadena) {
4   assertThat(BalanceadorParentesis.esBalanceada(cadena)).isTrue(↩
    );
5 }

```

Ejemplo 10.11: Iteración 5 — test parametrizado para casos válidos

Los tests fallan: nuestra implementación solo conoce paréntesis. Ampliamos la lógica para reconocer los tres tipos y verificar que el cierre corresponde con la apertura almacenada en la pila:

```

1 import java.util.ArrayDeque;
2 import java.util.Deque;
3
4 public class BalanceadorParentesis {
5
6   public static boolean esBalanceada(String cadena) {
7     Deque<Character> pila = new ArrayDeque<>();
8     for (char c : cadena.toCharArray()) {
9       if (c == '(' || c == '[' || c == '{') {
10        pila.push(c);
11      } else if (c == ')' || c == ']' || c == '}') {
12        if (pila.isEmpty()) return false;
13        char apertura = pila.pop();
14        if (!corresponde(apertura, c)) return false;
15      }
16    }
17    return pila.isEmpty();
18  }
19
20  private static boolean corresponde(char apertura, char cierre) ↩
  {
21    return (apertura == '(' && cierre == ')')
22      || (apertura == '[' && cierre == ']')
23      || (apertura == '{' && cierre == '}');
24  }
25 }

```

Ejemplo 10.12: Iteración 5 — implementación con los tres tipos

Todos los tests pasan. Verde.

Importante

`@ParameterizedTest` con `@ValueSource` ejecuta el mismo test con cada valor de la lista. Es la forma recomendada de probar el mismo comportamiento con múltiples entradas sin duplicar el cuerpo del test.

10.7. Iteración 6: tipos mezclados incorrectamente

El último bloque de casos: combinaciones inválidas como «([)]» o «)(».

De nuevo usamos `@ParameterizedTest`:

```

1 @ParameterizedTest
2 @ValueSource(strings = {"([)]", ")(", "{[]", "((("})
3 void deberia_devolver_false_con_tipos_mezclados_incorrectos(↵
    String cadena) {
4     assertThat(BalanceadorParentesis.esBalanceada(cadena)).isFalse(↵
        (↵
5 }

```

Ejemplo 10.13: Iteración 6 — test para casos inválidos

Los tests pasan: la lógica de **corresponde** ya rechaza «([)]», y `pila.isEmpty()` ya detecta las cadenas incompletas. Verde sin cambios.

Es el momento del **refactor final**. Revisamos el código con ojos críticos: ¿hay algo confuso, redundante o difícil de leer? En este caso, el código ya es limpio. No tocamos nada.

Consejo



Si al llegar al refactor final no hay nada que mejorar, no inventes refactorizaciones. El objetivo del paso Refactor es eliminar problemas reales, no aplicar cambios por costumbre.

Hemos completado las ocho filas de la tabla inicial. Cada caso tiene su test, todos en verde.

10.8. Código final

Tras seis iteraciones, el código de producción y los tests quedan así:

```

1 import java.util.ArrayDeque;
2 import java.util.Deque;
3
4 public class BalanceadorParentesis {
5
6     public static boolean esBalanceada(String cadena) {
7         Deque<Character> pila = new ArrayDeque<>();
8         for (char c : cadena.toCharArray()) {
9             if (c == '(' || c == '[' || c == '{') {
10                 pila.push(c);
11             } else if (c == ')' || c == ']' || c == '}') {

```

```

12         if (pila.isEmpty()) return false;
13         char apertura = pila.pop();
14         if (!corresponde(apertura, c)) return false;
15     }
16 }
17 return pila.isEmpty();
18 }
19
20 private static boolean corresponde(char apertura, char cierre)↵
21 {
22     return (apertura == '(' && cierre == ')')
23     || (apertura == '[' && cierre == ']')
24     || (apertura == '{' && cierre == '}');
25 }

```

Ejemplo 10.14: BalanceadorParentesis — código de producción completo

```

1 import org.junit.jupiter.api.Test;
2 import org.junit.jupiter.params.ParameterizedTest;
3 import org.junit.jupiter.params.provider.ValueSource;
4 import static org.assertj.core.api.Assertions.*;
5
6 class BalanceadorParentesisTest {
7
8     @Test
9     void deberia_devolver_true_con_cadena_vacia() {
10         assertThat(BalanceadorParentesis.esBalanceada("")).isTrue();
11     }
12
13     @Test
14     void deberia_devolver_false_con_parentesis_sin_cerrar() {
15         assertThat(BalanceadorParentesis.esBalanceada("(")).isFalse↵
16         ();
17     }
18
19     @Test
20     void deberia_devolver_true_con_un_par() {
21         assertThat(BalanceadorParentesis.esBalanceada("(")↵
22         ));
23     }
24
25     @Test
26     void deberia_devolver_true_con_parentesis_anidados() {
27         assertThat(BalanceadorParentesis.esBalanceada("↵
28         (())"));
29     }
30
31     @ParameterizedTest
32     @ValueSource(strings = {"() [] {}", "{[]}", "([{}])"})

```

```

30 void deberia_devolver_true_con_tipos_mixtos_correctos(String cadena) {
31     assertThat(BalanceadorParentesis.esBalanceada(cadena)).
32     assertTrue();
33 }
34
35 @ParameterizedTest
36 @ValueSource(strings = {"(())", "()((", "{[]", "(((("))
37 void deberia_devolver_false_con_tipos_mezclados_incorrectos(
38 String cadena) {
39     assertThat(BalanceadorParentesis.esBalanceada(cadena)).
40     assertFalse();
41 }

```

Ejemplo 10.15: BalanceadorParentesisTest — clase de test completa

10.9. Ejercicios

Practica el ciclo Rojo → Verde → Refactor con las siguientes katas. En cada una, empieza por escribir el primer test antes de crear la clase.

Ejercicio 10.1



Kata: validador de email

Implementa el método `static boolean esEmail(String texto)` que devuelve `true` si la cadena tiene formato de email válido básico (contiene exactamente un `@`, tiene al menos un carácter antes del `@` y al menos un punto después).

Entrada	Resultado esperado
«usuario@dominio.com»	true
«a@b.es»	true
«sinArroba»	false
«@dominio.com»	false
«doble@@dominio.com»	false
«usuario@sinpunto»	false
«»	false

Aplica TDD: escribe el test más sencillo primero, haz que pase con el mínimo código posible y añade un test nuevo en cada iteración.

Ejercicio 10.2



Kata: palíndromo

Implementa el método `static boolean esPalindromo(String texto)` que devuelve `true` si la cadena se lee igual de izquierda a derecha que de derecha a izquierda, ignorando mayúsculas y espacios.

Entrada	Resultado esperado
«ana»	true
«Ana»	true
«Anita lava la tina»	true
«hola»	false
«»	true
«a»	true

Ejercicio 10.3



Kata: contar palabras

Implementa el método `static int contarPalabras(String texto)` que devuelve el número de palabras de la cadena. Las palabras están separadas por uno o más espacios.

Entrada	Resultado esperado
«»	0
«hola»	1
«hola mundo»	2
« hola mundo »	2
«uno dos tres cuatro»	4

Ejercicio 10.4



Kata: número primo

Implementa el método `static boolean esPrimo(int n)` que devuelve `true` si `n` es un número primo.

Entrada	Resultado esperado
2	true
3	true
7	true
13	true
1	false
4	false
9	false
0	false

Pista: empieza por los casos límite (0, 1, 2) antes de atacar el algoritmo general.

Resumen

TDD es una disciplina de desarrollo, no una técnica de testing. La diferencia es sutil pero importante: no se trata de escribir pruebas exhaustivas al final, sino de dejar que las pruebas guíen el diseño del código desde el primer momento.

El ciclo **Rojo** → **Verde** → **Refactor** se repite en iteraciones cortas: primero un test que falla, luego el mínimo código para hacerlo pasar, y finalmente la limpieza del código sin romper nada. Las tres leyes de Uncle Bob formalizan este ritmo.

Lo que TDD *no es*: escribir todos los tests al principio y luego implementar; añadir tests a código ya escrito para aumentar la cobertura; o saltarse el paso Refactor porque “el código ya funciona”.

El resultado, después de suficientes iteraciones, es un sistema de pruebas que actúa como red de seguridad: puedes modificar, refactorizar o ampliar el código con confianza, porque los tests te dirán inmediatamente si algo deja de funcionar.

- **Test-Driven Development: By Example**, Kent Beck, Addison-Wesley, 2002.

La referencia original. Corto, directo y con ejemplos en Java y Python.

Uso de inteligencia artificial generativa

El papel del programador no ha desaparecido — ha evolucionado. Antes, cuando necesitabas saber cómo implementar un algoritmo de ordenación, buscabas en Stack Overflow o en la documentación oficial. Ahora puedes describir el problema en lenguaje natural y recibir código funcional en segundos.

Esta forma de trabajar no elimina la necesidad de entender el código. Al contrario: saber evaluar, depurar y mejorar el código que genera la IA requiere exactamente los mismos conocimientos que siempre. Las pruebas que aprendiste a escribir en el capítulo anterior son ahora tu herramienta más valiosa para verificar lo que la IA produce.

En este capítulo aprenderás cómo funcionan los modelos de lenguaje a nivel conceptual, qué herramientas de IA están disponibles para el desarrollo, y cómo integrarlas eficazmente en tu flujo de trabajo usando el **marco de las 4D**: Delegación, Descripción, Discernimiento y Diligencia.

11.1. Cómo funciona un modelo de lenguaje

Un modelo de lenguaje grande (*Large Language Model*, LLM) es un sistema de inteligencia artificial entrenado con enormes cantidades de texto: código fuente, libros, artículos y documentación técnica. A partir de ese entrenamiento, el modelo aprende patrones estadísticos sobre qué palabras — o fragmentos de palabras — suelen aparecer juntas.

Tokens

El modelo no trabaja con palabras enteras, sino con **tokens**: unidades mínimas de texto que pueden ser palabras completas, partes de palabras o incluso signos de puntuación. La palabra «programación» puede representarse como uno o varios tokens, según el vocabulario del modelo. El código fuente también se

tokeniza; por eso algunos identificadores largos o en *camelCase* pueden producir comportamientos inesperados en el modelo.

Contexto

Cuando le haces una pregunta a un asistente de IA, el modelo recibe como entrada todo el texto de la conversación hasta ese momento: tu pregunta, el historial de mensajes anteriores y cualquier instrucción del sistema. Este bloque de texto es el **contexto**. Los modelos tienen un límite en la cantidad de contexto que pueden procesar a la vez (la *ventana de contexto*); cuando la conversación es muy larga, los mensajes más antiguos pueden quedar fuera.

Predicción del siguiente token

La generación de texto funciona mediante la predicción repetida del siguiente token: el modelo recibe el contexto y calcula una distribución de probabilidad sobre los posibles tokens siguientes, de la que extrae uno. Añade ese token al texto generado y vuelve a predecir el siguiente. El resultado es una secuencia de tokens que, leída en conjunto, parece lenguaje natural — o código Java bien formado.

Este mecanismo explica por qué la IA puede equivocarse aparentando total confianza: no está razonando sobre si el código es correcto, sino generando lo que estadísticamente parece correcto dado el contexto. Un método con un bug sutil es, desde el punto de vista del modelo, igual de probable que uno correcto.

Importante

La IA no sabe si lo que genera es correcto. El modelo produce el texto más probable a partir de su entrenamiento, no necesariamente el código más correcto. Siempre verifica el código que genera: compílalo, ejecútalo y pásalo por tus pruebas antes de aceptarlo.

11.2. Herramientas de IA para el desarrollo

Existen varias herramientas de IA orientadas al desarrollo de software. La tabla 11.1 resume las cuatro más utilizadas en el momento de escribir este libro.

11.2.1. GitHub Copilot

GitHub Copilot es un plugin de IA desarrollado por GitHub (Microsoft) y OpenAI. Se instala como extensión en VS Code o IntelliJ IDEA y ofrece auto-

Herramienta	Tipo	Plan gratuito	Punto fuerte
GitHub Copilot	Plugin IDE	Sí (limitado)	Autocompletado inline
Cursor	IDE propio	Sí (limitado)	Modo agente multi-fichero
Claude	Web + CLI	Sí	Conversación y terminal
Zed	IDE propio	Sí	IA integrada nativa

Cuadro 11.1: Herramientas de IA para el desarrollo de software

completado inline directamente en el editor: mientras escribes el nombre de un método o un comentario que describe su intención, Copilot sugiere la implementación completa que puedes aceptar con la tecla **Tab**.

Para acceder al plan gratuito, crea una cuenta en github.com e instala la extensión *GitHub Copilot* desde el marketplace de VS Code o desde el menú de plugins de IntelliJ IDEA. La versión gratuita incluye un número limitado de sugerencias mensuales y acceso al chat de Copilot integrado en el editor.

Consejo



Copilot funciona mejor cuando el contexto del fichero es rico: buenos nombres de variables, comentarios que describan la intención, y métodos cortos con responsabilidad única. Los principios de código limpio que ya conoces también mejoran la calidad de las sugerencias de la IA.

11.2.2. Cursor

Cursor es un editor de código basado en VS Code, con IA integrada a nivel nativo. Además del autocompletado inline similar a Copilot, incluye un *modo agente* que puede editar múltiples ficheros a la vez, ejecutar comandos en el terminal y resolver tareas completas a partir de una descripción en lenguaje natural.

Para usarlo, descarga e instala Cursor desde cursor.com. No requiere plugins adicionales; la IA está integrada desde el primer momento. El plan gratuito incluye un número limitado de peticiones mensuales al modelo más potente y peticiones ilimitadas a modelos más ligeros.

11.2.3. Claude

Claude es el asistente de IA de Anthropic. Está disponible en dos modalidades: la interfaz web claude.ai, donde puedes mantener conversaciones y pegar código; y **Claude Code**, una herramienta de línea de comandos que se ejecuta

directamente en tu terminal y puede leer, escribir y modificar ficheros de tu proyecto.

Para acceder al plan gratuito, crea una cuenta en `claude.ai`. Claude Code se instala con `npm install -g @anthropic-ai/claude-code` y requiere autenticación con tu cuenta de Anthropic. Al ser independiente del editor, funciona con cualquier entorno de desarrollo.

11.2.4. Zed

Zed es un editor de código de alto rendimiento diseñado para la colaboración en tiempo real y la integración nativa de IA. A diferencia de Copilot — un plugin — o Cursor — basado en VS Code —, Zed está construido desde cero con la IA como parte central de su arquitectura.

Para descargarlo, visita `zed.dev`. En macOS y Linux está disponible directamente; la versión para Windows está disponible aunque con menor madurez que en macOS y Linux. El editor es gratuito y la integración con modelos de IA incluye un plan gratuito con créditos mensuales.

11.3. El marco de las 4D

El **marco de las 4D** es un esquema de cuatro principios que guían una colaboración eficaz entre el programador y la IA: **Delegación**, **Descripción**, **Discernimiento** y **Diligencia**. No es una receta rígida, sino un modelo mental para tomar mejores decisiones cuando trabajas con herramientas de IA.

11.3.1. Delegación (*Delegation*)

La delegación consiste en decidir qué tareas encargar a la IA y cuáles mantener bajo tu propio control.

La IA es especialmente útil para tareas repetitivas y bien definidas: generar código *boilerplate*, convertir estructuras de datos, implementar métodos CRUD, escribir Javadoc o traducir código de un lenguaje a otro. Son tareas donde el patrón es claro y el resultado es fácil de verificar.

En cambio, la IA no es fiable para lógica de negocio compleja, código de seguridad crítica o algoritmos que no puedes verificar con tests. Si aceptas código que no entiendes, estás delegando también tu responsabilidad — y eso es un problema cuando algo falla.

Consejo

Antes de pedir algo a la IA, pregúntate: ¿podré verificar el resultado? Si la respuesta es no, considera si es el tipo de tarea adecuada para delegar.

11.3.2. Descripción (*Description*)

La descripción es el arte de comunicar a la IA lo que necesitas de forma precisa y eficaz. Un *prompt* vago produce código vago; un *prompt* bien formulado produce código útil.

Un *prompt* efectivo incluye tres elementos:

- **Producto** (*Product*): qué resultado quieres exactamente — tipo de retorno, nombre del método, comportamiento esperado.
- **Proceso** (*Process*): cómo debe lograrlo — restricciones, bibliotecas a usar o evitar, casos especiales a tratar.
- **Rendimiento** (*Performance*): criterios de calidad — estilo de código, cobertura de tests, gestión de errores.

Compara estos dos *prompts* para la misma tarea:

```
1 | Haz un método que valide emails.
```

Ejemplo 11.1: Prompt poco efectivo

```
1 | Escribe un método Java estático que valide si una cadena tiene
2 | formato básico de correo electrónico.
3 |
4 | Producto:
5 | - Firma: public static boolean esEmailValido(String email)
6 | - Devuelve true si el formato es válido, false en caso contrario
7 |
8 | Proceso:
9 | - Una dirección válida contiene exactamente un '@'
10 | - Después del '@' debe haber al menos un '.'
11 | - No puede ser null ni estar vacía
12 | - Usa solo la biblioteca estándar de Java (sin dependencias ↪
    |   externas)
13 |
14 | Rendimiento:
15 | - Incluye tres casos de prueba JUnit 5: email válido,
16 |   email sin @, email sin punto después del @
```

Ejemplo 11.2: Prompt efectivo aplicando Producto-Proceso-Rendimiento

11.3.3. Discernimiento (*Discernment*)

El discernimiento es la capacidad de evaluar críticamente el código que genera la IA antes de aceptarlo. No basta con que compile — hay que verificar que hace lo que se supone que debe hacer.

Las preguntas que conviene hacerse ante cualquier código generado por IA:

- ¿Compila sin errores ni *warnings*?
- ¿Pasan los tests existentes?
- ¿El código hace exactamente lo que pedí, ni más ni menos?
- ¿Hay algún caso límite que no se esté manejando?
- ¿El código tiene vulnerabilidades de seguridad evidentes?

Aquí es donde las pruebas del capítulo 10 se vuelven imprescindibles. El ciclo TDD funciona igual de bien con código generado por IA: primero defines el comportamiento esperado en un test, luego pides a la IA que implemente el método, y finalmente verificas que el test pasa — y que no pasaba antes de la implementación.

11.3.4. Diligencia (*Diligence*)

La diligencia se refiere al uso ético y responsable de la IA. Incluye cuatro aspectos:

- **Transparencia:** si usas IA para generar código que entregas como trabajo propio, sé honesto sobre ello cuando se te pregunte. En un entorno profesional, el equipo debe saber qué partes del código fueron generadas por IA.
- **Verificación antes de compartir:** nunca compartas código generado por IA que no hayas revisado y probado tú mismo. Tú eres responsable del código que lleva tu nombre.
- **Copyright:** el código que genera la IA puede reproducir fragmentos de código con licencias restrictivas de su conjunto de entrenamiento. Si trabajas en un proyecto con requisitos de licencia estrictos, consulta la política del proveedor de IA que estés usando.
- **Datos sensibles:** nunca envíes a un modelo externo información que no deba salir de tu organización: contraseñas, claves de API, datos personales de clientes o secretos de negocio.

11.4. La IA en el flujo de desarrollo

Las herramientas de IA se integran en las cuatro tareas principales del ciclo de desarrollo: escribir código nuevo, revisar y mejorar el código existente, generar pruebas y producir documentación. En cada caso, el patrón es el mismo: describir la tarea con precisión (Descripción), obtener el resultado y verificarlo con criterio (Discernimiento).

11.4.1. Escribir código nuevo

La forma más directa de usar la IA para escribir código es mediante el auto-completado inline: mientras escribes el nombre de un método o un comentario que describe su intención, la herramienta sugiere la implementación completa.

También puedes iniciar la conversación con un prompt en el chat del asistente, proporcionando el contexto necesario:

```

1 Estoy implementando un gestor de tareas en Java.
2 Necesito una clase GestorTareas con los siguientes métodos:
3
4 - void agregar(String tarea): añade una tarea a la lista como ↪
  pendiente
5 - void completar(int indice): marca la tarea en esa posición ↪
  como completada
6 - List<String> listar(): devuelve todas las tareas precedidas ↪
  por su estado
7 ("[x]" si completada, "[ ]" si pendiente)
8 - int contarPendientes(): devuelve el número de tareas no ↪
  completadas
9
10 Usa ArrayList internamente. No uses librerías externas.
11 Incluye tres tests JUnit 5 que cubran el caso base de cada ↪
  método.
```

Ejemplo 11.3: Prompt para implementar una nueva clase

Consejo



Retoma el ciclo TDD incluso con código generado por IA: escribe primero el test que describe el comportamiento esperado, pide luego a la IA que implemente el método, y verifica que el test pasa. Así te aseguras de que el código hace lo que *tú* quieres, no lo que la IA interpreta.

11.4.2. Revisar y mejorar código

Cuando tienes código que funciona pero sospechas que podría mejorar, la IA puede ayudarte a identificar problemas: nombres poco descriptivos, lógica duplicada, métodos demasiado largos o vulnerabilidades de seguridad.

El prompt más útil para revisión de código es directo y proporciona el fichero completo como contexto:

```
1 Revisa este código Java e identifica los siguientes tipos de ↩
  problemas:
2 - Nombres de variables o métodos poco descriptivos
3 - Lógica duplicada
4 - Posibles bugs (incluyendo casos límite no manejados)
5 - Problemas de seguridad evidentes
6
7 Para cada problema, explica por qué es un problema
8 y propón cómo lo corregirías.
9
10 [pegar el código aquí]
```

Ejemplo 11.4: Prompt de revisión de código

El resultado será una lista de observaciones. Tu trabajo como programador es evaluar cada una: ¿el problema que señala es real? ¿la solución propuesta es la mejor? ¿ha pasado algo por alto?

11.4.3. Escribir tests

La IA puede generar suites de tests JUnit 5 para una clase existente. Es especialmente útil para identificar casos límite que quizás no habías considerado.

```
1 Genera pruebas JUnit 5 con AssertJ para la siguiente clase Java.
2
3 Requisitos:
4 - Cubre el caso base, al menos dos casos límite por método y un ↩
  caso de error
5 - Cada método de test debe tener un nombre descriptivo en ↩
  snake_case
6 - Usa @ParameterizedTest donde tenga sentido
7 - Los tests deben ser independientes entre sí
8
9 [pegar la clase aquí]
```

Ejemplo 11.5: Prompt para generar tests JUnit 5

Importante

Antes de aceptar los tests generados por la IA, ejecútalos. Verifica que los tests que deberían fallar con una implementación vacía efectivamente fallan — un test que pasa siempre, incluso sin código, no está probando nada.

11.4.4. Documentar

La IA genera documentación técnica con facilidad: Javadoc para métodos y clases, comentarios inline que explican lógica compleja, y ficheros README para proyectos.

```

1 Genera el Javadoc completo para todos los métodos públicos
2 de esta clase Java. Incluye @param, @return y @throws donde ↪
  corresponda.
3 La documentación debe describir el comportamiento observable del ↪
  método,
4 no los detalles de implementación.
5
6 [pegar la clase aquí]
```

Ejemplo 11.6: Prompt para generar Javadoc completo

Importante

Revisa siempre que la documentación generada describe lo que el código *realmente* hace. La IA puede documentar el comportamiento que parece probable a partir de los nombres, no el comportamiento real de la implementación. Un Javadoc que dice «devuelve el índice del elemento» cuando el método devuelve -1 si no lo encuentra es peor que no tener documentación.

11.5. Limitaciones y riesgos

Usar IA para programar tiene riesgos reales que conviene conocer antes de depender de ella en un proyecto.

Alucinaciones

Los modelos de lenguaje pueden generar código que compila y tiene el aspecto correcto, pero que hace algo distinto de lo que pediste — o directamente incorrecto. A esto se le llama *alucinación*. El código puede usar una API que

no existe, implementar un algoritmo con un error sutil, o devolver un resultado incorrecto en casos específicos.

Por ejemplo, un modelo de lenguaje puede generar este método para calcular si un año es bisiesto:

```
1 public static boolean esBisiesto(int anio) {  
2     return anio % 4 == 0; // incorrecto: los años múltiplos de 100  
3         // no son bisiestos salvo que sean múltiplos de 400  
4 }
```

Ejemplo 11.7: Ejemplo de alucinación: implementación incompleta generada por IA

El código compila, produce resultados correctos para la mayoría de los años (1996, 2000, 2024...) y solo falla para años como 1900 o 2100. Sin tests que cubran esos casos límite, el bug pasaría desapercibido.

Código inseguro

La IA puede generar código con vulnerabilidades de seguridad. Las más frecuentes son:

- **Inyección SQL:** construir consultas SQL concatenando texto de entrada del usuario en lugar de usar sentencias preparadas.
- **Credenciales en el código fuente:** incluir contraseñas, claves de API o tokens directamente como literales en el código.
- **Ausencia de validación:** aceptar datos de entrada del usuario sin comprobar su formato o rango antes de usarlos.

Dependencia excesiva

Aceptar código sin entenderlo crea una dependencia peligrosa: cuando el código falle — y fallará — no sabrás por dónde empezar a depurarlo. La IA es una herramienta que te ayuda a producir código más rápido, no un sustituto de la comprensión.

Si no puedes explicar en voz alta lo que hace cada línea del código que aceptas, dedica unos minutos a leerlo antes de incorporarlo al proyecto.

Copyright y licencias

Los modelos de lenguaje se entrenan con código disponible en internet, parte del cual tiene licencias restrictivas (GPL, AGPL). Es posible que el código generado reproduzca fragmentos literales de ese código. Si trabajas en un proyecto

con requisitos de licencia estrictos, consulta la política de uso del proveedor de IA que estés usando.

11.6. Ejercicios

Ejercicio 11.1



Implementar y comparar: GestorTareas

Implementa la clase `GestorTareas` con la siguiente API:

- `void agregar(String tarea)` — añade una tarea a la lista como pendiente.
- `void completar(int indice)` — marca la tarea en esa posición como completada.
- `List<String>listar()` — devuelve todas las tareas precedidas por su estado: `[x]` si completada, `[ ]` si pendiente.
- `int contarPendientes()` — devuelve el número de tareas aún no completadas.

Parte 1 — Sin IA (10-15 min): Implementa la clase a mano. Aplica el ciclo TDD: escribe al menos un test JUnit 5 por método antes de implementarlo.

Parte 2 — Con IA: Describe el problema a tu herramienta de IA aplicando el modelo Producto-Proceso-Rendimiento del marco 4D. Pide también los tests.

Reflexión: Compara las dos implementaciones. ¿Qué diferencias hay? ¿Qué tendrías que verificar o cambiar en la versión de la IA antes de aceptarla? ¿Qué casos no cubre alguna de las dos?

Ejercicio 11.2



Revisar y corregir: clase `Validador` con errores

La siguiente clase `Validador` contiene cuatro problemas deliberados. Encuéntralos y corrígelos.

```
1 public class Validador {
2
3     public static boolean esEmail(String s) {
4         if (s == null || s.isEmpty()) return false;
5         if (s.contains("@") && s.contains("@")) // problema 1
```

```

6         return s.contains(".");           // problema 2
7     return false;
8 }
9
10 public static boolean tieneLongitudMinima(String texto, ↵
11 int min) {
12     return texto.length() > min;         // problema 3
13 }
14
15 public static boolean contrasenaSegura(String pw) {
16     boolean tieneMayuscula = false;
17     boolean tieneDigito = false;
18     for (char c : pw.toCharArray()) {
19         if (Character.isUpperCase(c)) tieneMayuscula = true;
20         if (Character.isDigit(c)) tieneMayuscula = true; // ↵
21     }
22     return tieneMayuscula && tieneDigito;
23 }

```

Ejemplo 11.8: Clase Validador con errores para revisar

Parte 1 — Sin IA: Identifica los cuatro problemas y corrégelos. Para cada uno, escribe un test JUnit 5 que falle con el código original y pase con tu corrección.

Parte 2 — Con IA: Pide a la IA que revise la clase y señale los problemas. ¿Encontró los mismos que tú? ¿Señaló algo que tú no viste? ¿Se equivocó en alguna observación?

Ejercicio 11.3



Generar tests: clase CalculadoraImpuestos

La siguiente clase está ya implementada y lista para usar:

```

1 public class CalculadoraImpuestos {
2
3     public static final double IVA_GENERAL      = 0.21;
4     public static final double IVA_REDUCIDO     = 0.10;
5     public static final double IVA_SUPERREDUCIDO = 0.04;
6
7     public static double aplicarIVA(double precioBase) {
8         return precioBase * (1 + IVA_GENERAL);
9     }
10
11     public static double aplicarIVAReducido(double precioBase ↵
12 ) {

```

```

12     return precioBase * (1 + IVA_REDUCIDO);
13 }
14
15 public static double aplicarIVASuperreducido(double ↵
precioBase) {
16     return precioBase * (1 + IVA_SUPERREDUCIDO);
17 }
18
19 public static double extraerBase(double precioConIVA) {
20     return precioConIVA / (1 + IVA_GENERAL);
21 }
22 }

```

Ejemplo 11.9: Clase CalculadoraImpuestos para generar tests

Parte 1 — Sin IA: Escribe pruebas JUnit 5 para esta clase sin ayuda. Intenta cubrir el caso base, valores límite (precio cero, precio negativo) y la consistencia entre `aplicarIVA` y `extraerBase`.

Parte 2 — Con IA: Pide a la IA que genere pruebas para la misma clase. Ejecuta ambas suites: ¿pasan todos los tests? ¿Hay algún test de la IA que falle? ¿Qué casos cubrió la IA que tú no consideraste?

Resumen

La inteligencia artificial generativa es una herramienta — potente, pero falible. Como cualquier herramienta, su valor depende de quien la usa: un programador que entiende el código, que sabe escribir tests y que evalúa con criterio lo que la IA genera, saca mucho más partido de ella que quien acepta el primer resultado sin verificarlo.

El **marco de las 4D** resume cómo trabajar bien con la IA:

- **Delegación:** decide qué tareas son adecuadas para delegar — las repetitivas y verificables — y cuáles no.
- **Descripción:** comunica con precisión lo que necesitas usando el modelo Producto-Proceso-Rendimiento.
- **Discernimiento:** evalúa críticamente el output; compílalo, pruébalo y entiéndelo antes de aceptarlo.
- **Diligencia:** usa la IA de forma transparente, responsable y sin enviar datos sensibles a servicios externos.

Las pruebas unitarias que aprendiste en los capítulos anteriores son tu principal herramienta de discernimiento. Un programador que sabe definir el comportamiento esperado antes de escribir el código — y que verifica ese comportamiento con tests — es el que mejor aprovecha la IA y el que menos riesgos asume al hacerlo.



Ejercicios

Ejercicios de autoevaluación

12.1. Desarrollo de software

Ejercicio 12.1



1. ¿Qué diferencia hay entre código fuente, código objeto y código ejecutable? Pon un ejemplo del proceso de transformación de uno a otro.
2. ¿En qué se diferencian un compilador y un intérprete? Da un ejemplo de lenguaje que use cada enfoque.
3. Describe los cuatro paradigmas de programación vistos en el capítulo e indica un lenguaje representativo de cada uno.
4. Enumera las fases del desarrollo de una aplicación en orden y describe brevemente el objetivo de cada una.
5. ¿Qué diferencia fundamental existe entre las metodologías ágiles y el modelo en cascada?
6. ¿Cuáles son los tres roles principales de Scrum? ¿Qué hace cada uno?
7. ¿Qué ventajas aporta usar contenedores Docker en el desarrollo de software frente a instalar el software directamente en el sistema operativo?

Ver solución en la página 158.

12.2. Instalación y uso de entornos de desarrollo

Ejercicio 12.2



1. Enumera cinco funciones habituales de un entorno integrado de desarrollo (IDE).
2. ¿Qué diferencia hay entre un IDE y un editor de código como VS Code con extensiones?
3. ¿Para qué sirve un sistema de construcción automática (*build tool*)? Pon dos ejemplos de herramientas y el lenguaje al que están asociadas.
4. ¿Qué es el depurador (*debugger*) y qué es un punto de ruptura (*break-point*)?
5. ¿Qué es Maven y qué estructura de directorios crea por defecto en un proyecto Java?
6. ¿Qué ventaja tiene declarar las dependencias en `pom.xml` frente a incluir los `.jar` manualmente?
7. ¿Qué es la refactorización automática en un IDE? Describe un ejemplo concreto.

Ver solución en la página 159.

12.3. Diseño y realización de pruebas

Ejercicio 12.3



1. ¿Qué diferencia hay entre una prueba funcional y una prueba estructural? Da un ejemplo de cada tipo.
2. ¿Qué son las pruebas de regresión y cuándo se ejecutan?
3. ¿Qué es la cobertura de código? ¿Qué significa tener un 80 % de cobertura de sentencias?
4. ¿Qué son las clases de equivalencia? Aplica el concepto a una función que valida la edad de un usuario (valores aceptados: 0–120).

5. ¿Qué son los valores límite? Indica cuáles serían para el ejemplo anterior.
6. Distingue entre *mock*, *stub* y *spy* como tipos de dobles de prueba.
7. ¿Para qué sirven las anotaciones `@Test`, `@BeforeEach` y `@AfterEach` en JUnit 5?

Ver solución en la página 159.

12.4. Optimización y documentación

Ejercicio 12.4



1. ¿Qué es la refactorización? ¿En qué se diferencia de reescribir el código?
2. Describe el patrón de refactorización *Extract Method* e indica cuándo se aplica.
3. ¿Qué es un analizador estático de código (*linter*)? ¿Qué tipo de problemas detecta? Nombra dos herramientas para Java.
4. ¿Por qué es importante seguir convenciones de nombres en un equipo? ¿Qué convención usa Java para clases, métodos y constantes?
5. ¿Qué es Javadoc y qué etiquetas básicas utiliza?
6. ¿Qué es la observabilidad de una aplicación? Nombra y describe sus tres pilares.

Ver solución en la página 160.

12.5. Herramientas de control de versiones

Ejercicio 12.5



1. ¿Qué problema resuelve un sistema de control de versiones en un equipo de desarrollo?
2. ¿Cuál es la diferencia entre Git (el sistema) y GitHub (el servicio)?
3. ¿Qué información almacena un *commit* en Git además del contenido de los ficheros?
4. ¿Para qué se usan las ramas (*branches*) en Git? ¿Qué es un *merge*?
5. ¿Qué diferencia hay entre `git pull` y `git fetch`?
6. ¿Qué es un conflicto de merge y cómo se resuelve?
7. ¿Qué es la integración continua (CI) y qué ventajas aporta? ¿Qué es GitHub Actions?

Ver solución en la página 161.

12.6. Elaboración de diagramas de clases

Ejercicio 12.6



1. ¿Qué tres compartimentos tiene el rectángulo de una clase en UML y qué contiene cada uno?
2. ¿Qué significan los símbolos `+`, `-` y `#` en un diagrama de clases?
3. ¿Qué diferencia hay entre asociación, agregación y composición? Pon un ejemplo de cada relación.
4. ¿Qué significa la multiplicidad `1..*` en una relación entre clases?
5. ¿Cómo se representa la herencia en UML? ¿Y la realización de una interfaz?
6. ¿Qué es la ingeniería inversa aplicada a diagramas de clases?

Ver solución en la página 162.

12.7. Elaboración de diagramas de comportamiento

Ejercicio 12.7



1. Nombra los cinco tipos de diagramas de comportamiento estudiados e indica en una frase para qué sirve cada uno.
2. ¿Qué diferencia hay entre un diagrama de secuencia y un diagrama de comunicación?
3. ¿Qué es un actor en un diagrama de casos de uso? ¿Puede ser un actor otro sistema software?
4. ¿Para qué se usa la relación *include* entre casos de uso? ¿Y *extend*?
5. ¿Qué es un nodo de decisión en un diagrama de actividad y cómo se representa?
6. ¿Qué diferencia hay entre bifurcación (*fork*) y decisión en un diagrama de actividad?
7. ¿Qué representa una transición en un diagrama de estados? ¿Qué tres elementos puede llevar asociados?

Ver solución en la página 163.

Soluciones

Solución del ejercicio 12.1



1. El **código fuente** es el texto escrito por el programador en un lenguaje de alto nivel (p.ej. `Hola.java`). El **código objeto** es la salida del compilador en un formato intermedio listo para enlazarse o ejecutarse; en Java, `javac` genera ficheros `.class` con *bytecode* de la JVM, no instrucciones nativas de la máquina. El **ejecutable** es el fichero final listo para ejecutarse, que resulta de enlazar el código objeto con las bibliotecas necesarias.
2. Un **compilador** traduce todo el programa antes de ejecutarlo (C, Java). Un **intérprete** traduce y ejecuta línea a línea sin generar un ejecutable previo (Python, JavaScript en el navegador).
3. **Estructurado**: secuencia, selección y bucles (C). **Orientado a objetos**: clases y objetos (Java). **Funcional**: funciones puras, inmutabilidad (Haskell, Clojure). **Lógico**: reglas y hechos, el motor infiere soluciones (Prolog).
4. **Análisis**: qué debe hacer el sistema. **Diseño**: cómo se estructurará. **Codificación**: escribir el código. **Pruebas**: verificar que funciona. **Documentación**: describir el sistema. **Despliegue**: poner en producción. **Mantenimiento**: corregir y mejorar.
5. Las metodologías ágiles trabajan en ciclos cortos (sprints) con entrega continua de valor y adaptación a cambios. El modelo en cascada define todas las fases de forma secuencial y no permite volver atrás fácilmente; el producto final se entrega al final del proyecto.
6. **Product Owner**: define qué construir (gestiona el backlog). **Scrum Master**: elimina impedimentos y vela por el proceso. **Equipo de desarrollo**: construye el producto durante los sprints.
7. Los contenedores encapsulan la aplicación con todas sus dependencias, garantizando que funciona igual en cualquier entorno. Son ligeros, se inician en segundos y permiten tener varios entornos (desarrollo, pruebas, producción) idénticos en la misma máquina.

Solución del ejercicio 12.2



1. Editor de código con resaltado de sintaxis, autocompletado, compilación integrada, depurador, gestor de proyectos y control de versiones integrado (entre otros).
2. Un IDE es una suite completa con compilador, depurador y herramientas integradas de serie (IntelliJ IDEA, VS Code con extensiones Java). Un editor de código como VS Code es más ligero y extensible, pero depende de extensiones para cada funcionalidad; la experiencia puede ser más fragmentada.
3. Automatiza la compilación, gestión de dependencias, ejecución de tests y empaquetado. Ejemplos: **Maven** y **Gradle** (Java), **npm/Vite** (JavaScript).
4. El **depurador** permite ejecutar el programa paso a paso e inspeccionar el valor de las variables en cada momento. Un **punto de ruptura** es una marca en una línea concreta donde la ejecución se detiene para que el programador pueda inspeccionar el estado.
5. Maven es una herramienta de construcción para Java. Su estructura por defecto es: `src/main/java` (código fuente), `src/main/resources` (recursos), `src/test/java` (tests) y `target` (compilados y empaquetados).
6. Maven descarga automáticamente las dependencias y sus dependencias transitivas desde repositorios centrales, garantiza versiones consistentes en todo el equipo y facilita la actualización de bibliotecas.
7. Es una transformación del código que no cambia su comportamiento pero mejora su estructura. Ejemplo: *Rename* cambia automáticamente el nombre de una variable o método en todos los sitios donde se usa, sin riesgo de errores manuales.

Solución del ejercicio 12.3



1. Las **funcionales** (caja negra) verifican que el sistema cumple los requisitos sin conocer el código interno (p.ej. comprobar que el login acepta credenciales correctas). Las **estructurales** (caja blanca) prueban la lógica interna del código (p.ej. verificar que se recorre cada rama de un `if`).

2. Las pruebas de **regresión** comprueban que los cambios nuevos no han roto funcionalidad ya existente. Se ejecutan habitualmente tras cada cambio en el código, idealmente de forma automática en el pipeline de CI.
3. La **cobertura de código** mide qué proporción del código es ejecutada por los tests. Un 80 % de cobertura de sentencias significa que 8 de cada 10 líneas de código fuente son ejecutadas durante la suite de pruebas; el 20 % restante no está cubierto.
4. Las **clases de equivalencia** agrupan entradas que deben comportarse igual. Para edad 0–120: clase válida (0–120), clase inválida por debajo (< 0) y clase inválida por encima (> 120). Basta con probar un valor representativo de cada clase.
5. Los **valores límite** son los extremos de cada clase: -1 , 0 , 1 , 119 , 120 , 121 .
6. Un **stub** devuelve respuestas predefinidas sin lógica real. Un **mock** además verifica que se llamó a sus métodos con los parámetros esperados. Un **spy** envuelve un objeto real y permite observar sus interacciones sin reemplazarlo del todo.
7. **@Test** marca un método como caso de prueba. **@BeforeEach** se ejecuta antes de cada test (para preparar el estado inicial). **@AfterEach** se ejecuta después de cada test (para limpiar recursos).

Solución del ejercicio 12.4



1. La **refactorización** transforma el código para mejorar su estructura interna sin cambiar su comportamiento observable. A diferencia de reescribir, se hace en pasos pequeños y seguros, apoyados por tests que garantizan que nada se rompe.
2. *Extract Method* extrae un fragmento de código repetido o demasiado largo en un método propio con nombre descriptivo. Se aplica cuando un método hace demasiadas cosas o cuando el mismo bloque aparece en varios sitios.
3. Un **analizador estático** examina el código sin ejecutarlo y detecta errores potenciales, violaciones de estilo, complejidad excesiva, código muerto o vulnerabilidades de seguridad. Para Java: **Checkstyle**

(estilo) y **PMD** o **SpotBugs** (bugs y malas prácticas).

4. Las convenciones evitan ambigüedades y hacen el código predecible para cualquier miembro del equipo. En Java: **clases** en PascalCase (MiClase), **métodos y variables** en camelCase (miMetodo), **constantes** en UPPER_SNAKE_CASE (MAX_INTENTOS).
5. **Javadoc** genera documentación HTML a partir de comentarios especiales `/** ... */`. Etiquetas básicas: `@param` (describe un parámetro), `@return` (describe el valor de retorno), `@throws` (documenta las excepciones que puede lanzar).
6. La **observabilidad** es la capacidad de entender el estado interno de un sistema a partir de sus salidas. Sus tres pilares son: **métricas** (valores numéricos agregados: latencia, tasa de error), **logs** (registros de eventos con contexto) y **trazas** (seguimiento de una petición a través de varios servicios).

Solución del ejercicio 12.5



1. Permite registrar el historial completo de cambios del código, revertir a versiones anteriores, trabajar en paralelo sin pisarse y fusionar los cambios de varios desarrolladores de forma controlada.
2. **Git** es el sistema de control de versiones distribuido (se ejecuta localmente). **GitHub** es una plataforma web que aloja repositorios Git remotos y añade funcionalidades de colaboración: pull requests, issues, Actions, etc.
3. Un commit almacena: el identificador (hash SHA-1), el mensaje descriptivo, el autor, la fecha y la referencia al commit anterior (padre). Es un snapshot completo del estado del proyecto.
4. Las **ramas** permiten desarrollar funcionalidades o corregir errores de forma aislada sin afectar a la rama principal. Un **merge** integra los cambios de una rama en otra, unificando los historiales.
5. `git fetch` descarga los cambios del repositorio remoto pero no los aplica al directorio de trabajo. `git pull` hace `fetch` y además fusiona automáticamente los cambios en la rama local actual.
6. Un **conflicto de merge** ocurre cuando dos ramas han modificado las

mismas líneas de un fichero de forma incompatible. Git marca el conflicto en el fichero con delimitadores (<<<<<<, =====, >>>>>>). El desarrollador debe editar el fichero manualmente para resolver la diferencia, añadirlo al staging y completar el merge con un nuevo commit.

7. La **integración continua** automatiza la compilación y ejecución de tests en cada push, detectando errores rápidamente. **GitHub Actions** es el servicio de CI/CD de GitHub: se configura mediante ficheros YAML en `.github/workflows/` que definen *triggers*, *jobs* y *steps*.

Solución del ejercicio 12.6



1. **Nombre** (superior): nombre de la clase. **Atributos** (central): las propiedades con su tipo y visibilidad. **Métodos** (inferior): las operaciones con sus parámetros, tipo de retorno y visibilidad.
2. + indica visibilidad **pública** (accesible desde cualquier clase). - indica visibilidad **privada** (solo accesible desde la propia clase). # indica visibilidad **protegida** (accesible desde la clase y sus subclases).
3. **Asociación**: relación genérica entre clases (un **Profesor** imparte a un **Alumno**). **Agregación**: contención débil; el objeto contenido puede existir sin el contenedor (un **Departamento** tiene **Profesores**, que siguen existiendo si el departamento desaparece). **Composición**: contención fuerte; el objeto contenido no existe sin el contenedor (un **Pedido** está compuesto de **Líneas**, que desaparecen con el pedido).
4. 1..* significa «uno o más»: un objeto de la primera clase se relaciona con al menos uno y posiblemente varios objetos de la segunda.
5. La **herencia** se representa con una línea sólida y una flecha triangular vacía apuntando a la superclase. La **realización de interfaz** se representa con una línea discontinua y una flecha triangular vacía apuntando a la interfaz.
6. La **ingeniería inversa** genera automáticamente un diagrama de clases a partir del código fuente existente, permitiendo visualizar y documentar la arquitectura de un sistema ya implementado.

Solución del ejercicio 12.7

1. **Casos de uso:** qué funcionalidades ofrece el sistema a sus actores.
Secuencia: cómo interactúan los objetos en el tiempo mediante mensajes. **Comunicación:** qué objetos se relacionan y qué mensajes intercambian (sin eje temporal). **Actividad:** cómo fluye un proceso con decisiones y paralelismo. **Estados:** qué estados tiene un objeto y cómo transita entre ellos.
2. Ambos muestran la misma interacción. El **de secuencia** usa un eje vertical de tiempo y enfatiza el orden de los mensajes. El **de comunicación** usa un grafo de objetos con mensajes numerados sobre los enlaces y enfatiza las relaciones estructurales entre objetos.
3. Un **actor** es cualquier entidad externa que interactúa con el sistema: puede ser una persona, otro sistema software o un dispositivo. Sí puede ser otro sistema (p.ej. un servicio de pago externo).
4. **Include:** el caso de uso base incorpora siempre el comportamiento del caso incluido (relación obligatoria). **Extend:** el caso de uso extendido añade comportamiento opcional al base solo bajo ciertas condiciones.
5. Un **nodo de decisión** es un punto donde el flujo se bifurca según una condición. Se representa con un rombo del que parten varias transiciones etiquetadas con las condiciones de guarda entre corchetes.
6. La **decisión** escoge uno de varios caminos alternativos (solo uno se ejecuta). La **bifurcación** divide el flujo en varios hilos que se ejecutan en paralelo simultáneamente; una unión posterior los resincroniza.
7. Una **transición** es el cambio de un estado a otro. Puede llevar: el **evento** que la desencadena, una **condición de guarda** entre corchetes que debe cumplirse, y una **acción** que se ejecuta durante la transición.

13

Ejercicios prácticos

13.1. Desarrollo de software

Ejercicio 13.1



Una empresa de logística quiere desarrollar una aplicación para gestionar el seguimiento de paquetes en tiempo real. El sistema debe permitir a los repartidores registrar la localización de cada paquete mediante la aplicación móvil, y a los clientes consultar el estado de su envío desde la web.

1. Identifica al menos cinco requisitos funcionales y dos no funcionales del sistema.
2. Elige una metodología de desarrollo (Scrum, Kanban, XP o cascada) y justifica tu elección en función de las características del proyecto.
3. Elabora una planificación inicial indicando las fases del desarrollo, su orden y una estimación de duración relativa de cada una.
4. ¿Qué paradigma de programación consideras más adecuado para implementar el backend? Razona la respuesta.

13.2. Instalación y uso de entornos de desarrollo

Ejercicio 13.2



Configura un proyecto Java con Maven en IntelliJ IDEA (o VS Code con extensión Java) siguiendo estos pasos:

1. Crea un nuevo proyecto Maven con el *archetype* `maven-archetype-quickstart`. Usa el *groupId* `es.fp.entornos` y el *artifactId*

calculadora.

2. Implementa una clase `Calculadora` con los métodos `sumar`, `restar`, `multiplicar` y `dividir`. El método `dividir` debe lanzar una excepción `ArithmeticException` si el divisor es cero.
3. Añade la dependencia de JUnit 5 al `pom.xml` y crea al menos dos tests por método, cubriendo el caso normal y los casos límite.
4. Añade el plugin `maven-checkstyle-plugin` al `pom.xml` y configúralo para que el build falle si hay más de 5 violaciones de estilo. Ejecuta `mvn checkstyle:check` y corrige los avisos encontrados.
5. Crea un fichero `.github/workflows/ci.yml` que compile el proyecto y ejecute los tests automáticamente en cada push.

13.3. Pruebas unitarias

Ejercicio 13.3



Crea todos los tests unitarios necesarios para implementar una función en Java que reciba una URL como cadena y la divida en sus componentes: protocolo, dominio y puerto. Considera casos como URLs con y sin puerto, diferentes protocolos (`http`, `https`, `ftp`), y URLs mal formadas. Especifica los casos de prueba y los resultados esperados.

- **Entrada:** `http://example.com:8080`
 - **Salida esperada:** protocolo = `http`, dominio = `example.com`, puerto = `8080`
- **Entrada:** `https://www.test.org`
 - **Salida esperada:** protocolo = `https`, dominio = `www.test.org`, puerto = (vacío)
- **Entrada:** `ftp://ftp.server.net:21`
 - **Salida esperada:** protocolo = `ftp`, dominio = `ftp.server.net`, puerto = `21`
- **Entrada:** `http://localhost`

- **Salida esperada:** protocolo = http, dominio = localhost, puerto = (vacío)
- **Entrada:** malformada-url
 - **Salida esperada:** Error o valores nulos indicando URL mal formada.

13.4. Optimización y documentación

Ejercicio 13.4



A continuación se muestra una clase Java con varios problemas de calidad. Aplica al menos cuatro patrones de refactorización distintos para mejorarla, documenta cada cambio indicando el patrón aplicado y justifica por qué mejora el código.

```

1 public class GestorPedidos {
2     public double calcular(int t, double p, int c, boolean b) ↩
3     {
4         double r = p * c;
5         if (b == true) {
6             if (t == 1) { r = r - (r * 0.1); }
7             if (t == 2) { r = r - (r * 0.15); }
8             if (t == 3) { r = r - (r * 0.2); }
9         }
10        double iva = r * 0.21;
11        double total = r + iva;
12        System.out.println("Total: " + total);
13        return total;
14    }
15 }
```

Aspectos a considerar: nombres poco descriptivos, comparación innecesaria con `true`, constantes mágicas (0.1, 0.15, 0.2, 0.21), mezcla de cálculo con presentación, y estructura condicional repetitiva.

13.5. Control de versiones

Ejercicio 13.5



Simula un flujo de trabajo colaborativo con Git. Puedes trabajar en local con dos clones del mismo repositorio para simular dos desarrolladores.

1. Crea un repositorio Git con un proyecto Java básico. Realiza un commit inicial con `README.md` y la estructura del proyecto.
2. Crea una rama `feature/login` y añade en ella una clase `LoginService` con un método `autenticar(String usuario, String clave)`.
3. Desde la rama `main`, añade al `README.md` una sección «Instrucciones de instalación» y haz commit.
4. Fusiona la rama `feature/login` en `main`. Si hay conflictos, resuélvelos manualmente y documenta qué decisión has tomado y por qué.
5. Crea una etiqueta (*tag*) `v1.0` en el commit resultante del merge.
6. Escribe un fichero `.github/workflows/ci.yml` que compile el proyecto con Maven y ejecute los tests en cada push a `main`.

13.6. Diagramas de clases

Ejercicio 13.6



Diseña el diagrama de clases UML de una plataforma de streaming de vídeo con las siguientes características:

- Un **Usuario** tiene nombre, email y contraseña. Puede tener una **Suscripción** activa (con fecha de inicio, fecha de fin y tipo: básica, estándar o premium).
- Un usuario puede crear varias **ListasReproducción**, cada una con un nombre y una lista ordenada de **Contenidos**.
- **Contenido** es una clase abstracta con título, descripción y duración. De ella heredan **Película** (con género y director) y **Serie** (con número de temporadas).

- Una **Serie** está compuesta por varios **Episodios**, cada uno con número, título y duración.
- El sistema tiene un **Catálogo** que agrupa todos los contenidos disponibles.

Indica en el diagrama: visibilidad de atributos y métodos, tipos de relaciones con su multiplicidad, y al menos un método relevante en cada clase. Genera el diagrama con PlantUML y adjunta el código `.puml`.

13.7. Diagramas de comportamiento

Ejercicio 13.7



Modela el proceso de compra en una tienda online utilizando los tres tipos de diagrama siguientes. Usa PlantUML para generarlos y adjunta el código `.puml` de cada uno.

1. **Diagrama de casos de uso:** identifica los actores (cliente, sistema de pagos, almacén) y los casos de uso principales (buscar producto, añadir al carrito, realizar pago, gestionar envío). Incluye al menos una relación *include* y una *extend*.
2. **Diagrama de actividad:** modela el flujo completo desde que el cliente inicia el proceso de pago hasta que recibe la confirmación del pedido. Incluye al menos una decisión (por ejemplo, si el pago es aceptado o rechazado) y una bifurcación paralela (por ejemplo, enviar email de confirmación y notificar al almacén simultáneamente).
3. **Diagrama de estados:** modela los estados por los que pasa un **Pedido** desde su creación hasta su entrega o cancelación. Define al menos cinco estados y todas las transiciones posibles con sus eventos.